

**АКАДЕМИЈА ТЕХНИЧКО-УМЕТНИЧКИХ СТРУКОВНИХ СТУДИЈА
БЕОГРАД
ОДСЕК ВИСОКА ШКОЛА ЕЛЕКТРОТЕХНИКЕ И РАЧУНАРСТВА**

Алекса Стојановић

**Пројектовање и реализација релационог модела базе
података за управљање и анализу пословања Ауто школе
“AS” у MS SQL сервер систему**

- завршни рад –



Београд, септембар 2026.

Кандидат: **Алекса Стојановић**

Број индекса: **НРТ-4/22**

Студијски програм: **Нове рачунарске технологије**

Тема: **Аутошкола – пројектовање и имплементација релационе базе података у MS SQL Server окружењу**

Основни задаци:

1. **Теоријски концепти релационих база података и опис коришћених технологија**
2. **Пројектовање релационог модела базе података.**
3. **Реализација релационог модела базе података за управљање и анализу пословања Ауто школе “AS” у MS SQL сервер систему.**

Ментор:

Београд, септембар 2026. године.

др Габријела Димић, виши предавач
АТУСС, Одсек ВИШЕР

РЕЗИМЕ:

Циљ овог завршног рада је пројектовање и имплементација базе података намењене подршци пословању аутошколе. База обухвата кључне податке потребне за праћење процеса уписа кандидата, организације обуке, евиденције часова, полагања и финансијског дела пословања. На тај начин омогућено је лакше вођење евиденције, боља повезаност података и једноставнији приступ информацијама важним за свакодневни рад аутошколе. У раду су приказани анализа захтева, израда модела базе података и њена реализација у изабраном окружењу. Поред тога, описани су основни делови система, начин њиховог повезивања, као и провера рада кроз тест податке и примере упита. Резултат рада је база података која представља основу за даљи развој информационог система аутошколе.

Кључне речи: релациона база података, аутошкола, MS SQL Server, T-SQL, оптимизација упита.

ABSTRACT:

The aim of this final paper is to design and implement a database intended to support the business operations of a driving school. The database covers the key data needed to track candidate enrollment, training organization, lesson records, exams, and the financial part of the business. In this way, it enables easier record keeping, better data connectivity, and simpler access to information important for the daily work of a driving school. The paper presents requirements analysis, database modeling, and implementation in the selected environment. In addition, it describes the main parts of the system, the way they are connected, and the verification of the solution through test data and sample queries. The result of the paper is a database that represents a foundation for further development of a driving school information system.

Keywords: relational database, driving school, MS SQL Server, T-SQL, query optimization.

Садржај:

1. УВОД	1
2. ФАЗЕ РАЗВОЈА БАЗЕ ПОДАТАКА	2
2.1. Прикупљање и анализа захтева	2
2.2. Пројектовање, односно моделовање	2
2.3. Имплементација	2
2.4. Тестирање	2
2.5. Одржавање	3
3. ТЕХНОЛОГИЈЕ КОРИШЋЕНЕ У РАДУ	4
3.1. SQL и T-SQL	4
3.2. MS SQL Server	4
3.3. SQL Server Management Studio	5
3.4. SAP PowerDesigner	5
4. ТЕОРИЈСКЕ ОСНОВЕ БАЗЕ ПОДАТАКА	6
4.1. Основни концепти модела података	6
4.1.1. Релациони и нерелациони модел података	6
4.1.2. Основе релационог модела	6
4.1.3. Ентитети, атрибути, релације	6
4.2. Основне групе SQL наредби	7
4.3. Структура и ограничења релационе базе	8
4.3.1. Примарни и страни кључеви	8
4.3.2. Интегритет података	8
4.4. Организација и оптимизација базе података	9
4.4.1. Нормализација	9
4.4.2. Индекси	9
4.4.3. Трансакције	10
4.4.4. Одабрани објекти базе података	10
5. АНАЛИЗА ЗАХТЕВА СИСТЕМА	12
5.1. Опис проблема	12
5.2. Корисничке улоге	12
5.3. Захтеви система	12
5.3.1. Основне могућности система	13
5.3.2. Квалитет и правила рада система	13
6. ПРОЈЕКТОВАЊЕ БАЗЕ ПОДАТАКА	15
6.1. Концептуални модел	15
6.1.1. Идентификација ентитета	15
6.1.2. Везе између ентитета	16

6.2.	Логички модел	16
6.2.1.	Превођење концептуалних ентитета у логички модел	17
6.2.2.	Дефинисање примарних, алтернативних и страних идентификатора	17
6.2.3.	Кардиналности	18
6.3.	Физички модел	18
6.3.1.	Типови података	19
6.3.2.	Ограничења над колонама	19
7.	ИМПЛЕМЕНТАЦИЈА БАЗЕ ПОДАТАКА	20
7.1.	Креирање базе података	20
7.2.	Креирање табела	20
7.2.1.	Kandidat	21
7.2.2.	Zaposleni	22
7.2.3.	Funkcije_Zaposlenih	23
7.2.4.	Zaposleni_Funkcija	23
7.2.5.	Zaposleni_Izostanak	23
7.2.6.	Kategorija_vozacke	24
7.2.7.	Tip_obuke	24
7.2.8.	Grupa	24
7.2.9.	Obuka	25
7.2.10.	Kandidat_grupa	25
7.2.11.	Vozilo	26
7.2.12.	Polaganje	26
7.2.13.	Polaganje_kandidat	27
7.2.14.	Nadzornici_polaganja	27
7.2.15.	Cas	28
7.2.16.	Cenovnik	29
7.2.17.	Uplata	29
7.3.	Дефинисање веза између табела	30
7.3.1.	Zaposleni - Zaposleni_Funkcija	30
7.3.2.	Funkcije_Zaposlenih - Zaposleni_Funkcija	30
7.3.3.	Zaposleni - Zaposleni_Izostanak	31
7.3.4.	Zaposleni - Nadzornici_polaganja	31
7.3.5.	Polaganje - Nadzornici_polaganja	32
7.3.6.	Kandidat - Obuka	32
7.3.7.	Kategorija_vozacke - Obuka	33
7.3.8.	Tip_obuke – Obuka	33
7.3.9.	Zaposleni - Obuka	34
7.3.10.	Grupa - Kandidat_grupa	34
7.3.11.	Obuka - Kandidat_grupa	35
7.3.12.	Kategorija_vozacke - Vozilo	35
7.3.13.	Zaposleni - Cas	35
7.3.14.	Tip_obuke - Cas	36
7.3.15.	Obuka - Cas	37
7.3.16.	Vozilo - Cas	37
7.3.17.	Grupa - Cas	38
7.3.18.	Tip_obuke - Polaganje	38
7.3.19.	Polaganje - Polaganje_kandidat	39
7.3.20.	Obuka - Polaganje_kandidat	39

7.3.21.	Cenovnik - Uplata	39
7.3.22.	Obuka - Uplata	40
7.4.	Унос тест података	40
7.5.	Погледи	41
7.5.1.	vw_Kandidati_Obuke	41
7.5.2.	vw_Raspored_Casova	42
7.5.3.	vw_Uplate_Po_Obuci	43
7.5.4.	vw_Rezultati_Polaganja	44
7.5.5.	vw_Zaposleni_Funkcije	44
7.5.6.	vw_Stanje_Obuke	45
7.6.	Функције	46
7.6.1.	fn_UkupnoUplacenoZaObuku	46
7.6.2.	fn_BrojCasovaZaObuku	47
7.6.3.	fn_BrojPolaganjaZaObuku	47
7.6.4.	fn_KandidatImaAktivnuObuku	48
7.6.5.	fn_RasporedInstruktora	49
7.7.	Процедуре и трансакције	50
7.7.1.	usp_UpisKandidataNaObuku	50
7.7.2.	usp_ZakaziPrakticniCas	51
7.7.3.	usp_EvidentirajUplatu	52
7.7.4.	usp_PrijaviKandidataNaPolaganje	53
7.7.5.	usp_EvidentirajRezultatPolaganja	54
7.8.	Тригери	55
7.8.1.	Аутоматско ажурирање поља Izmenjen_datum	55
7.8.2.	trg_Uplata_ZabraniBrisanje	55
7.8.3.	trg_Vozilo_Kilometraza_NeSmanjuje	56
7.8.4.	trg_Cas_Odrzan_PovecajKilometrazu	56
7.8.5.	trg_PolaganjeKandidat_Uspesno_ZavrshiObuku	57
7.8.6.	trg_Obuka_PromenaInstruktora_PrebaciBudućeCasove	57
8.	ОПТИМИЗАЦИЈА И АНАЛИЗА ПЕРФОРМАНСИ	58
8.1.	План извршавања	58
8.2.	Избор индекса за анализу	59
8.2.1.	Возило по датуму	59
8.2.2.	Часови по кандидату	60
8.2.3.	Обуке по кандидату и категорији	62
8.2.4.	Уплате по обуци	63
8.2.5.	Уплате по датуму	65
8.2.6.	Полагања по кандидату	66
8.2.7.	Кандидати по групи	67
8.2.8.	Запослени по функцији	69
8.2.9.	Распоред часова	70
9.	СМЕРНИЦЕ ЗА УНАПРЕЂЕЊЕ ОПИСАНОГ СИСТЕМА	72
9.1.	Корисничка апликација и пријава корисника	72
9.2.	Корисничке улоге и права приступа	72
9.3.	Заштита личних података	72

9.4.	Резервне копије и опоравак базе _____	72
9.5.	Додатни извештаји _____	73
9.6.	Проширење модела базе података _____	73
10.	ЗАКЉУЧАК _____	74
11.	ИНДЕКС ПОЈМОВА _____	75
12.	ЛИТЕРАТУРА _____	76
13.	СПИСАК СЛИКА _____	77
14.	СПИСАК ПРИЛОГА _____	79
15.	ИЗЈАВА О АКАДЕМСКОЈ ЧЕСТИТОСТИ _____	81

1. УВОД

У савременом пословању све је већа потреба за поузданим, уређеним и брзо доступним подацима. Ово је посебно важно у системима у којима се свакодневно обрађује велики број међусобно повезаних информација. Аутошкола, као организација која прати кандидате од уписа до завршетка обуке и полагања, представља пример система у ком ручно вођење евиденције или коришћење неповезаних докумената може довести до грешака, дуплирања података и отежаног извештавања.

Предмет овог рада је пројектовање и имплементација релационе базе података за потребе аутошколе. База је осмишљена тако да обједини податке о кандидатима, запосленима, функцијама запослених, обукама, часовима, возилима, групама, полагањима, ценовнику и уплатама. На тај начин се подаци не посматрају као изоловане целине, већ као повезан систем који прати стварни ток пословања аутошколе.

Циљ рада је да се покаже како се један реалан пословни процес може превести у модел релационе базе података. Кроз овај приступ могуће је боље разумети како се подаци организују, како се између њих успостављају везе и како база може да помогне у свакодневном раду организације.

Резултат рада треба да буде база података која представља основу за даљи развој информационог система аутошколе. Таква база може се касније проширити корисничком апликацијом и другим функционалностима које би омогућиле још ефикасније пословање.

2. ФАЗЕ РАЗВОЈА БАЗЕ ПОДАТАКА

Развој базе података представља поступак у ком се пословни проблем постепено преводи у уређен и употребљив информациони систем. Тај поступак се не своди само на писање SQL наредби, већ обухвата разумевање захтева, пројектовање модела, имплементацију, проверу исправности и касније одржавање базе. У овом раду тај ток је примењен на пословање аутошколе, где је потребно повезати податке о кандидатима, запосленима, обукама, часовима, возилима, полагањима и уплатама. [1]

2.1. Прикупљање и анализа захтева

Прва фаза развоја базе података односи се на прикупљање и анализу захтева. У овој фази се утврђује које податке систем треба да чува, ко ће те податке користити и која правила морају да важе у свакодневном раду. Код система аутошколе то значи да је потребно сагледати цео ток пословања, од уписа кандидата до завршетка обуке, полагања и финансијске евиденције.

На основу анализе захтева издвајају се главне целине система. Поред самих података, у овој фази се уочавају и пословна правила, као што су јединственост ЈМБГ-а, повезаност табела.

2.2. Пројектовање, односно моделовање

Након анализе захтева следи пројектовање базе података. Циљ ове фазе је да се пословни захтеви прикажу кроз модел који јасно показује структуру података и њихове међусобне везе. Пројектовање се најчешће посматра кроз концептуални, логички и физички модел, при чему сваки наредни модел садржи више техничких детаља. [1]

2.3. Имплементација

Имплементација представља фазу у којој се пројектовани модел претвара у физичку базу података. У овој фази креирају се табеле, дефинишу примарни и страни кључеви, додају ограничења, подразумеване вредности и остали објекти базе података. [2] [3]

Поред основних табела, у имплементацији су креирани и погледи, функције, процедуре са трансакцијама, тригери и индекси. Ови објекти омогућавају да база не буде само место за чување података, већ и део система који контролише исправност података, поједностављује читање информација и подржава важне пословне операције.

2.4. Тестирање

Тестирање базе података служи да се провери да ли креирани модел и SQL објекти исправно подржавају постављене захтеве. У овој фази се база попуњава пробним подацима и проверавају се различити сценарији коришћења, као што су упис кандидата, креирање обуке, заказивање часова, евиденција полагања и унос уплата.

Поред провере да ли се упити извршавају без грешке, важно је проверити и да ли ограничења, процедуре и тригери спречавају неисправне уносе. На пример, тестира се да ли систем дозвољава дупли ЈМБГ, да ли се страна кључна поља

правилно повезују и да ли трансакције враћају податке у претходно стање ако дође до грешке у току извршавања.

2.5. Одржавање

Одржавање базе података почиње након њене прве имплементације и наставља се током целог животног века система. Како се пословање мења, могу се јавити нови захтеви, нове врсте извештаја или потреба за изменом постојећих правила. Због тога је важно да структура базе буде јасна и да су објекти организовани тако да се касније могу лако проширивати.

У одржавање спадају и задаци као што су праћење перформанси, анализа упита, додавање или измена индекса и провера да ли база и даље чува податке на исправан начин. Код система аутошколе то је посебно важно јер се подаци стално допуњују новим кандидатима, часовима, полагањима и финансијским евиденцијама.

3. ТЕХНОЛОГИЈЕ КОРИШЋЕНЕ У РАДУ

За израду рада коришћене су технологије које покривају две главне целине: пројектовање базе података и њену имплементацију. За моделовање је коришћен SAP PowerDesigner, док су за физичку реализацију базе коришћени T-SQL, MS SQL Server и SQL Server Management Studio.

3.1. SQL и T-SQL

SQL је стандардизовани декларативни језик за рад са релационим базама података. Настао је седамдесетих година двадесетог века, у оквиру развоја релационог модела података, а временом је постао основни језик за комуникацију са системима за управљање базама података. То што је SQL декларативни језик значи да корисник описује шта жели да добије из базе, а не пише корак по корак како база треба да дође до резултата. Сам систем за управљање базом одлучује на који начин ће захтев бити извршен.

У MS SQL Server окружењу користи се Transact-SQL, односно Microsoft-ова варијанта SQL језика. T-SQL задржава основне SQL наредбе за креирање, измену и читање података, али додаје могућности за писање сложеније логике унутар базе. То подразумева употребу променљивих, условних наредби, процедура, функција, тригера, трансакција и обраду грешака. Због тога се у овом раду T-SQL користи у деловима базе који, поред основног рада са подацима, садрже и пословну логику. [3]



Слика 3.1 T-SQL логотип.

Извор: educba.com

3.2. MS SQL Server

MS SQL Server је систем за управљање релационим базама података, односно серверски део који физички чува податке и извршава SQL упите. За разлику од SQL Server Management Studio алата, који служи као графичко окружење за рад са базом, SQL Server представља сам механизам базе података. Он управља табелама, индексима, трансакцијама, ограничењима интегритета, правима приступа и начином извршавања упита. [2]

Поред SQL Server-а постоје и други системи за управљање релационим базама података, као што су MySQL, PostgreSQL и Oracle Database. Ови системи такође могу да подрже објекте као што су погледи, процедуре, функције, тригери, трансакције и индекси, али се разликују по синтакси, алатима и начину анализе извршавања упита. У овом раду је изабран MS SQL Server зато што у комбинацији са SQL Server Management Studio алатом омогућава једноставну реализацију и приказ свих захтева рада, укључујући T-SQL објекте, трансакције, индексе и анализу перформанси преко плана извршавања (engl. Execution Plan).



Слика 3.2 MS SQL Server 2025 логотип.

Извор: mychoicesoftware.com

3.3. SQL Server Management Studio

SQL Server Management Studio, односно SSMS, представља радно окружење преко ког се корисник повезује на MS SQL Server и управља базом података. SSMS сам не чува податке и не извршава упите, већ служи као алат преко ког се SQL наредбе шаљу SQL Server-у, а добијени резултати приказују кориснику. [4]



SSMS је у овом раду изабран зато што обједињује више корисних алата потребних за израду и проверу базе података:

Слика 3.3 SQL Server Management Studio (SSMS).

- Query Editor - омогућава писање и покретање SQL скрипти
- Object Explorer - приказује структуру сервера и базе података
- Results и Messages прозори - омогућавају преглед резултата и порука након извршавања упита
- Execution Plan - приказује начин на који SQL Server извршава упит и помаже у одлуци где је потребно додати индекс или изменити упит. [4]

3.4. SAP PowerDesigner

SAP PowerDesigner је алат за пројектовање и моделовање база података. У овом раду је коришћен зато што омогућава да се база података најпре прикаже кроз моделе, пре него што се крене у њену физичку имплементацију у SQL Server-у. На тај начин је било могуће јасно приказати табеле, атрибуте, примарне и стране кључеве, као и везе између различитих делова система. [5]



Слика 3.4 SAP PowerDesigner логотип.

Извор: powerdesigner.biz

PowerDesigner је изабран зато што физички модел у њему не служи само као слика структуре базе, већ може да буде основа за генерисање SQL кода. Када се у физичком моделу дефинишу табеле, атрибути, типови података, примарни и страни кључеви, PowerDesigner може на основу тог модела да формира SQL скрипту за креирање базе. То је значајно јер омогућава да моделовање и имплементација буду повезани, а не два потпуно одвојена корака.

4. ТЕОРИЈСКЕ ОСНОВЕ БАЗЕ ПОДАТАКА

Теоријске основе база података објашњавају појмове који су неопходни за разумевање пројектованог система. У овом поглављу описани су основни концепти модела података, структура релационе базе, ограничења интегритета, нормализација, индекси и трансакције. Ови појмови су важни зато што директно утичу на начин на који је база аутошколе пројектована и имплементирана.

4.1. Основни концепти модела података

Модел података представља начин да се подаци из реалног система опишу кроз јасну структуру. Он помаже да се препознају битне целине, њихова својства и односи између њих. У систему аутошколе модел података омогућава да се пословне целине, као што су кандидат, обука, час, возило и уплата, прикажу као повезани делови једне базе.

4.1.1. Релациони и нерелациони модел података

Релациони модел података заснива се на табелама, редовима и колонама. Свака табела представља једну врсту података, док се везе између табела успостављају помоћу кључева. Овај модел је погодан када су подаци добро структурирани и када је важно очувати тачност веза између њих. [6] [7]

Нерелациони модели података користе другачије начине чувања података, као што су документи, парови кључ-вредност или графови. Они могу бити погодни за системе у којима се структура података често мења или није строго дефинисана. За систем аутошколе релациони модел је природнији избор, јер постоје јасне целине и строга правила повезивања података.

4.1.2. Основе релационог модела

У релационом моделу подаци се чувају у табелама. Табела има колоне које описују особине података и редове који представљају конкретне записе. На пример, табела Кандидат може да садржи колоне за име, презиме, ЈМБГ, телефон и адресу електронске поште, док сваки ред представља једног кандидата. [6]

Важна особина релационог модела је могућност повезивања табела. Уместо да се исти подаци понављају на више места, табеле се повезују преко примарних и страних кључева. Тако се, на пример, обука повезује са кандидатом, категоријом возачке дозволе и инструктором, док се уплата повезује са конкретном обуком.

4.1.3. Ентитети, атрибути, релације

Ентитет представља објекат или појаву из реалног система о којој је потребно чувати податке. У систему аутошколе ентитети су, на пример, кандидат, запослени, обука, час, возило, полагање и уплата. Препознавање ентитета је први корак у одређивању табела које ће постојати у бази.

Атрибути представљају особине ентитета. На пример, кандидат може имати атрибуте као што су име, презиме, ЈМБГ, телефон и адреса електронске поште, док возило може имати регистрацију, марку, модел и годиште. На основу атрибута се касније дефинишу колоне у табелама.

Релације описују начин на који су ентитети међусобно повезани. У релационом моделу најчешће се јављају следеће врсте веза:

- Веза један-према-један (1:1) значи да једном запису из једне табеле одговара највише један запис из друге табеле.
- Веза један-према-више (1:N) значи да један запис из једне табеле може бити повезан са више записа из друге табеле.
- Веза више-према-више (N:M) значи да више записа из једне табеле може бити повезано са више записа из друге табеле. У релационој бази ова врста везе најчешће се реализује помоћном табелом.

Одређивање врсте релације важно је зато што утиче на начин моделовања базе. Код везе 1:N страни кључ се најчешће поставља у табелу која се налази на страни „више”. Код везе N:M потребно је увести помоћну табелу која повезује две главне табеле. На тај начин се већ у фази моделовања одређује како ће подаци бити повезани и како ће касније бити реализовани у физичком моделу базе.

4.2. Основне групе SQL наредби

SQL наредбе су инструкције које корисник задаје систему за управљање базом података. Помоћу њих се дефинише структура базе, **уносе** и **мењају** подаци, **читају** подаци из табела и управља радом базе. Према намени, SQL наредбе се могу поделити у неколико група:

- **DDL (Data Definition Language)** обухвата наредбе које се користе за дефинисање структуре базе података. У ову групу спадају наредбе као што су CREATE, ALTER и DROP. Помоћу њих се креирају и мењају табеле, ограничења, индекси и други објекти базе.
- **DML (Data Manipulation Language)** обухвата наредбе које се користе за рад са подацима унутар већ дефинисаних табела. У ову групу спадају INSERT, UPDATE и DELETE. Ове наредбе омогућавају унос нових података, измену постојећих података и брисање података из табеле.
- **DQL (Data Query Language)** односи се на наредбе за читање података из базе. Основна наредба је SELECT. Помоћу SELECT наредбе могуће је добити податке из једне или више табела, приказати само одређене колоне и комбиновати податке који су међусобно повезани.
- **DCL (Data Control Language)** обухвата наредбе за управљање правима приступа, као што су GRANT и REVOKE.
- **TCL (Transaction Control Language)** обухвата наредбе за управљање трансакцијама, као што су COMMIT и ROLLBACK.

Клаузуле су делови SQL упита којима се прецизније одређује како се подаци читају, филтрирају, повезују, групишу или сортирају. Најчешће коришћене клаузуле су:

- WHERE се користи за филтрирање редова који испуњавају задати услов.
- JOIN за повезивање података из више табела.
- GROUP BY за груписање резултата.
- HAVING за филтрирање груписаних резултата.

- ORDER BY за сортирање приказаних података.

4.3. Структура и ограничења релационе базе

Структура релационе базе обухвата табеле, колоне, типове података, кључеве и ограничења. Ограничења имају важну улогу јер дефинишу правила која подаци морају да испуне пре него што буду уписани у базу. На тај начин база не чува само податке, већ активно учествује у очувању њихове исправности.

4.3.1. Примарни и страни кључеви

Примарни кључ је атрибут или група атрибута који јединствено идентификују ред у табели. Он омогућава да сваки запис буде недвосмислено препознат. У овом раду примарни кључеви су реализовани као целобројне вредности са аутоматским увећањем, што значи да база приликом уноса новог реда сама додељује наредну вредност примарног кључа.

Страни кључ служи за повезивање једне табеле са другом. Он обезбеђује да вредност у једној табели мора да постоји као примарни кључ у другој табели. На пример, `Obuka.Kandidat_id` повезује обуку са кандидатом, док `Cas.Obuka_id` повезује час са обуком. На овај начин се чува референцијални интегритет података. [6] [7]

4.3.2. Интегритет података

Интегритет података означава тачност, потпуност и доследност података у бази. У релационим базама података интегритет се обезбеђује кроз примарне кључеве, стране кључеве, ограничења типа NOT NULL, UNIQUE, CHECK и DEFAULT, као и кроз додатну логику у процедурама и тригерима. [6] [7]

У овом раду посебно су важне следеће врсте интегритета података:

- **Интегритет ентитета** подразумева да сваки ред у табели има јединствену идентификацију. За то се користи PRIMARY KEY ограничење, које обезбеђује да сваки запис у табели има јединствену вредност примарног кључа.
- **Референцијални интегритет** односи се на исправност веза између табела. За то се користи FOREIGN KEY ограничење, које обезбеђује да вредност у једној табели мора да постоји као примарни кључ у другој табели. На пример, обука не може бити повезана са кандидатом који не постоји у табели кандидата.
- **Доменски интегритет** подразумева да вредности у колонама буду у складу са дефинисаним типом података и ограничењима. За то се користе NOT NULL, CHECK и DEFAULT ограничења. NOT NULL спречава да обавезно поље остане празно, CHECK проверава да ли вредност испуњава задати услов, а DEFAULT поставља подразумевану вредност ако она није посебно унета.
- **Јединственост података** обезбеђује се UNIQUE ограничењем. Оно спречава да се у табели појаве две исте вредности у колони која мора бити јединствена. На пример, ЈМБГ кандидата или запосленог не треба да се понови у више редова.

Код страних кључева може се дефинисати и понашање базе приликом измене или брисања реда у родитељској табели. То се одређује опцијама ON UPDATE и ON DELETE. Најчешће опције су: [6]

- **NO ACTION** је подразумевано понашање. Ако у дете табели постоје редови који зависе од реда у родитељској табели, измена или брисање тог реда неће бити дозвољени.
- **CASCADE** омогућава да се измена или брисање у родитељској табели пренесе и на повезане редове у дете табели.
- **SET NULL** поставља вредност страног кључа у дете табели на NULL.
- **SET DEFAULT** поставља вредност страног кључа у дете табели на његову подразумевану вредност.

4.4. Организација и оптимизација базе података

Добро пројектована база података треба да буде и логички исправна и довољно ефикасна за употребу. Организација базе односи се на правилну поделу података по табелама и доменима, док се оптимизација односи на начине да се упити извршавају брже и са мање непотребног читања података.

4.4.1. Нормализација

Нормализација је поступак организовања података у релационој бази тако да се смањи понављање података и избегну проблеми приликом уноса, измене и брисања. Она помаже да свака табела има јасну улогу и да се подаци чувају на месту ком природно припадају.

Најчешће се посматрају прва, друга и трећа нормална форма:

- **Прва нормална форма (1NF)** подразумева да су вредности у пољима атомичне, односно да се у једном пољу не чува више различитих вредности. Поред тога, сваки запис у табели треба да буде јединствено препознат преко примарног кључа.
- **Друга нормална форма (2NF)** подразумева да табела најпре испуњава услове прве нормалне форме. Поред тога, сви атрибути који нису део примарног кључа треба да зависе од целог примарног кључа, а не само од његовог дела.
- **Трећа нормална форма (3NF)** подразумева да табела испуњава услове друге нормалне форме и да не постоје транзитивне зависности. То значи да не-кључни атрибути не треба да зависе једни од других, већ од примарног кључа. [8]

У бази аутошколе нормализација се види кроз раздвајање података у више повезаних табела. На пример, подаци о кандидату не чувају се поново у табели Обука, већ се обука повезује са кандидатом преко страног кључа. Исто важи и за категорије возачке дозволе, типове обуке, ценовник и друге помоћне целине. Оваква организација смањује дуплирање података и олакшава касније измене, јер се један податак мења на једном месту.

4.4.2. Индекси

Индекси су структуре које омогућавају бржи проналазак података у табели. Они су посебно корисни над колонама које се често користе у WHERE условима, JOIN повезивањима и ORDER BY сортирањима. У релационим базама података индекси се најчешће организују као **B-tree структура**, која омогућава да систем не мора да

чита целу табелу ред по ред, већ може брже да пронађе тражени опсег или конкретну вредност. Добро изабран индекс може значајно да смањи број прочитаних редова и убрза извршавање упита. [9]

Индекси нису потребни на свакој колони, јер сваки индекс заузима простор и успорава операције уноса, измене и брисања. Због тога су у овом раду индекси разматрани на основу реалних начина читања података, као што су часови по инструктору и датуму, обуке по кандидату, уплате по датуму и запослени по функцији.

4.4.3. Трансакције

Трансакција представља скуп повезаних операција над базом података које се извршавају као једна логичка целина. Ако су све операције успешно извршене, промене се трајно потврђују наредбом COMMIT. Уколико дође до грешке, наредбом ROLLBACK поништавају се све промене настале од почетка трансакције. На тај начин спречава се настанак непотпуних и међусобно неусаглашених података. [7] [3]

У систему аутошколе трансакције су корисне у процедурама које обављају више повезаних корака. На пример, приликом заказивања часа или евидентирања уплате потребно је проверити више услова и тек након тога уписати податке. Ако неки услов није испуњен, трансакција омогућава да се све започете промене пониште.

Поуздано извршавање трансакција описује се ACID особинама:

- **Атомичност (Atomicity)** обезбеђује да се све операције у оквиру трансакције изврше успешно или да се све пониште.
- **Доследност (Consistency)** обезбеђује да база након завршетка трансакције пређе из једног исправног стања у друго.
- **Изолованост (Isolation)** одређује да истовремене трансакције не смеју неконтролисано да утичу једна на другу.
- **Трајност (Durability)** гарантује да успешно потврђене промене остају сачуване и након прекида рада система. [7] [3]

4.4.4. Одабрани објекти базе података

Поред табела, ограничења и индекса, релационе базе података могу садржати и додатне трајно сачуване објекте који олакшавају рад са подацима и омогућавају да се део логике система реализује унутар саме базе. У овом раду посебно су значајни погледи, функције, процедуре и тригери, јер се они касније користе у практичном делу рада за приказ конкретне реализације система аутошколе. Трансакције се разматрају одвојено, пошто представљају механизам управљања извршавањем операција, а не трајно сачувани објекат базе. [6] [7] [3]

- **Погледи** - представљају сачуване упите који приказују податке из једне или више табела. Корисни су када је потребно поједноставити читање података или често користити исти сложени упит.
- **Функције** - су објекти базе који на основу улазних параметара враћају одређену вредност или табеларни резултат. Користе се када се иста логика читања или израчунавања понавља на више места.

- **Процедуре** - представљају трајно сачуване објекте базе који садрже скуп SQL, односно T-SQL наредби и пословну логику. Могу да имају параметре, услове и обраду грешака и могу се позивати више пута. Процедура и трансакција немају исту улогу: процедура одређује које операције треба извршити, док трансакција обезбеђује да се више повезаних операција заједно потврди или поништи. Једна процедура може у себи садржати трансакцију.
- **Тригери** - су објекти који се аутоматски покрећу као реакција на догађај у табели, најчешће након уноса, измене или брисања података. Користе се када база треба сама да изврши додатну проверу или промену података.

5. АНАЛИЗА ЗАХТЕВА СИСТЕМА

Сврха анализе захтева система јесте да читаоцу прикаже везу између свакодневног пословања аутошколе и базе података која се описује у наредним поглављима. Пословању аутошколе приступа се као проблему организације података, праћења пословних процеса и ефикасног коришћења расположивих ресурса. Да би се тај проблем правилно разумео, потребно је описати свакодневни ток рада аутошколе, као и податке и ограничења који се у том процесу јављају.

5.1. Опис проблема

У пословању аутошколе потребно је водити евиденцију о мноштву повезаних података, на основу којих треба да извештава надлежне институције. Потребно је чувати податке о кандидатима, запосленима, возилима, различитим обукама, часовима, полагањима и уплатама. Чување ових података ручно или у неповезаним документима смањује контролу над пословањем и ресурсима којима фирма располаже. Лако може доћи до грешака, дуплирања или губљења података. За фирму са оваквом одговорношћу такве ствари нису прихватљиве.

Један од најбољих начина решавања ових проблема је јединствена база података, која треба да обезбеди тачност података, повезаност између различитих делова система и брз приступ информацијама које су важне за свакодневни рад.

На пример, за једног кандидата потребно је знати за коју категорију похађа обуку, који инструктор је задужен за њега, које часове је имао, које уплате су извршене и какав је резултат његовог полагања. Сви ови подаци припадају различитим целинама, али у исто време морају да буду међусобно повезани како би систем био употребљив.

5.2. Корисничке улоге

У систему аутошколе може се препознати више корисничких улога. Свака улога има другачије потребе и користи различите делове система.

Администратор је корисник који има најшири приступ систему. Он може да уноси и мења податке о кандидатима, запосленима, обукама, часовима, полагањима и уплатама.

Запослени у администрацији користи систем за свакодневну евиденцију. Његов задатак је да уписује кандидате, прати њихове обуке, евидентира уплате и припрема податке потребне за организацију наставе и полагања.

Инструктор користи систем за преглед својих часова и кандидата са којима ради. За инструктора су посебно важни подаци о распореду часова, статусу обуке и возилу које се користи за практичну наставу.

Кандидат је особа која похађа обуку у аутошколи. Иако у овом раду није реализована апликација за директан приступ кандидата систему, база чува све податке који се односе на његову обуку, часове, полагања и уплате.

5.3. Захтеви система

Захтеви система представљају оно што база података, са техничке стране, треба да омогући. Како би захтеви били прегледнији, подељени су на **основне могућности система** и на **квалитет и правила рада система**. Основне могућности

система описују шта систем треба да омогући корисницима, док квалитет и правила рада описују поузданост, тачност, доследност и ефикасност система.

5.3.1. Основне могућности система

Основне могућности система у овом раду описане су кроз основни ток пословања аутошколе. На тај начин се јасно види које податке систем мора да чува и које операције треба да подржи.

Рад аутошколе започиње уписом кандидата. Приликом уписа потребно је евидентирати основне личне и контакт податке кандидата, као што су име, презиме, ЈМБГ, телефон и адреса електронске поште.

Након уписа, кандидат се уписује на обуку, која може бити теоријска, практична или обука из прве помоћи. Генерални ток обуке за полагање једне категорије може се представити редоследом: **теоријска обука, прва помоћ и практична обука**. Обука представља срж система, јер се преко ње кандидату додељује инструктор који води обуку.

Током обуке, администрација организује теоријске или практичне часове. За сваки час потребно је знати датум, време почетка и краја, инструктора и локацију одржавања.

Поред часова, систем треба да омогући евиденцију возила која се користе у практичној обуци. За свако возило потребно је чувати основне податке о возилу и категорију возачке дозволе за коју се возило користи. Ова евиденција омогућава бољу контролу доступних возила.

По завршетку основног дела обуке, кандидат приступа полагању. Приликом полагања учествује и представник надлежне установе, због чега је потребно имати информације о томе ко присуствује полагању, када је заказан термин, који је тип полагања и какав је резултат полагања.

Финансијски део пословања обухвата евиденцију уплата кандидата. Ова евиденција је важна јер омогућава праћење измирених обавеза кандидата и наставак кандидата кроз различите врсте обуке у аутошколи.

Поред наведених целина, систем треба да омогући и евиденцију запослених и њихових функција. Један запослени може имати више функција у аутошколи, као што су инструктор, предавач, администратор или друга улога.

Овако описан ток пословања представља основу за препознавање главних функционалних захтева и касније пројектовање табела у бази података.

5.3.2. Квалитет и правила рада система

База података треба да обезбеди тачност и доследност података. То значи да систем не сме да дозволи унос података који нису исправни, као што је кандидат без имена или час који није повезан са постојећим инструктором.

Један од важних захтева за квалитетан рад система је спречавање дуплирања података. На пример, ЈМБГ кандидата и запосленог треба да буде јединствен, јер представља податак који идентификује особу.

Систем треба да буде организован тако да се подаци могу лако проширивати. Уколико се у будућности појави потреба за новим категоријама, новим типовима

обуке или додатним извештајима, структура базе треба да омогући такво проширење без великих измена.

Такође, база треба да омогући ефикасно читање података. Због тога је потребно анализирати најчешће упите и додати индексе на колоне које се често користе у претрази, повезивању табела и сортирању.

6. ПРОЈЕКТОВАЊЕ БАЗЕ ПОДАТАКА

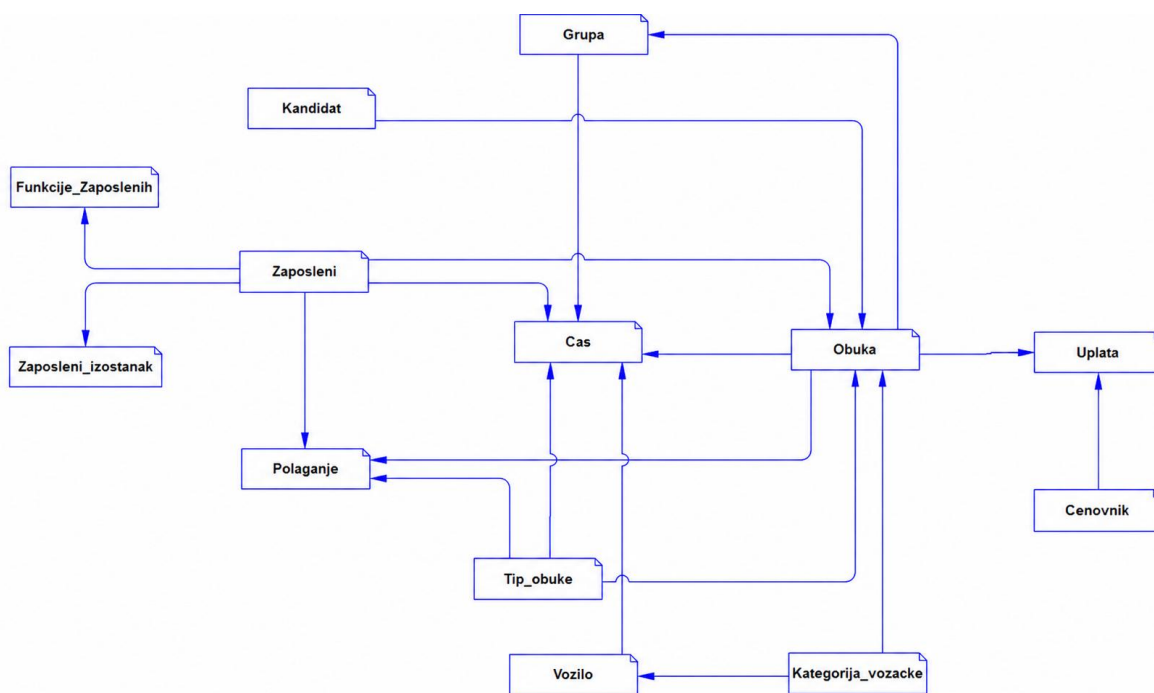
Пројектовање базе података у овом раду изведено је кроз три нивоа модела: **концептуални, логички и физички**. Оваква подела омогућава да се систем најпре сагледа као пословни проблем, затим као структура табела и веза, а на крају као конкретна имплементација у MS SQL Server окружењу.

Модели су израђени у SAP PowerDesigner алату. Концептуални модел приказује главне пословне целине аутошколе, логички модел приказује ентитете, кључеве и кардиналности, док физички модел приказује типове података, ограничења, индексе и техничке детаље који су потребни за креирање базе. [5]

6.1. Концептуални модел

Концептуални модел приказује основне појмове система аутошколе без детаља који су везани за конкретан систем за управљање базом података.

Циљ концептуалног модела је да се покаже како су главни делови пословања повезани. На пример, кандидат похађа обуку, обука садржи часове, запослени води обуку или час, возило се користи у практичној настави, а уплата се односи на обуку и ставку ценовника.



Слика 6.1 Концептуални модел базе података аутошколе.

6.1.1. Идентификација ентитета

Ентитети су одређени на основу пословних података које аутошкола мора да прати у свакодневном раду.

Као главни ентитети издвојени су:

- Кандидат
- Обука
- Запослени
- Час

Логички модел представља прелаз из концептуалног приказа у структуру која је ближа релационој бази података. У њему су приказани ентитети, атрибути, примарни идентификатори, алтернативни идентификатори, страни идентификатори и кардиналности веза.

6.2.1. Превођење концептуалних ентитета у логички модел

Приликом преласка из концептуалног у логички модел, главни ентитети система су задржани, али су детаљније дефинисани њихови атрибути, идентификатори и међусобне везе. Ентитети као што су **Kandidat**, **Zaposleni**, **Obuka**, **Cas**, **Polaganje**, **Vozilo**, **Uplata** и **Cenovnik** представљају основне пословне целине система аутошколе.

Посебно важан део логичког моделовања је разлагање веза више-према-више. За разлику од концептуалног модела, где се таква веза може приказати директно, у релационом моделу она се реализује помоћном табелом са страним кључевима ка повезаним ентитетима.

Због тога су у логичком моделу додате помоћне табеле које решавају овакве односе:

- **Zaposleni_Funkcija** повезује запослене и њихове функције, јер један запослени може имати више функција, а једна функција може припадати већем броју запослених.
- **Kandidat_grupa** повезује обуке кандидата са групама, јер једна група може садржати више кандидата, док кандидат кроз различите обуке може бити распоређен у различите групе.
- **Polaganje_kandidat** повезује полагања и обуке кандидата, јер на једном полагању може учествовати више кандидата, а један кандидат може током обуке имати више полагања.
- **Nadzornici_polaganja** повезује полагања и запослене који имају улогу надзорника, јер једно полагање може имати више надзорника, а један запослени може надzirати више полагања.

6.2.2. Дефинисање примарних, алтернативних и страних идентификатора

Сваки ентитет има **примарни идентификатор**, који је у моделу означен ознаком <pi> и којим се свака појава ентитета јединствено одређује. Примарни идентификатори у овом моделу најчешће су представљени атрибутом Id. Код спојних ентитета примарни идентификатор може бити сложен и састојати се од комбинације идентификатора повезаних ентитета.

Алтернативни идентификатор означен је ознаком <ai> и дефинише се за атрибут који такође мора имати јединствену вредност, али није изабран као примарни идентификатор. На пример, ЈМБГ може представљати алтернативни идентификатор ентитета **Kandidat**, јер се његова вредност не сме поновити код више кандидата.

Страни идентификатор означен је ознаком <fi>. Он настаје преношењем идентификатора повезаног ентитета и користи се за представљање веза у логичком моделу. На пример, ентитет **Obuka** садржи стране идентификаторе повезане са ентитетима **Kandidat**, **Kategorija_vozacke**, **Tip_obuke** и **Zaposleni**. На тај

начин свака обука повезује се са одговарајућим кандидатом, категоријом возачке дозволе, типом обуке и запосленим.

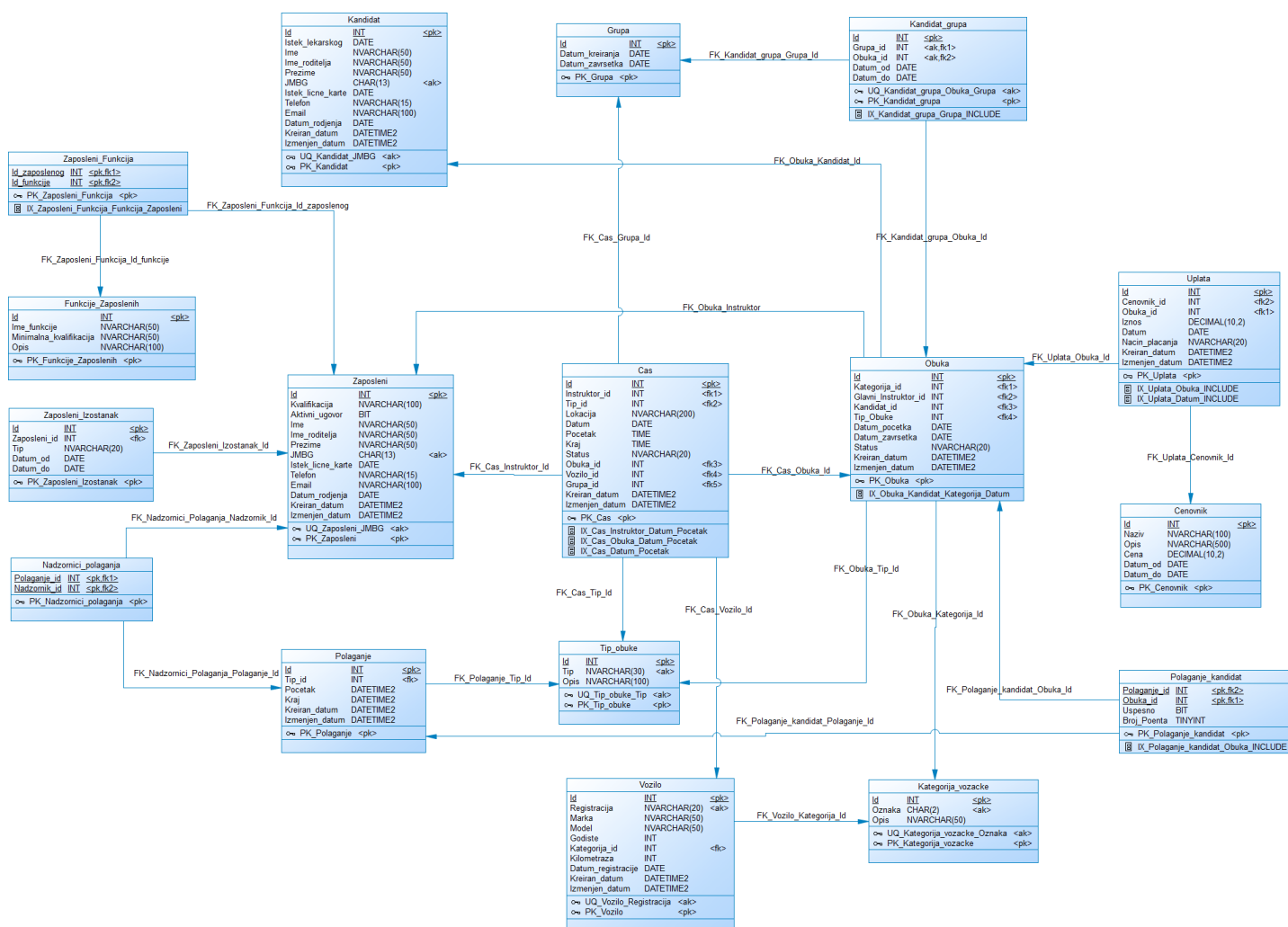
6.2.3. Кардиналности

Кардиналности у логичком моделу приказују колико појава једног ентитета може бити повезано са појавама другог ентитета. На линијама веза користе се ознаке које показују минималан и максималан број повезаних појава.

Кружић на крају везе означава да је учешће у вези опционо, односно да појава ентитета не мора имати повезан запис са друге стране. Вертикална црта означава једну обавезну појаву, док разгранавање означава више појава.

На пример, веза између *Zaposleni* и *Cas* показује да један запослени може водити више часова, док сваки час мора имати једног инструктора. То значи да је веза за час обавезна, а за запосленог може постојати више повезаних часова.

6.3. Физички модел



Слика 6.3 Физички модел базе података аутошколе.

Физички модел приказује како је логички модел припремљен за реализацију у MS SQL Server-у. У њему су дефинисани:

- конкретни типови података
- ограничења
- примарни кључеви
- страни кључеви
- јединствена ограничења
- индекси

Овај модел је основа за SQL скрипте које се налазе у фолдеру “SQL”. На основу њега су формиран фајлови за креирање табела.

6.3.1. Типови података

Типови података одабрани су према врсти и опсегу вредности које се чувају. Тип INT користи се за идентификаторе, док се TINYINT примењује за мање целобројне вредности у опсегу од 0 до 255.

Текстуални подаци променљиве дужине чувају се као NVARCHAR, који подржава српска слова. Тип CHAR користи се за податке фиксне дужине. Због тога се ЈМБГ чува као CHAR(13). Иако садржи цифре, ЈМБГ није намењен математичким операцијама и може почети нулом.

Тип DATE чува календарски датум, TIME време у току дана, док DATETIME2 обједињује датум и прецизно време догађаја. Финансијски износи чувају се као DECIMAL(10,2), јер тај тип омогућава тачно представљање новчаних вредности.

6.3.2. Ограничења над колонама

У физичком моделу су приказана основна ограничења: <pk> означава примарни кључ, <fk> страни кључ, а <ak> алтернативни кључ. Примарни кључ јединствено идентификује ред у табели, страни кључ повезује две табеле, док алтернативни кључ означава атрибут чија вредност мора бити јединствена, али није примарни кључ.

Обавезни атрибути у моделу не могу имати NULL вредност. Детаљнија ограничења, као што су CHECK и DEFAULT, нису директно приказана у физичком моделу, већ су дефинисана у SQL скриптама за креирање табела.

7. ИМПЛЕМЕНТАЦИЈА БАЗЕ ПОДАТАКА

Имплементација базе података изведена је кроз SQL скрипте које се покрећу у MS SQL Server окружењу. Скрипте су организоване тако да се прво креира база и основне табеле, затим уносе тест подаци, а након тога креирају погледи, функције, процедуре, тригери и примери за анализу перформанси.

7.1. Креирање базе података

Пре креирања табела потребно је у MS SQL Server-у креирати базу података Autoskola и поставити је као активну базу. То се ради следећим наредбама:

```
CREATE DATABASE Autoskola;  
GO  
USE Autoskola;  
GO
```

Прилог 7.1 Креирање и избор базе података.

Наредба CREATE DATABASE креира нову базу података, док наредба USE поставља ту базу као активну, како би се све наредне SQL наредбе извршавале над њом.

Читава база се може креирати покретањем фајла 00_master.sql, чиме се све скрипте извршавају у унапред дефинисаном редоследу. Овај начин је погодан када желимо да будемо сигурни да ће табеле и остали објекти бити креирани правилним редоследом. Скрипте је могуће покретати и појединачно, али је у том случају потребно водити рачуна о зависностима између табела, јер табела која садржи страни кључ мора бити креирана након табеле на коју се тај кључ односи.

7.2. Креирање табела

Табеле су креиране T-SQL наредбом CREATE TABLE. У самој дефиницији табеле наведени су:

- примарни кључеви
- страни кључеви
- обавезна поља
- подразумеване вредности
- CHECK ограничења

Примарни кључеви су дефинисани као INT IDENTITY(1,1). На тај начин SQL Server аутоматски генерише јединствену целобројну вредност приликом додавања новог реда у табелу, па није потребно ручно уносити идентификатор.

Табеле базе података подељене су у више SQL фајлова према области система на коју се односе:

- 01_ljudi.sql
- 02_obuka.sql
- 03_casovi.sql
- 04_polaganje.sql
- 05_finansije.sql

У наставку су приказане CREATE TABLE дефиниције табела из ових фајлова.

7.2.1. Kandidat

Табела за вођење евиденције кандидата дата је у Прилог 7.2. У њој се чувају основни лични и контакт подаци кандидата, као и подаци о лекарском прегледу, личној карти и датуму рођења. ЈМБГ представља јединствену вредност кандидата и не сме се поновити код више записа. Ограничења над колонама проверавају да име, презиме и телефон нису празни, да ЈМБГ садржи само цифре, да имејл има основни формат и да датум рођења није у будућности. Поља *Kreiran_datum* и *Izmenjen_datum* омогућавају праћење када је кандидат унет и када су његови подаци последњи пут мењани.

```
CREATE TABLE Kandidat (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Istek_lekarskog DATE,
    Ime NVARCHAR(50) NOT NULL,
    Ime_roditelja NVARCHAR(50),
    Prezime NVARCHAR(50) NOT NULL,
    JMBG CHAR(13) NOT NULL,
    Istek_licne_karte DATE,
    Telefon NVARCHAR(15) NOT NULL,
    Email NVARCHAR(100),
    Datum_rodjenja DATE NOT NULL,
    Kreiran_datum DATETIME2 NOT NULL
    CONSTRAINT DF_Kandidat_Kreiran_datum DEFAULT (getdate()),
    Izmenjen_datum DATETIME2,
    CONSTRAINT UQ_Kandidat_JMBG UNIQUE (JMBG),
    -- Obavezna tekstualna polja ne smeju biti prazna.
    CONSTRAINT CK_Kandidat_Tekst_Obavezan CHECK (
        LEN(LTRIM(RTRIM(Ime))) > 0
        AND LEN(LTRIM(RTRIM(Prezime))) > 0
        AND LEN(LTRIM(RTRIM(Telefon))) > 0
    ),
    -- Ime i prezime smeju imati samo slova i moraju poceti velikim slovom.
    CONSTRAINT CK_Kandidat_Ime_Prezime_Format CHECK (
        Ime NOT LIKE N'%[^A-Za-zćčšđžćčšđž]%'
        AND Ime LIKE N'[A-ZĆČŠĐŽ]%'
        AND Ime NOT LIKE N'% %'
        AND Prezime NOT LIKE N'%[^A-Za-zćčšđžćčšđž]%'
        AND Prezime LIKE N'[A-ZĆČŠĐŽ]%'
        AND Prezime NOT LIKE N'% %'
    ),
    -- Opciona tekstualna polja, ako su uneta, ne smeju biti prazna.
    CONSTRAINT CK_Kandidat_Tekst_Optional CHECK (
        (Ime_roditelja IS NULL OR LEN(LTRIM(RTRIM(Ime_roditelja))) > 0)
        AND (Email IS NULL OR LEN(LTRIM(RTRIM(Email))) > 0)
    ),
    -- Email, ako je unet, mora imati osnovni format.
    CONSTRAINT CK_Kandidat_Email_Format CHECK (Email IS NULL OR Email LIKE '%@%.%'),
    -- JMBG sme da ima samo cifre.
    CONSTRAINT CK_Kandidat_JMBG_Format CHECK (JMBG NOT LIKE '%[^0-9]%),
    -- Datum rodjenja mora biti manji od danasnjeg datuma.
    CONSTRAINT CK_Kandidat_Datum_rodjenja CHECK (Datum_rodjenja < CONVERT(date,
getdate()))
);
```

Прилог 7.2 Дизајн табеле Kandidat.

7.2.2. Zaposleni

Табела за вођење евиденције запослених дата је у Прилог 7.3. Она садржи личне податке, контакт податке, квалификацију и информацију да ли запослени има активан уговор. ЈМБГ запосленог је јединствен, чиме се спречава дуплирање истог лица у систему. Ограничења проверавају исправност имена, презимена, ЈМБГ-а, имејла и датума рођења, док се поље `Aktivni_ugovor` подразумевано поставља као активно. Датуми креирања и измене записа користе се за контролу промена података о запосленима.

```
CREATE TABLE Zaposleni (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Kvalifikacija NVARCHAR(100),
    Aktivni_ugovor BIT NOT NULL
        CONSTRAINT DF_Zaposleni_Aktivni_ugovor DEFAULT (1),
    Ime NVARCHAR(50) NOT NULL,
    Ime_roditelja NVARCHAR(50),
    Prezime NVARCHAR(50) NOT NULL,
    JMBG CHAR(13) NOT NULL,
    Istek_licne_karte DATE,
    Telefon NVARCHAR(15) NOT NULL,
    Email NVARCHAR(100),
    Datum_rodjenja DATE NOT NULL,
    Kreiran_datum DATETIME2 NOT NULL
        CONSTRAINT DF_Zaposleni_Kreiran_datum DEFAULT (getdate()),
    Izmenjen_datum DATETIME2,
    CONSTRAINT UQ_Zaposleni_JMBG UNIQUE (JMBG),
    -- Obavezna tekstualna polja ne smeju biti prazna.
    CONSTRAINT CK_Zaposleni_Tekst_Obavezan CHECK (
        LEN(LTRIM(RTRIM(Ime))) > 0
        AND LEN(LTRIM(RTRIM(Prezime))) > 0
        AND LEN(LTRIM(RTRIM(Telefon))) > 0
    ),
    -- Ime i prezime smeju imati samo slova i moraju poceti velikim slovom.
    CONSTRAINT CK_Zaposleni_Ime_Prezime_Format CHECK (
        Ime NOT LIKE N'%[^A-Za-zćčšđžĆČŠĐŽ]%'
        AND Ime LIKE N'[A-ZĆČŠĐŽ]%'
        AND Ime NOT LIKE N'% %'
        AND Prezime NOT LIKE N'%[^A-Za-zćčšđžĆČŠĐŽ]%'
        AND Prezime LIKE N'[A-ZĆČŠĐŽ]%'
        AND Prezime NOT LIKE N'% %'
    ),
    -- Opciona tekstualna polja, ako su uneta, ne smeju biti prazna.
    CONSTRAINT CK_Zaposleni_Tekst_Optional CHECK (
        (Kvalifikacija IS NULL OR LEN(LTRIM(RTRIM(Kvalifikacija))) > 0)
        AND (Ime_roditelja IS NULL OR LEN(LTRIM(RTRIM(Ime_roditelja))) > 0)
        AND (Email IS NULL OR LEN(LTRIM(RTRIM(Email))) > 0)
    ),
    -- Email, ako je unet, mora imati osnovni format.
    CONSTRAINT CK_Zaposleni_Email_Format CHECK (Email IS NULL OR Email LIKE '%@%.%'),
    -- JMBG sme da ima samo cifre.
    CONSTRAINT CK_Zaposleni_JMBG_Format CHECK (JMBG NOT LIKE '%[^0-9]%' ),
    -- Datum rodjenja mora biti manji od danasnjeg datuma.
    CONSTRAINT CK_Zaposleni_Datum_rodjenja CHECK (Datum_rodjenja < CONVERT(date,
getdate()))
);
```

Прилог 7.3 Дизајн табеле Zaposleni.

7.2.3. Funkcije_Zaposlenih

Табела функција запослених дата је у Прилог 7.4. У њој се чувају могуће функције које запослени може имати у аутошколи, као што су инструктор, предавач или администратор. Табела садржи назив функције, минималну квалификацију и опис. Ограничења обезбеђују да назив функције не буде празан, као и да опциони текстуални подаци, ако су унети, имају исправну вредност.

```
CREATE TABLE Funkcije_Zaposlenih (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Ime_funkcije NVARCHAR(50) NOT NULL,  
    Minimalna_kvalifikacija NVARCHAR(50),  
    Opis NVARCHAR(100),  
    -- Tekstualna polja ne smeju biti prazna.  
    CONSTRAINT CK_Funkcije_Zaposlenih_Tekst CHECK (  
        LEN(LTRIM(RTRIM(Ime_funkcije))) > 0  
        AND (Minimalna_kvalifikacija IS NULL OR  
        LEN(LTRIM(RTRIM(Minimalna_kvalifikacija))) > 0)  
        AND (Opis IS NULL OR LEN(LTRIM(RTRIM(Opis))) > 0)  
    )  
);
```

Прилог 7.4 Дизајн табеле Funkcije_Zaposlenih.

7.2.4. Zaposleni_Funkcija

Табела која повезује запослене и њихове функције дата је у Прилог 7.5. Она је потребна зато што један запослени може имати више функција, а иста функција може припадати већем броју запослених. Табела садржи идентификатор запосленог и идентификатор функције, који заједно чине сложени примарни кључ. Страни кључеви обезбеђују да се функција може доделити само постојећем запосленом и постојећој функцији.

```
CREATE TABLE Zaposleni_Funkcija (  
    Id_zaposlenog INT NOT NULL REFERENCES Zaposleni (Id),  
    Id_funkcije INT NOT NULL REFERENCES Funkcije_Zaposlenih (Id),  
    CONSTRAINT PK_Zaposleni_Funkcija PRIMARY KEY (Id_zaposlenog, Id_funkcije)  
);
```

Прилог 7.5 Дизајн табеле Zaposleni_Funkcija.

7.2.5. Zaposleni_Izostanak

Табела за евиденцију изостанака запослених дата је у Прилог 7.6. У њој се чува ком запосленом изостанак припада, који је тип изостанка и у ком периоду траје. Тип изостанка ограничен је на вредности BOL и GODISNJI. Додатно ограничење проверава да датум завршетка изостанка, ако је унет, не може бити пре датума почетка.

```
CREATE TABLE Zaposleni_Izostanak (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Zaposleni_id INT NOT NULL CONSTRAINT FK_Zaposleni_Izostanak_Id REFERENCES Zaposleni (Id),  
    Tip NVARCHAR(20) NOT NULL,  
    Datum_od DATE NOT NULL,  
    Datum_do DATE,  
    -- Tip izostanka moze biti samo bolovanje ili godisnji odmor.  
    CONSTRAINT CK_Zaposleni_Izostanak_Tip CHECK (Tip IN ('BOL', 'GODISNJI')),  
    -- Datum zavrsetka izostanka, ako postoji, ne sme biti pre datuma pocetka.  
    CONSTRAINT CK_Zaposleni_Izostanak_Datum CHECK (Datum_do IS NULL OR Datum_do >= Datum_od)  
);
```

Прилог 7.6 Дизајн табеле Zaposleni_Izostanak.

7.2.6. Kategorija_vozacke

Табела категорија возачке дозволе дата је у Прилог 7.7. Она представља шифарник категорија које се користе у обукама и код возила. Ознака категорије је јединствена, јер се иста категорија не сме више пута појављивати у систему. Ограничења проверавају да ознака није празна и да опис, ако је унет, има исправну текстуалну вредност.

```
CREATE TABLE Kategorija_vozacke (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Oznaka CHAR(2) NOT NULL,  
    Opis NVARCHAR(50),  
    -- Oznaka kategorije ne sme da se ponavlja.  
    CONSTRAINT UQ_Kategorija_vozacke_Oznaka UNIQUE (Oznaka),  
    -- Tekstualna polja ne smeju biti prazna.  
    CONSTRAINT CK_Kategorija_vozacke_Tekst CHECK (  
        LEN(LTRIM(RTRIM(Oznaka))) > 0  
        AND (Opis IS NULL OR LEN(LTRIM(RTRIM(Opis))) > 0)  
    )  
);
```

Прилог 7.7 Дизајн табеле Kategorija_vozacke.

7.2.7. Tip_obuke

Табела типова обуке дата је у Прилог 7.8. У њој се чувају дозвољени типови обуке који се користе код часова, обука и полагања. Назив типа обуке је јединствен, а дозвољене вредности су: теоријска, практична и прва помоћ. Опис је опциони податак, али ако се унесе, не сме бити празан.

```
CREATE TABLE Tip_obuke (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Tip NVARCHAR(30) NOT NULL,  
    Opis NVARCHAR(100),  
    -- Naziv tipa obuke ne sme da se ponavlja.  
    CONSTRAINT UQ_Tip_obuke_Tip UNIQUE (Tip),  
    -- Opis, ako je unet, ne sme biti prazan.  
    CONSTRAINT CK_Tip_obuke_Opis CHECK (Opis IS NULL OR LEN(LTRIM(RTRIM(Opis))) > 0),  
    -- Tip obuke moze biti samo jedna od dozvoljenih vrednosti.  
    CONSTRAINT CK_Tip_obuke_Tip CHECK (Tip IN ('Teorijska', 'Prakticna', 'Prva  
Помоћ'))  
);
```

Прилог 7.8 Дизајн табеле Tip_obuke.

7.2.8. Grupa

Табела група дата је у Прилог 7.9. Она служи за евиденцију група кандидата, најчешће код теоријске наставе или других групних активности. Датум креирања групе подразумевано се поставља на тренутни датум. Ако група има датум завршетка, он не сме бити пре датума креирања.

```
CREATE TABLE Grupa (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Datum_kreiranja DATE NOT NULL  
    CONSTRAINT DF_Grupa_Datum_kreiranja DEFAULT (getdate()),  
    Datum_zavrsetka DATE,  
    -- Datum zavrsetka grupe, ako postoji, ne sme biti pre datuma kreiranja.  
    CONSTRAINT CK_Grupa_Datum CHECK (Datum_zavrsetka IS NULL OR Datum_zavrsetka >=  
Datum_kreiranja)  
);
```

Прилог 7.9 Дизајн табеле Grupa.

7.2.9. Obuka

Табела обуке дата је у Прилог 7.10. Она представља једну од главних табела система, јер повезује кандидата, категорију возачке дозволе, тип обуке и главног инструктора. Страни кључеви обезбеђују да обука може бити повезана само са постојећим кандидатом, категоријом, типом обуке и запосленим. Статус обуке ограничен је на вредности: активан, завршен и прекинут, а датум завршетка не сме бити пре датума почетка. Поља *Kreiran_datum* и *Izmenjen_datum* омогућавају праћење креирања и измена обуке.

```
CREATE TABLE Obuka (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Kategorija_id INT NOT NULL REFERENCES Kategorija_vozacke (Id),
    Glavni_Instruktor_id INT REFERENCES Zaposleni (Id),
    Kandidat_id INT NOT NULL REFERENCES Kandidat (Id),
    Tip_Obuke INT NOT NULL REFERENCES Tip_obuke (Id),
    Datum_pocetka DATE NOT NULL,
    Datum_zavrsetka DATE,
    Status NVARCHAR(20) NOT NULL
        CONSTRAINT DF_Obuka_Status DEFAULT (N'Aktivan'),
    Kreiran_datum DATETIME2 NOT NULL
        CONSTRAINT DF_Obuka_Kreiran_datum DEFAULT (getdate()),
    Izmenjen_datum DATETIME2,
    -- Status obuke moze biti samo jedna od dozvoljenih vrednosti.
    CONSTRAINT CK_Obuka_Status CHECK (Status IN (N'Aktivan', N'Završen', N'Prekinut')),
    -- Datum zavrsetka obuke, ako postoji, ne sme biti pre datuma pocetka.
    CONSTRAINT CK_Obuka_Datum CHECK (Datum_zavrsetka IS NULL OR Datum_zavrsetka >=
Datum_pocetka)
);
```

Прилог 7.10 Дизајн табеле *Obuka*.

7.2.10. Kandidat_grupa

Табела која повезује кандидата, односно његову обуку, са групом дата је у Прилог 7.11. Она омогућава да се прати у којој групи се кандидат налази током одређене обуке. Јединствено ограничење спречава да иста обука буде додата у исту групу више пута. Датум изласка из групе, ако постоји, не сме бити пре датума уласка.

```
CREATE TABLE Kandidat_grupa (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Grupa_id INT NOT NULL REFERENCES Grupa (Id),
    Obuka_id INT NOT NULL REFERENCES Obuka (Id),
    Datum_od DATE NOT NULL,
    Datum_do DATE,
    -- Ista obuka ne moze biti dodata u istu grupu vise puta.
    CONSTRAINT UQ_Kandidat_grupa_Obuka_Grupa UNIQUE (Obuka_id, Grupa_id),
    -- Datum izlaska iz grupe, ako postoji, ne sme biti pre datuma ulaska.
    CONSTRAINT CK_Kandidat_grupa_Datum CHECK (Datum_do IS NULL OR Datum_do >= Datum_od)
);
```

Прилог 7.11 Дизајн табеле *Kandidat_grupa*.

7.2.11. Vozilo

Табела возила дата је у Прилог 7.12. У њој се чувају подаци о возилима која се користе у практичној обуци, као што су регистрација, марка, модел, годиште, категорија и километража. Регистрација је јединствена вредност, јер једно возило не сме бити више пута заведено у систему. Ограничења проверавају да текстуална поља нису празна, да годиште буде у реалном опсегу и да километража не може бити негативна. Датуми креирања и измене записа помажу у праћењу промена података о возилу.

```
CREATE TABLE Vozilo (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Registracija NVARCHAR(20) NOT NULL,
    Marka NVARCHAR(50) NOT NULL,
    Model NVARCHAR(50) NOT NULL,
    Godiste INT NOT NULL,
    Kategorija_id INT NOT NULL REFERENCES Kategorija_vozacke (Id),
    Kilometraza INT NOT NULL
        CONSTRAINT DF_Vozilo_Kilometraza DEFAULT (0),
    Datum_registracije DATE NOT NULL,
    Kreiran_datum DATETIME2 NOT NULL
        CONSTRAINT DF_Vozilo_Kreiran_datum DEFAULT (getdate()),
    Izmenjen_datum DATETIME2,
    -- Registracija vozila ne sme da se ponavlja.
    CONSTRAINT UQ_Vozilo_Registracija UNIQUE (Registracija),
    -- Godiste vozila mora biti u realnom opsegu.
    CONSTRAINT CK_Vozilo_Godiste CHECK (Godiste BETWEEN 1980 AND YEAR(getdate()) + 1),
    -- Kilometraza ne moze biti negativna.
    CONSTRAINT CK_Vozilo_Kilometraza CHECK (Kilometraza >= 0),
    -- Tekstualna polja ne smeju biti prazna.
    CONSTRAINT CK_Vozilo_Tekst CHECK (
        LEN(LTRIM(RTRIM(Registracija))) > 0
        AND LEN(LTRIM(RTRIM(Marka))) > 0
        AND LEN(LTRIM(RTRIM(Model))) > 0
    )
);
```

Прилог 7.12 Дизајн табеле Vozilo.

7.2.12. Polaganje

Табела полагања дата је у Прилог 7.13. У њој се чувају термини полагања, тип полагања и време почетка и завршетка. Тип полагања повезан је са табелом Tip_obuke преко страног кључа. Ограничење проверава да време завршетка полагања мора бити после времена почетка. Датуми креирања и измене записа користе се за праћење промена термина полагања.

```
CREATE TABLE Polaganje (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Tip_id INT NOT NULL REFERENCES Tip_obuke (Id),
    Pocetak DATETIME2 NOT NULL,
    Kraj DATETIME2 NOT NULL,
    Kreiran_datum DATETIME2 NOT NULL
        CONSTRAINT DF_Polaganje_Kreiran_datum DEFAULT (getdate()),
    Izmenjen_datum DATETIME2,
    -- Vreme zavrsetka polaganja mora biti posle vremena pocetka.
    CONSTRAINT CK_Polaganje_Vreme CHECK (Kraj > Pocetak)
);
```

Прилог 7.13 Дизајн табеле Polaganje.

7.2.13. Polaganje_kandidat

Табела која повезује полагање и обуку кандидата дата је у Прилог 7.14. Она је потребна зато што једно полагање може имати више кандидата, а један кандидат може имати више покушаја полагања. Сложени примарни кључ чине Polaganje_id и Obuka_id. Ограничења проверавају да број поена буде у опсегу од 0 до 100 и да се резултат и број поена уносе заједно.

```
CREATE TABLE Polaganje_kandidat (
    Polaganje_id INT NOT NULL REFERENCES Polaganje (Id),
    Obuka_id INT NOT NULL REFERENCES Obuka (Id),
    Uspesno BIT,
    Broj_Poenta TINYINT,
    -- Ista obuka ne moze biti prijavljena na isto polaganje vise puta.
    CONSTRAINT PK_Polaganje_kandidat PRIMARY KEY (Polaganje_id, Obuka_id),
    -- Broj poena, ako je unet, mora biti u opsegu od 0 do 100.
    CONSTRAINT CK_Polaganje_kandidat_Broj_Poenta CHECK (Broj_Poenta IS NULL OR
    Broj_Poenta BETWEEN 0 AND 100),
    -- Ako rezultat nije evidentiran, poeni su NULL; ako jeste, poeni moraju biti uneti.
    CONSTRAINT CK_Polaganje_kandidat_Uspesno_Poeni CHECK (
        (Uspesno IS NULL AND Broj_Poenta IS NULL)
        OR
        (Uspesno IS NOT NULL AND Broj_Poenta IS NOT NULL)
    )
);
```

Прилог 7.14 Дизајн табеле Polaganje_kandidat.

7.2.14. Nadzornici_polaganja

Табела надзорника полагања дата је у Прилог 7.15. Она повезује полагања са запосленима који имају улогу надзорника. Сложени примарни кључ чине Polaganje_id и Nadzornik_id, па исти надзорник не може бити додат на исто полагање више пута. Страни кључеви обезбеђују да се могу повезати само постојеће полагање и постојећи запослени.

```
CREATE TABLE Nadzornici_polaganja (
    Polaganje_id INT NOT NULL REFERENCES Polaganje (Id),
    Nadzornik_id INT NOT NULL REFERENCES Zaposleni (Id),
    -- Isti nadzornik ne moze biti dodat na isto polaganje vise puta.
    CONSTRAINT PK_Nadzornici_polaganja PRIMARY KEY (Polaganje_id, Nadzornik_id)
);
```

Прилог 7.15 Дизајн табеле Nadzornici_polaganja.

7.2.15. Cas

Табела часова дата је у Прилог 7.16. Она чува податке о заказаним, одржаним и отказаним часовима, укључујући инструктора, тип часа, датум, време, статус, локацију и повезану обуку или групу. Статус часа ограничен је на вредности заказан, одржан и отказан, а време завршетка мора бити после времена почетка. Ограничење CK_Cas_Tip_Polja раздваја практичне и групне часове: практични час мора бити повезан са обуком и возилом, док групни час мора бити повезан са групом. На тај начин један час не може истовремено припадати и обуци и групи. Поља Kreiran_datum и Izmenjen_datum омогућавају праћење креирања и накнадних измена записа.

```
CREATE TABLE Cas (
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Instruktor_id INT NOT NULL REFERENCES Zaposleni (Id),
    Tip_id INT NOT NULL REFERENCES Tip_obuke (Id),
    Lokacija NVARCHAR(200),
    Datum DATE NOT NULL,
    Pocetak TIME NOT NULL,
    Kraj TIME NOT NULL,
    Status NVARCHAR(20) NOT NULL
    CONSTRAINT DF_Cas_Status DEFAULT (N'Zakazan'),
    Obuka_id INT REFERENCES Obuka (Id),
    Vozilo_id INT REFERENCES Vozilo (Id),
    Grupa_id INT REFERENCES Grupa (Id),
    Kreiran_datum DATETIME2 NOT NULL
    CONSTRAINT DF_Cas_Kreiran_datum DEFAULT (getdate()),
    Izmenjen_datum DATETIME2,
    -- Lokacija, ako je uneta, ne sme biti prazna.
    CONSTRAINT CK_Cas_Lokacija CHECK (Lokacija IS NULL OR LEN(LTRIM(RTRIM(Lokacija))) > 0),
    -- Status casa moze biti samo jedna od dozvoljenih vrednosti.
    CONSTRAINT CK_Cas_Status CHECK (Status IN (N'Zakazan', N'Odrzan', N'Otkazan')),
    -- Vreme zavrsetka casa mora biti posle vremena pocetka.
    CONSTRAINT CK_Cas_Vreme CHECK (Kraj > Pocetak),
    -- Prakticni cas ima obuku i vozilo, a teorijski cas ima samo grupu.
    CONSTRAINT CK_Cas_Tip_Polja CHECK (
        (
            Obuka_id IS NOT NULL
            AND Vozilo_id IS NOT NULL
            AND Grupa_id IS NULL
        )
        OR
        (
            Grupa_id IS NOT NULL
            AND Obuka_id IS NULL
            AND Vozilo_id IS NULL
        )
    )
);
```

Прилог 7.16 Дизајн табеле Cas.

7.2.16. Cenovnik

Табела ценовника дата је у Прилог 7.17. У њој се чувају ставке ценовника, њихова цена, опис и период важења. Цена мора бити већа од нуле, јер ставка ценовника не може имати нулту или негативну вредност. Ако је унет датум престанка важења, он не сме бити пре датума почетка важења. Назив ставки не сме бити празан.

```
CREATE TABLE Cenovnik (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Naziv NVARCHAR(100) NOT NULL,  
    Opis NVARCHAR(500),  
    Cena DECIMAL(10,2) NOT NULL,  
    Datum_od DATE NOT NULL,  
    Datum_do DATE,  
    -- Tekstualna polja ne smeju biti prazna.  
    CONSTRAINT CK_Cenovnik_Tekst CHECK (  
        LEN(LTRIM(RTRIM(Naziv))) > 0  
        AND (Opis IS NULL OR LEN(LTRIM(RTRIM(Opis))) > 0)  
    ),  
    -- Cena mora biti pozitivna.  
    CONSTRAINT CK_Cenovnik_Cena CHECK (Cena > 0),  
    -- Datum prestanka vazenja, ako postoji, ne sme biti pre datuma pocetka vazenja.  
    CONSTRAINT CK_Cenovnik_Datum CHECK (Datum_do IS NULL OR Datum_do >= Datum_od)  
);
```

Прилог 7.17 Дизајн табеле Cenovnik.

7.2.17. Uplata

Табела уплата дата је у Прилог 7.18. Она чува финансијске уплате кандидата и повезује их са конкретном обуком и ставком из ценовника. Износ уплате не сме бити нула, док је начин плаћања ограничен на вредности: готовина, картица и пренос. Датум уплате се подразумевано поставља на тренутни датум ако није ручно унет. Поља Kreiran_datum и Izmenjen_datum омогућавају праћење евидентирања и накнадних измена финансијских података.

```
CREATE TABLE Uplata (  
    Id INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Cenovnik_id INT NOT NULL REFERENCES Cenovnik (Id),  
    Obuka_id INT NOT NULL REFERENCES Obuka (Id),  
    Iznos DECIMAL(10,2) NOT NULL,  
    Datum DATE NOT NULL  
        CONSTRAINT DF_Uplata_Datum DEFAULT (getdate()),  
    Nacin_placanja NVARCHAR(20) NOT NULL,  
    Kreiran_datum DATETIME2 NOT NULL  
        CONSTRAINT DF_Uplata_Kreiran_datum DEFAULT (getdate()),  
    Izmenjen_datum DATETIME2,  
    -- Iznos ne sme biti nula; negativan iznos je dozvoljen kao korekcija.  
    CONSTRAINT CK_Uplata_Iznos CHECK (Iznos <> 0),  
    -- Nacin placanja moze biti samo jedna od dozvoljenih vrednosti.  
    CONSTRAINT CK_Uplata_Nacin_placanja CHECK (Nacin_placanja IN (N'Gotovina',  
N'Kartica', N'Prenos'))  
);
```

Прилог 7.18 Дизајн табеле Uplata.

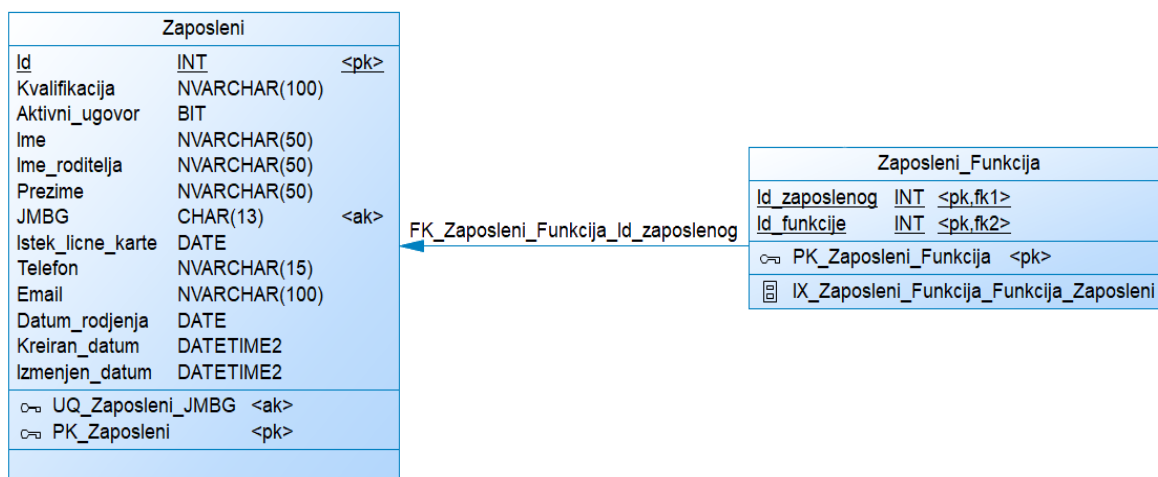
7.3. Дефинисање веза између табела

Везе између табела дефинисане су преко страних кључева у оквиру SQL скрипти. Оне омогућавају да се подаци из различитих делова система повежу у целину и да се спречи унос података који немају одговарајући запис у повезаној табели.

У наставку су описане везе приказане у физичком моделу. За сваку везу наведено је пословно значење, основна кардиналност и прилог у ком ће бити приказан графички приказ релације.

7.3.1. Zaposleni - Zaposleni_Funkcija

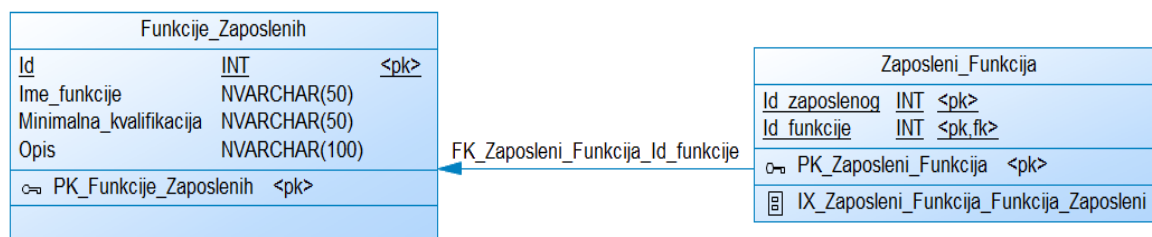
Радна улога запосленог није уписана директно у табелу Zaposleni, јер једно лице у аутошколи може имати више задужења. Зато се функције воде преко спојне табеле Zaposleni_Funkcija, где сваки запис припада једном запосленом и једној функцији. Овакав модел олакшава касније додавање нових улога без измене основне табеле запослених. Приказ релације означен је као Слика 7.1.



Слика 7.1 Приказ релације Zaposleni - Zaposleni_Funkcija.

7.3.2. Funkcije_Zaposlenih - Zaposleni_Funkcija

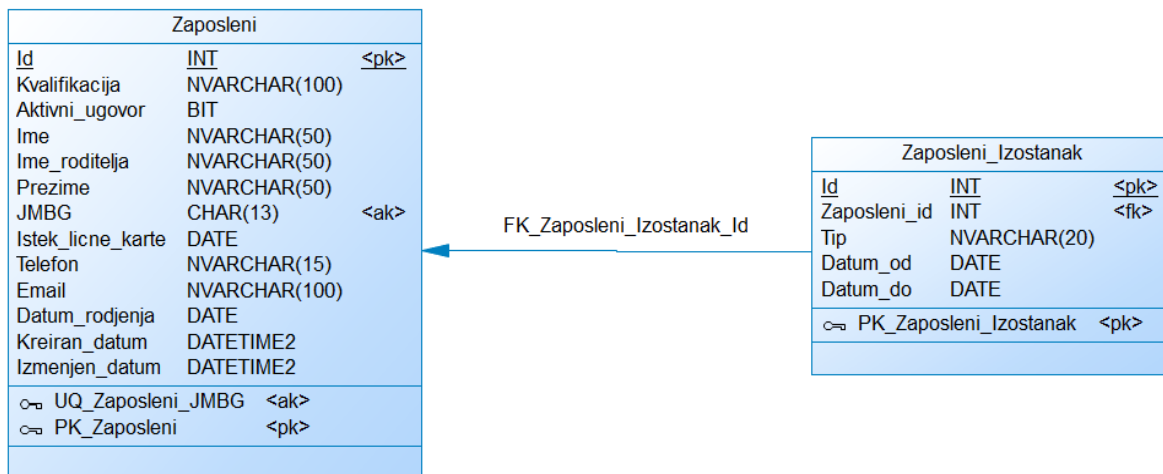
Функције запослених издвојене су у посебну табелу како се исти назив функције не би понављао код више лица. Веза са табелом Zaposleni_Funkcija омогућава да једна функција, на пример инструктор или предавач, буде додељена већем броју запослених. На овај начин је однос запослених и функција реализован као веза више-према-више. Приказ релације означен је као Слика 7.2.



Слика 7.2 Приказ релације Funkcije_Zaposlenih - Zaposleni_Funkcija.

7.3.3. Zaposleni - Zaposleni_Izostanak

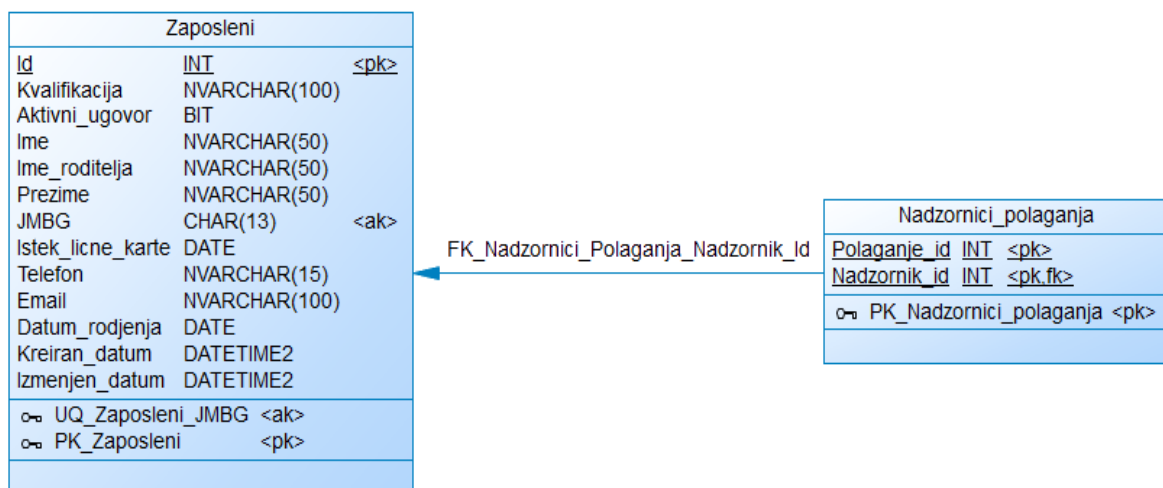
Изостанци су издвојени од основних података о запосленом, јер један запослени током времена може имати више периода одсуства. Сваки изостанак мора бити везан за конкретног запосленог, што спречава да у систему постоји одсуство без припадајућег лица. Оваква организација је погодна за праћење боловања и годишњих одмора. Приказ релације означен је као Слика 7.3.



Слика 7.3 Приказ релације Zaposleni - Zaposleni_Izostanak.

7.3.4. Zaposleni - Nadzornici_polaganja

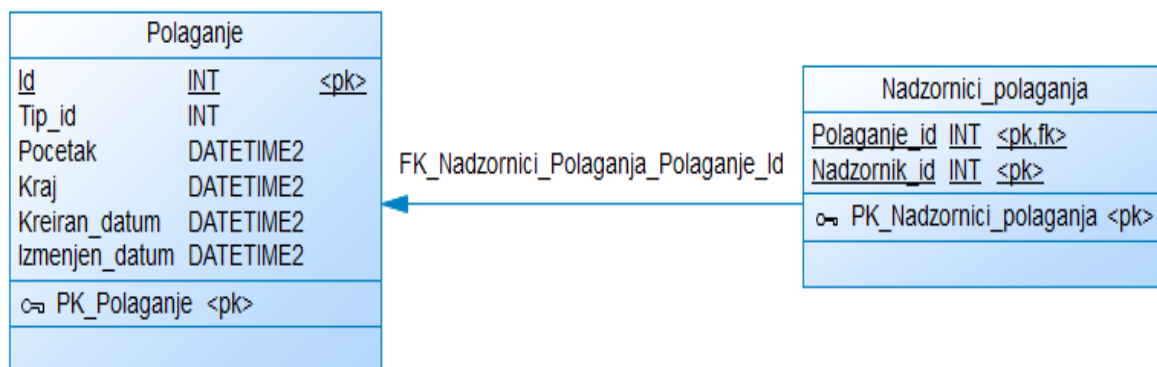
Надзорник полагања је у моделу представљен као запослени, а не као посебна независна целина. Због тога се преко табеле Nadzornici_polaganja бележи који је запослени додељен одређеном полагању. Један запослени може надзирати више полагања, али сваки запис о надзору мора имати тачно једног запосленог. Приказ релације означен је као Слика 7.4.



Слика 7.4 Приказ релације Zaposleni - Nadzornici_polaganja.

7.3.5. Polaganje - Nadzornici_polaganja

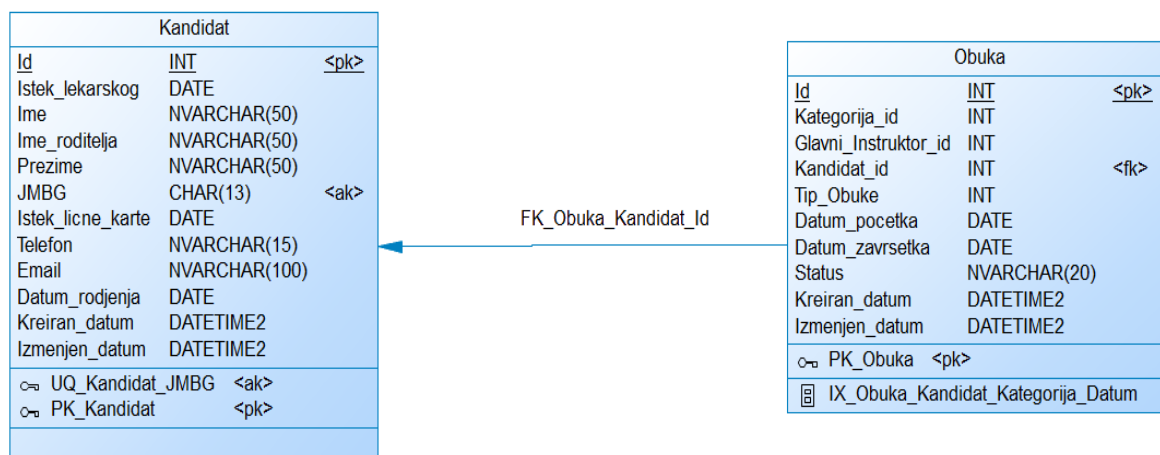
Једно полагање може захтевати више надзорника, па је тај однос решен преко спојне табеле **Nadzornici_polaganja**. Сваки запис у тој табели припада једном полагању и једном надзорнику. На тај начин се може евидентирати више надзорних лица за исти термин полагања, без понављања података о самом полагању. Приказ релације означен је као Слика 7.5.



Слика 7.5 Приказ релације Polaganje - Nadzornici_polaganja.

7.3.6. Kandidat - Obuka

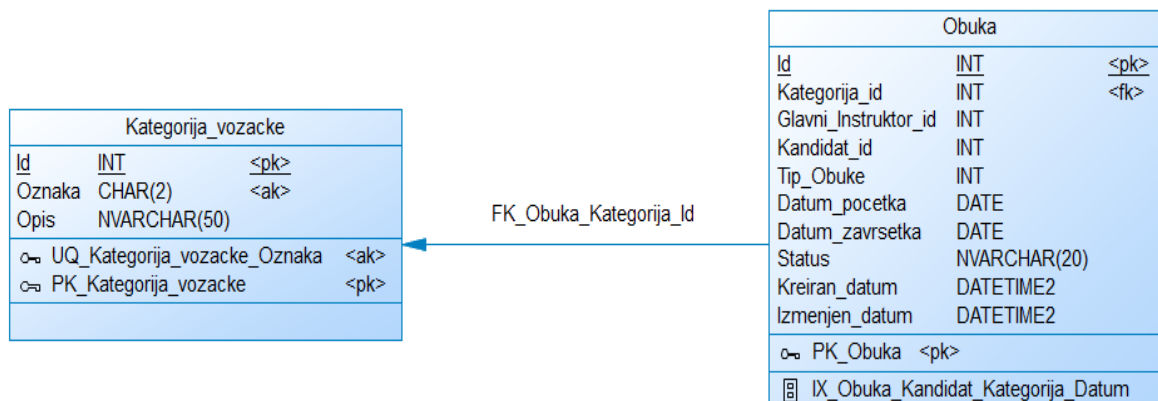
Веза између кандидата и обуке представља једну од основних веза у систему. Један кандидат може током времена похађати више обука, на пример за различите категорије или типове обуке, док свака обука мора припадати једном кандидату. Овим се ток обуке прати преко конкретног кандидата, а не као изолован запис. Приказ релације означен је као Слика 7.6.



Слика 7.6 Приказ релације Kandidat – Obuka.

7.3.7. Kategorija_vozacke - Obuka

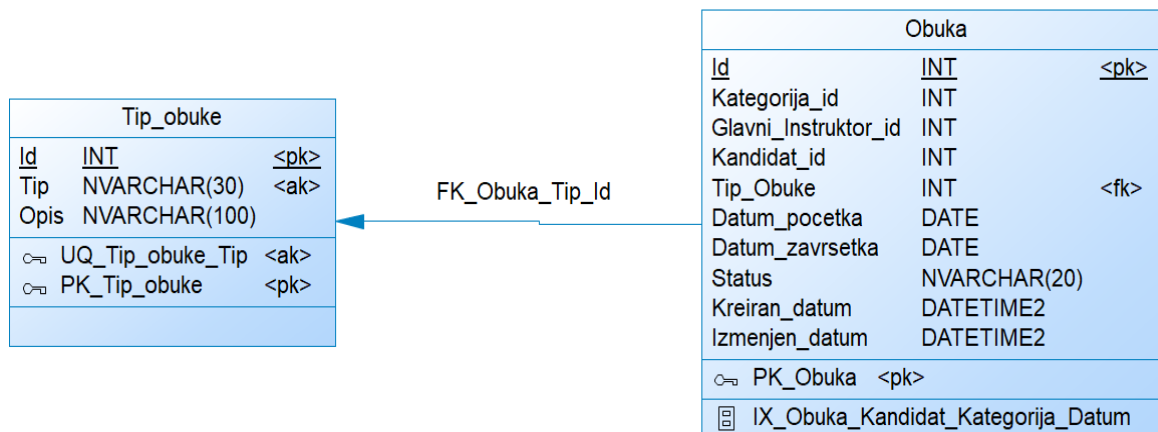
Категорија возачке дозволе издвојена је као шифарник, јер се исте категорије користе у више обука. Једна категорија може бити повезана са већим бројем обука, док свака обука мора имати тачно једну категорију. Ова веза јасно одређује за коју дозволу се кандидат обучава. Приказ релације означен је као Слика 7.7.



Слика 7.7 Приказ релације Kategorija_vozacke – Obuka.

7.3.8. Tip_obuke – Obuka

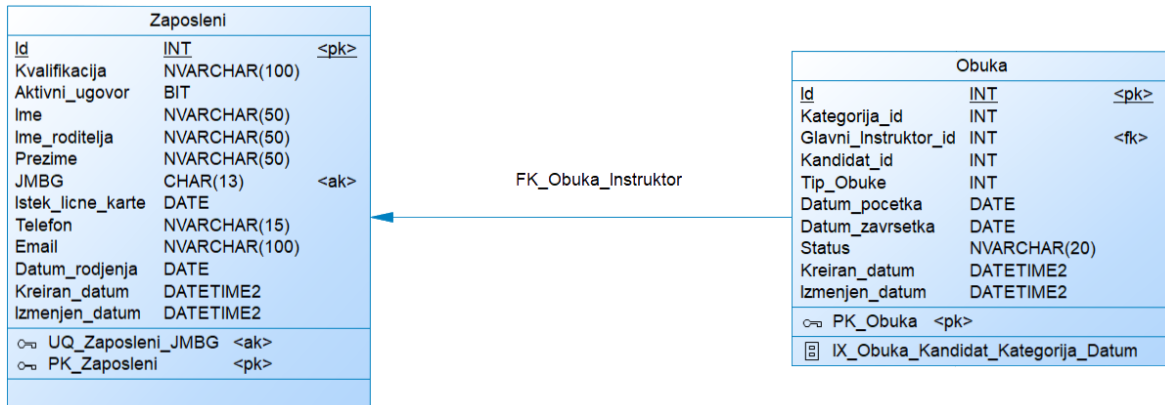
Тип обуке одређује да ли се обука односи на теорију, практичну наставу или прву помоћ. Пошто се исти тип појављује код више обука, издвојен је у табелу Tip_obuke. Свака обука мора имати један тип, чиме се избегава неуједначен унос назива типова обуке. Приказ релације означен је као Слика 7.8.



Слика 7.8 Приказ релације Tip_obuke – Obuka.

7.3.9. Zaposleni - Obuka

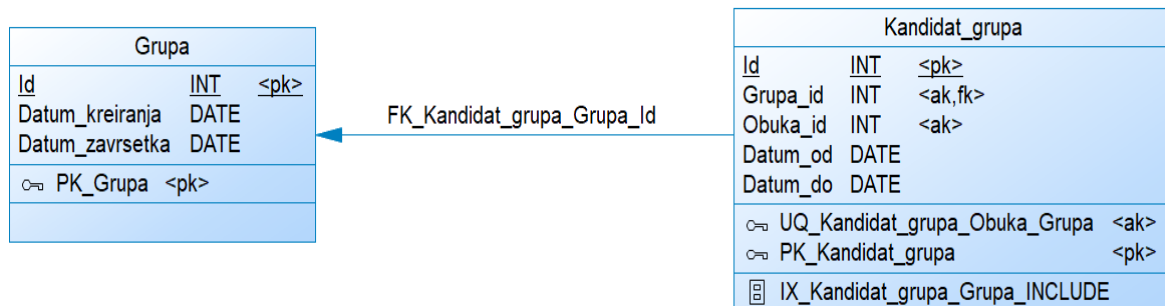
Главни инструктор је повезан са обуком преко запосленог који је задужен за њено вођење. Један запослени може бити главни инструктор на више обука, док обука може остати без инструктора ако он још није додељен. Оваква веза оставља простор да се обука креира пре коначне организације наставе. Приказ релације означен је као Слика 7.9.



Слика 7.9 Приказ релације Zaposleni – Obuka.

7.3.10. Grupa - Kandidat_grupa

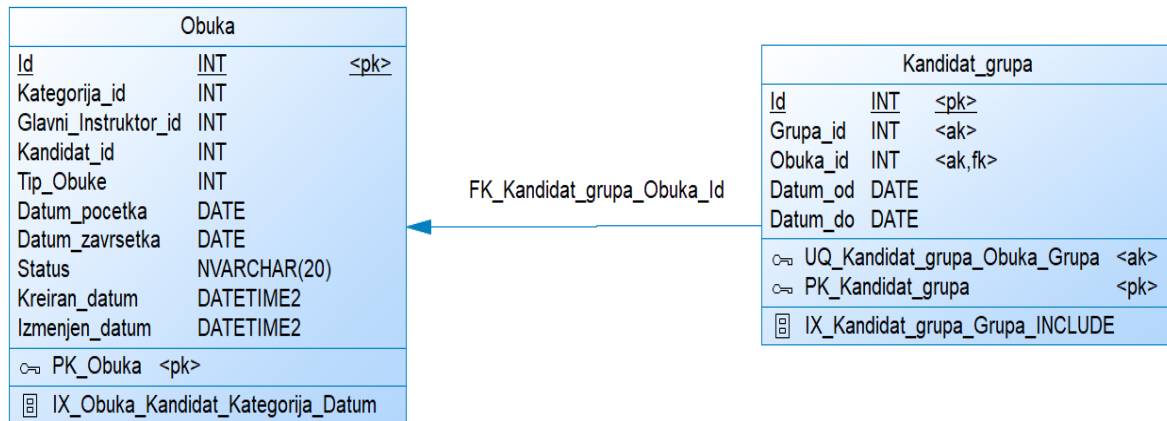
Група не садржи кандидате директно, већ се њен састав прати преко табеле Kandidat_grupa. Једна група може имати више чланстава, а свако чланство мора припадати једној групи. Овим се омогућава праћење састава групе кроз време, укључујући датум уласка и изласка из групе. Приказ релације означен је као Слика 7.10.



Слика 7.10 Приказ релације Grupa - Kandidat_grupa.

7.3.11. Obuka - Kandidat_grupa

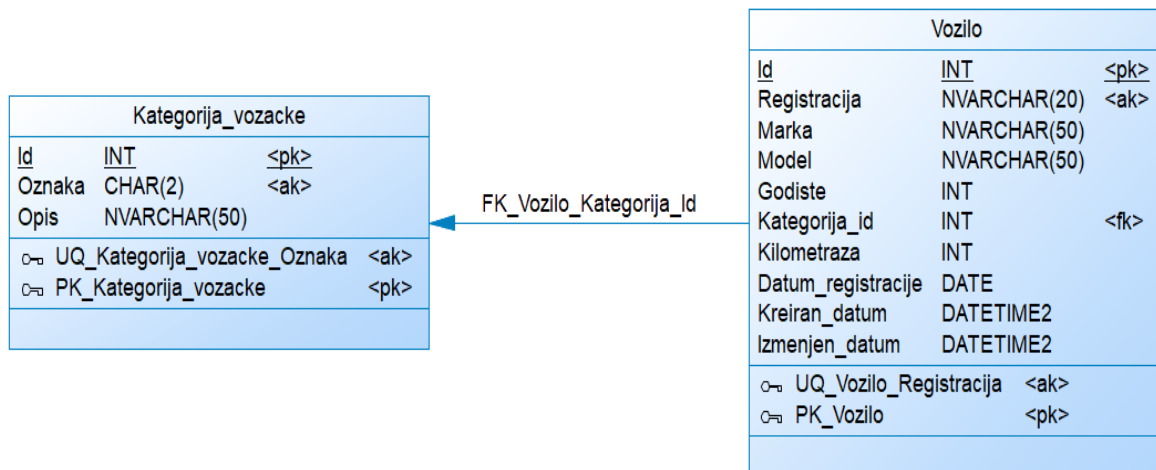
Кандидат се у групу не распоређује самостално, већ преко конкретне обуке коју похађа. Због тога је табела **Kandidat_grupa** повезана са табелом **Obuka**. Овај приступ је важан јер исти кандидат може имати више обука, па припадност групи треба посматрати у контексту одређене обуке. Приказ релације означен је као Слика 7.11.



Слика 7.11 Приказ релације **Obuka – Kandidat_grupa**.

7.3.12. Kategorija_vozacke - Vozilo

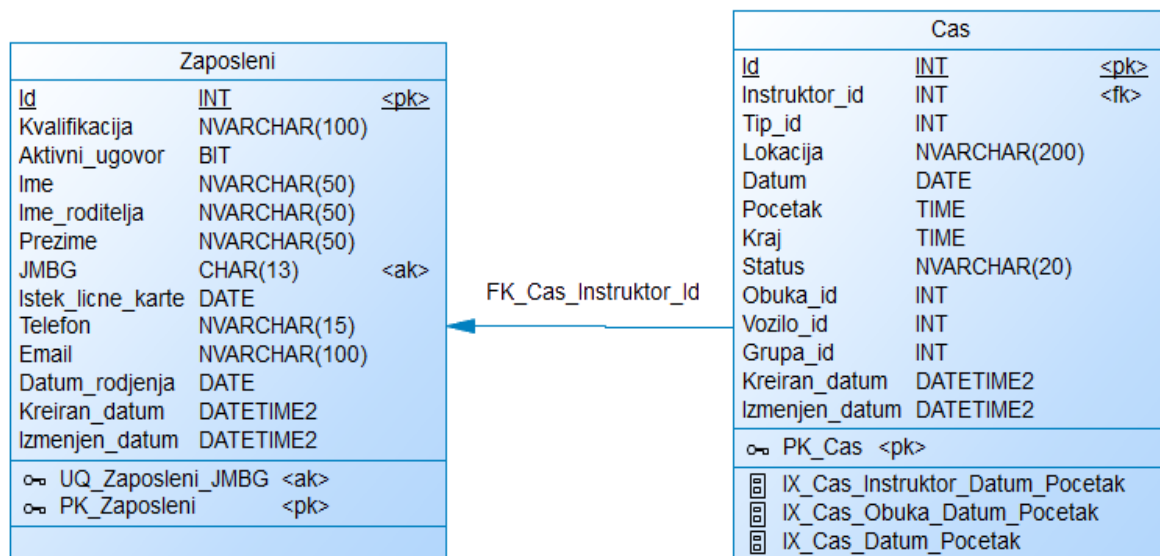
Возило је повезано са категоријом возачке дозволе за коју се користи у практичној обуци. Једна категорија може имати више возила, док свако возило мора припадати једној категорији. Ова веза помаже да се приликом планирања часа користи возило које одговара категорији обуке. Приказ релације означен је као Слика 7.12.



Слика 7.12 Приказ релације **Kategorija_vozacke – Vozilo**.

7.3.13. Zaposleni - Cas

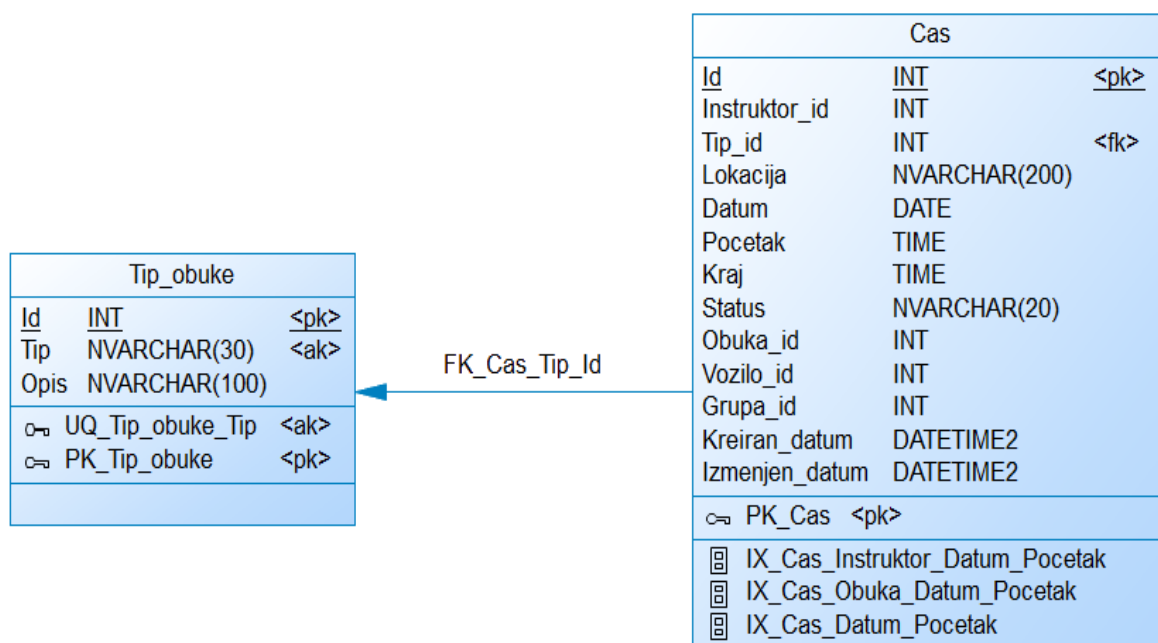
Сваки час мора имати инструктора који га води или је за њега задужен. Један запослени може водити више часова, па ова веза омогућава праћење распореда и оптерећења инструктора. Она је посебно важна код провере заузетости инструктора приликом заказивања новог часа. Приказ релације означен је као Слика 7.13.



Слика 7.13 Приказ релације Zaposleni – Cas.

7.3.14. Tip_obuke - Cas

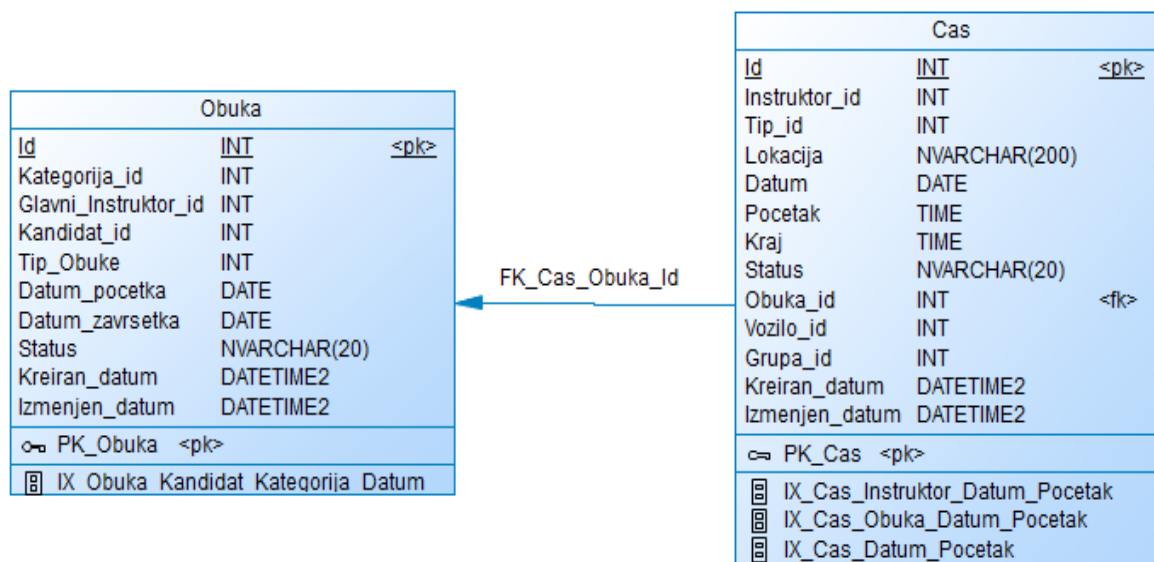
Тип часа одређује како се час тумачи у систему, на пример као теоријски, практични или час прве помоћи. Пошто се исти тип може појавити код више часова, податак се чува у табели Tip_obuke, а час садржи само везу ка том типу. Тако се обезбеђује доследно означавање врсте часа. Приказ релације означен је као Слика 7.14.



Слика 7.14 Приказ релације Tip_obuke – Cas.

7.3.15. Obuka - Cas

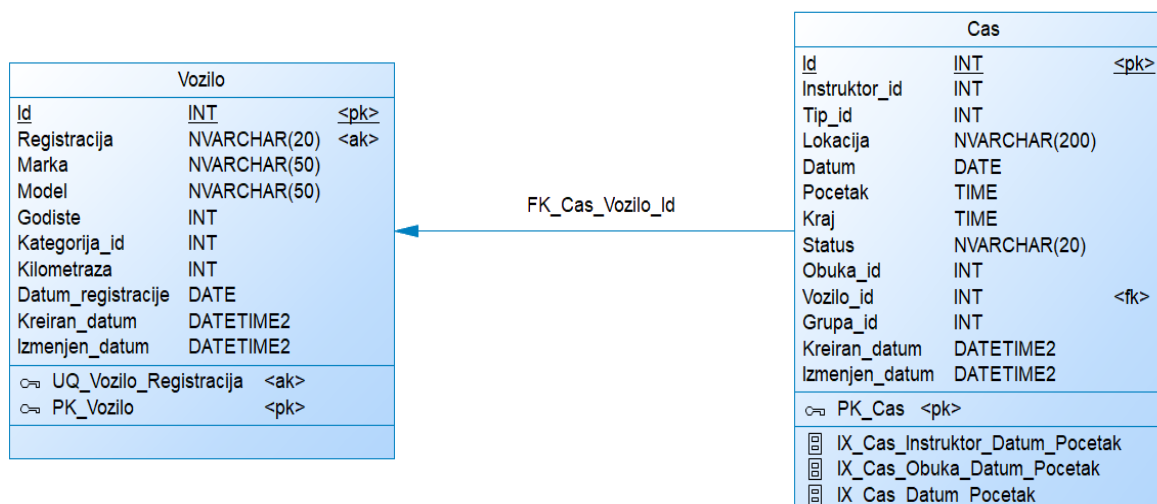
Час се повезује са конкретном обуком кандидата, јер се преко те везе прати напредак у обуци. Једна обука може имати више практичних часова, док један практични час припада једној обуци. Код групних часова ова веза није обавезна, јер се они воде преко групе. Приказ релације означен је као Слика 7.15.



Слика 7.15 Приказ релације Obuka – Cas.

7.3.16. Vozilo - Cas

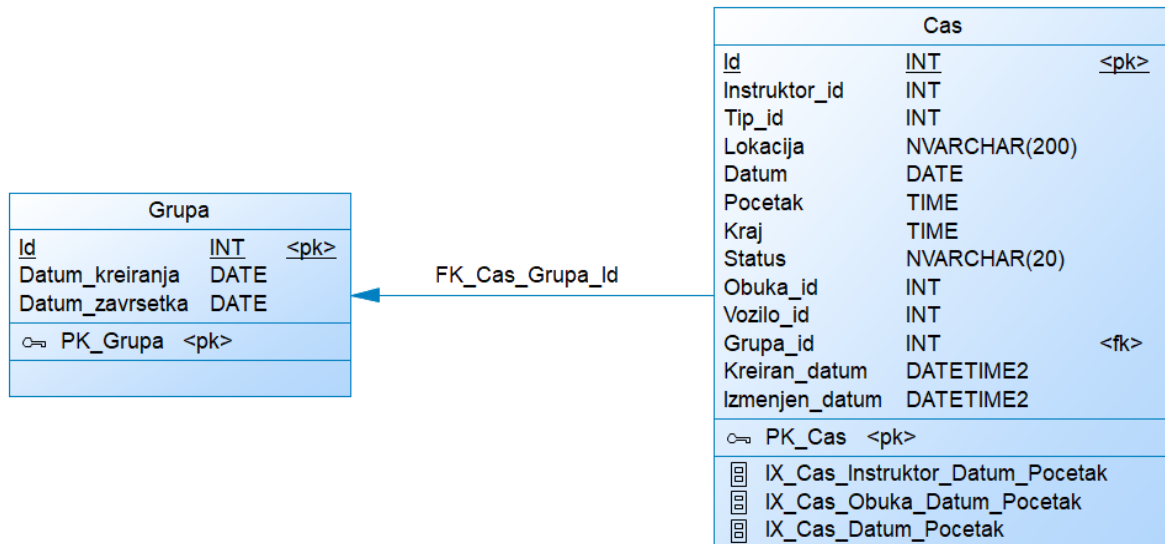
Возило се користи само код практичних часова, па је његова веза са табелом Cas условљена типом часа. Једно возило може бити ангажовано на више часова, али један практични час може имати једно возило. Ова релација је важна за проверу доступности возила у одређеном термину. Приказ релације означен је као Слика 7.16.



Слика 7.16 Приказ релације Vozilo – Cas.

7.3.17. Grupa - Cas

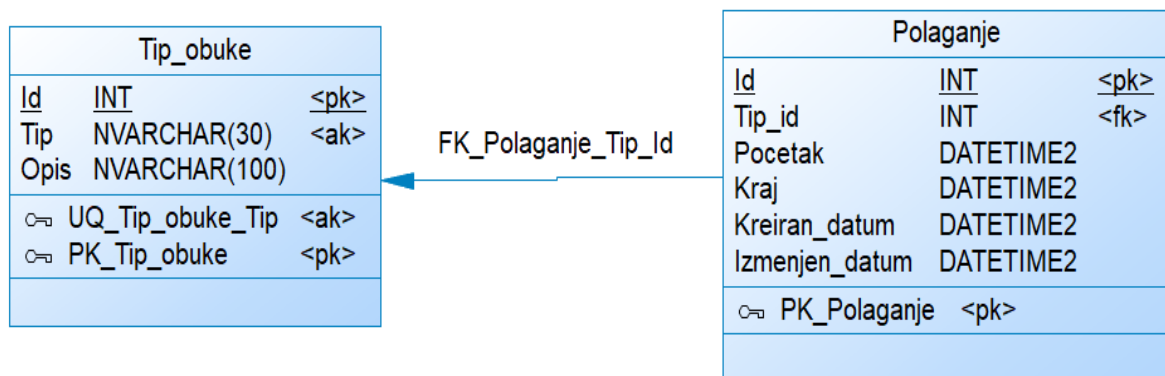
Групни часови се повезују са групом, а не са појединачном обуком кандидата. Једна група може имати више часова, док један групни час припада једној групи. Овим се теоријска настава и друге групне активности могу водити без понављања истог часа за сваког кандидата посебно. Приказ релације означен је као Слика 7.17.



Слика 7.17 Приказ релације Grupa – Cas.

7.3.18. Tip_obuke - Polaganje

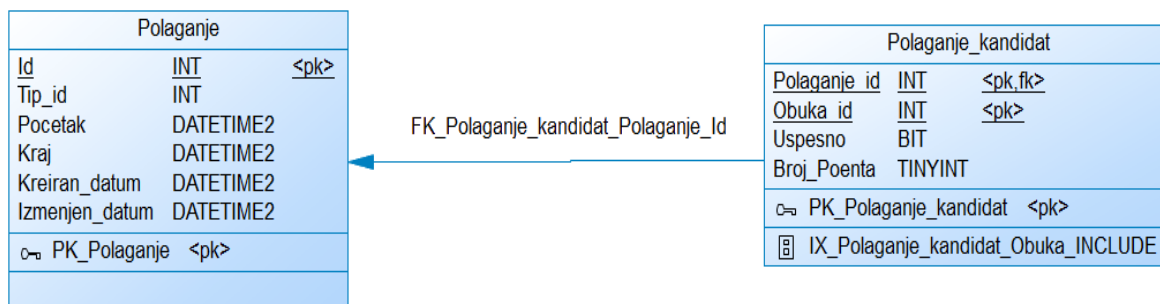
Полагање мора имати тип како би се знало да ли се односи на теорију, практичну обуку или прву помоћ. Један тип може бити коришћен на више термина полагања, док сваки термин има један тип. Ова веза омогућава да се резултати полагања касније тумаче у складу са врстом провере. Приказ релације означен је као Слика 7.18.



Слика 7.18 Приказ релације Tip_obuke – Polaganje.

7.3.19. Polaganje - Polaganje_kandidat

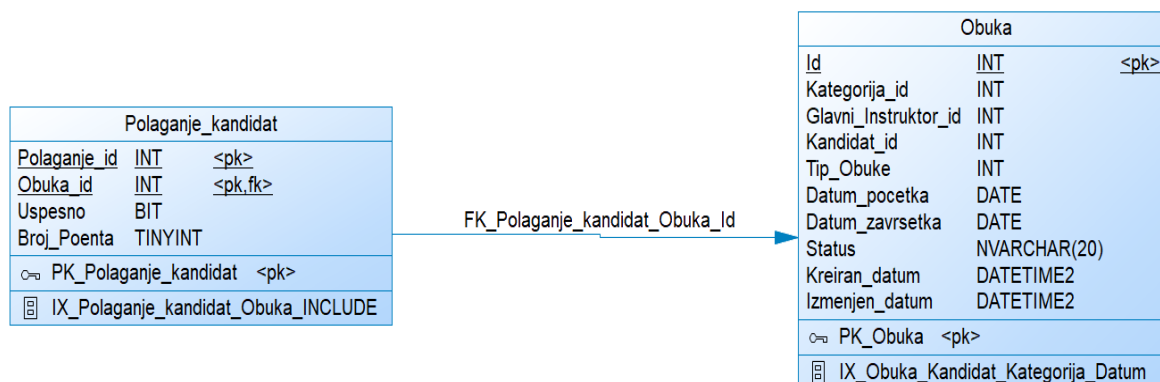
Један термин полагања може обухватити више кандидата, па се резултати не чувају директно у табели Polaganje. Уместо тога користи се табела Polaganje_kandidat, у којој сваки запис припада једном полагању и једној обуци кандидата. Овим се раздваја термин полагања од појединачног резултата кандидата. Приказ релације означен је као Слика 7.19.



Слика 7.19 Приказ релације Polaganje - Polaganje_kandidat.

7.3.20. Obuka - Polaganje_kandidat

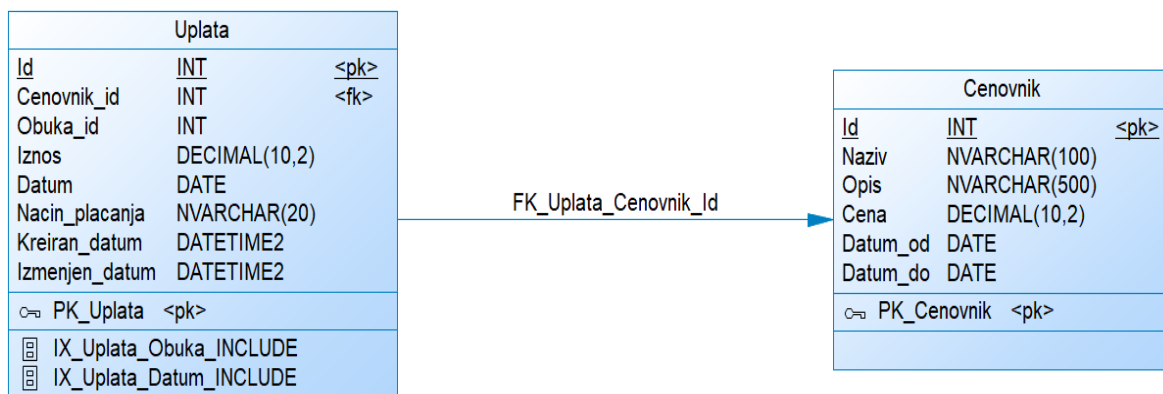
Резултат полагања везује се за обуку, а не само за кандидата, јер кандидат може имати више различитих обука. Једна обука може имати више покушаја полагања, док сваки запис резултата мора припадати једној обуци. На тај начин се тачно зна у оквиру које обуке је кандидат полагао. Приказ релације означен је као Слика 7.20.



Слика 7.20 Приказ релације Obuka - Polaganje_kandidat.

7.3.21. Cenovnik - Uplata

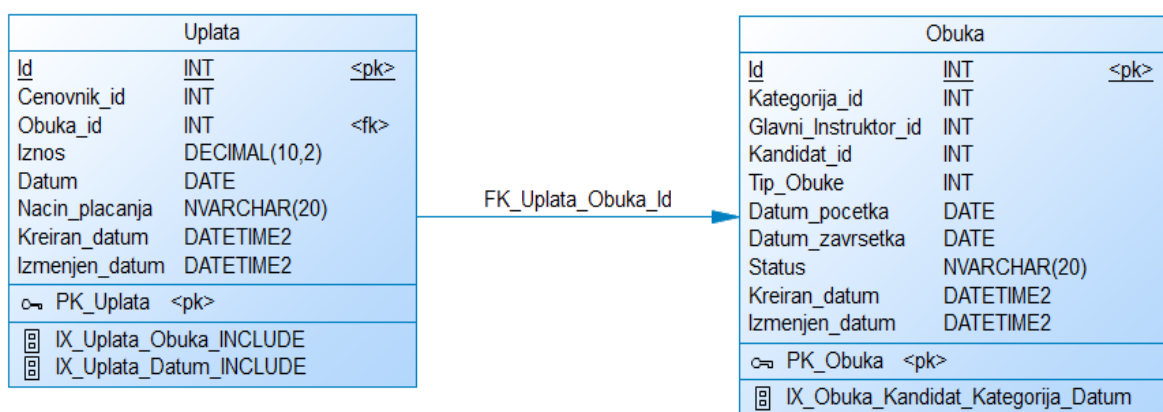
Свака уплата мора бити повезана са ставком ценовника, како би се знало на шта се уплата односи. Једна ставка ценовника може се појавити у више уплата, док уплата има једну изабрану ставку. Ова веза омогућава прецизније праћење финансијског дела обуке. Приказ релације означен је као Слика 7.21.



Слика 7.21 Приказ релације Cenovnik – Uplata.

7.3.22. Obuka - Uplata

Уплата се везује за конкретну обуку кандидата, јер финансијске обавезе зависе од обуке коју кандидат похађа. Једна обука може имати више уплата, док свака уплата мора припадати једној обуци. Овим се омогућава преглед плаћених и преосталих обавеза по обуци. Приказ релације означен је као Слика 7.22.



Слика 7.22 Приказ релације Obuka – Uplata.

7.4. Унос тест података

Тест подаци се налазе у фајлу: 06_test_podaci.sql. Они омогућавају да се након креирања базе провери да ли су табеле правилно повезане и да ли упити, погледи, функције, процедуре и тригери раде очекивано.

Подаци су припремљени тако да није потребно ручно уносити идентификаторе. Примарни кључеви се аутоматски генеришу као INT IDENTITY вредности, а скрипта користи већ унете податке како би правилно повезала кандидате, обуке, часове, уплате и полагања.

7.5. Погледи

Погледи су креирани у фајлу: 08_pogledi.sql са циљем да сложене упите над више повезаних табела представе као један именовани објекат базе података. На тај начин се често коришћена JOIN логика не мора писати више пута, већ се подаци могу читати директно из погледа као из табеле.

7.5.1. vw_Kandidati_Obuke

Поглед vw_Kandidati_Obuke приказује кандидате заједно са њиховим обукама. У резултату се виде подаци о кандидату, категорији возачке дозволе, типу обуке, статусу обуке и главном инструктору. Овај поглед је користан када је потребно брзо добити преглед обука без ручног повезивања табела Kandidat, Obuka, Kategorija_vozacke, Tip_obuke и Zaposleni.

```
CREATE OR ALTER VIEW dbo.vw_Kandidati_Obuke
AS
SELECT
    o.Id AS Obuka_id,
    k.Id AS Kandidat_id,
    k.Ime AS Kandidat_ime,
    k.Prezime AS Kandidat_prezime,
    k.JMBG AS Kandidat_JMBG,
    k.Telefon AS Kandidat_telefon,
    k.Email AS Kandidat_email,
    kv.Oznaka AS Kategorija,
    kv.Opis AS Kategorija_opis,
    t.Tip AS Tip_obuke,
    o.Datum_pocetka,
    o.Datum_zavrsetka,
    o.Status,
    z.Id AS Instruktor_id,
    z.Ime + N' ' + z.Prezime AS Glavni_instruktor
FROM Obuka o
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id
JOIN Tip_obuke t ON t.Id = o.Tip_Obuke
LEFT JOIN Zaposleni z ON z.Id = o.Glavni_Instruktor_id;
```

Прилог 7.19 Дефиниција погледа Kandidati_Obuke.

	Obuka_id	Kandidat_id	Kandidat_ime	Kandidat_prezime	Kandidat_JMBG	Kandidat_telefon	Kandidat_email	Kategorija	Kategorija_opis	Tip_obuke	Datum_pocetka	Datum_zavrsetka	Status	Instruktor_id	Glavni_instruktor
1	1	5	Luka	Nikolic	0505005710005	0605678901	luka.nikolic@example.com	C	Teretna vozila	Prakticna	2026-02-01	2026-03-05	Prekinut	1	Milos Savic
2	2	4	Ana	Ilic	1204004715004	0604567890	ana.ilic@example.com	B	Putnicka vozila	Teorijska	2026-05-05	NULL	Aktivan	2	Dejan Kostic
3	3	1	Marko	Markovic	0101000710001	0601234567	marko.markovic@example.com	B	Putnicka vozila	Prakticna	2026-04-05	NULL	Aktivan	2	Dejan Kostic
4	4	2	Jelena	Jovanovic	1502002715002	0602345678	jelena.jovanovic@example.com	B	Putnicka vozila	Prakticna	2026-04-10	NULL	Aktivan	3	Ivan Pavlovic
5	5	3	Nikola	Petrovic	2303003710003	0603456789	nikola.petrovic@example.com	A	Motocikli	Prakticna	2026-03-15	2026-05-20	Zavrsen	3	Ivan Pavlovic

Слика 7.23 Приказ резултата погледа Kandidati_Obuke.

7.5.2. vw_Raspored_Casova

Овај поглед приказује распоред часова. Он обједињује податке о датуму, времену, статусу, локацији, инструктору, кандидату, возилу и групи. Пошто часови могу бити практични или теоријски, у погледу су коришћена и LEFT JOIN повезивања како би се приказали подаци који постоје само за одређени тип часа.

```
CREATE OR ALTER VIEW dbo.vw_Raspored_Casova
AS
SELECT
    c.Id AS Cas_id,
    c.Datum,
    c.Pocetak,
    c.Kraj,
    c.Status AS Status_casa,
    c.Lokacija,
    t.Tip AS Tip_casa,
    z.Id AS Instruktor_id,
    z.Ime + N' ' + z.Prezime AS Instruktor,
    o.Id AS Obuka_id,
    k.Id AS Kandidat_id,
    k.Ime + N' ' + k.Prezime AS Kandidat,
    kv.Oznaka AS Kategorija,
    v.Registracija,
    v.Marka,
    v.Model,
    g.Id AS Grupa_id,
    g.Datum_kreiranja AS Datum_kreiranja_grupe
FROM Cas c
JOIN Tip_obuke t ON t.Id = c.Tip_id
JOIN Zaposleni z ON z.Id = c.Instruktor_id
LEFT JOIN Obuka o ON o.Id = c.Obuka_id
LEFT JOIN Kandidat k ON k.Id = o.Kandidat_id
LEFT JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id
LEFT JOIN Vozilo v ON v.Id = c.Vozilo_id
LEFT JOIN Grupa g ON g.Id = c.Grupa_id;
```

Прилог 7.20 Дефиниција погледа Raspored_Casova.

Results		Messages																	
	Cas_id	Datum	Pocetak	Kraj	Status_casa	Lokacija	Tip_casa	Instruktor_id	Instruktor	Obuka_id	Kandidat_id	Kandidat	Kategorija	Registracija	Marka	Model	Grupa_id	Datum_kreiranja_grupe	
1	1	2026-05-06	17:00:00.0000000	18:30:00.0000000	Odrzan	Ucionica 1	Teorijska	5	Stefan Lazic	NULL	NULL	NULL	NULL	NULL	NULL	NULL	2	2026-05-01	
2	2	2026-05-10	16:00:00.0000000	18:00:00.0000000	Zakazan	Ucionica prve pomoci	Prva Pomoc	5	Stefan Lazic	NULL	NULL	NULL	NULL	NULL	NULL	NULL	1	2026-04-01	
3	3	2026-05-07	09:00:00.0000000	10:30:00.0000000	Odrzan	Poligon Novi Beograd	Prakticna	2	Dejan Kostic	3	1	Marko Markovic	B	BG-123-AA	Toyota	Yaris	NULL	NULL	
4	4	2026-05-08	11:00:00.0000000	12:30:00.0000000	Odrzan	Gradska voznja	Prakticna	3	Ivan Pavlovic	4	2	Jelena Jovanovic	B	NS-456-BB	Volkswagen	Golf	NULL	NULL	
5	5	2026-05-09	13:00:00.0000000	14:30:00.0000000	Otkazan	Poligon	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
6	6	2026-06-02	09:00:00.0000000	09:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
7	7	2026-06-03	10:00:00.0000000	10:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
8	8	2026-06-04	11:00:00.0000000	11:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
9	9	2026-06-05	12:00:00.0000000	12:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
10	10	2026-06-06	13:00:00.0000000	13:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
11	11	2026-06-07	14:00:00.0000000	14:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
12	12	2026-06-08	15:00:00.0000000	15:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
13	13	2026-06-09	08:00:00.0000000	08:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
14	14	2026-06-10	09:00:00.0000000	09:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
15	15	2026-06-11	10:00:00.0000000	10:45:00.0000000	Otkazan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
16	16	2026-06-12	11:00:00.0000000	11:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
17	17	2026-06-13	12:00:00.0000000	12:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	
18	18	2026-06-14	13:00:00.0000000	13:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	3	Ivan Pavlovic	5	3	Nikola Petrovic	A	BG-789-CC	Yamaha	MT-07	NULL	NULL	

Слика 7.24 Приказ резултата погледа Raspored_Casova.

7.5.3. vw_Uplate_Po_Obuci

Уместо ручног повезивања више табела, овај поглед даје припремљен приказ финансијског прегледа обуке. Он групише уплате по обуци и приказује број уплата, укупан уплаћен износ, прву и последњу уплату. Овај поглед је користан јер администрацији омогућава да без додатног рачунања види стање уплата за сваку обуку.

```
CREATE OR ALTER VIEW dbo.vw_Uplate_Po_Obuci
AS
SELECT
    o.Id AS Obuka_id,
    k.Id AS Kandidat_id,
    k.Ime + N' ' + k.Prezime AS Kandidat,
    k.JMBG,
    kv.Oznaka AS Kategorija,
    o.Status AS Status_obuke,
    COUNT(u.Id) AS Broj_uplata,
    COALESCE(SUM(u.Iznos), 0) AS Ukupno_uplaceno,
    MIN(u.Datum) AS Prva_uplata,
    MAX(u.Datum) AS Poslednja_uplata
FROM Obuka o
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id
LEFT JOIN Uplata u ON u.Obuka_id = o.Id
GROUP BY
    o.Id,
    k.Id,
    k.Ime,
    k.Prezime,
    k.JMBG,
    kv.Oznaka,
    o.Status;
```

Прилог 7.21 Дефиниција погледа Uplate_Po_Obuci.

	Obuka_id	Kandidat_id	Kandidat	JMBG	Kategorija	Status_obuke	Broj_uplata	Ukupno_uplaceno	Prva_uplata	Poslednja_uplata
1	1	5	Luka Nikolic	0505005710005	C	Prekinut	201	540000.00	2026-02-01	2028-01-22
2	2	4	Ana Ilic	1204004715004	B	Aktivan	201	520000.00	2026-05-06	2028-01-22
3	3	1	Marko Markovic	0101000710001	B	Aktivan	202	555000.00	2026-04-05	2028-01-22
4	4	2	Jelena Jovanovic	1502002715002	B	Aktivan	201	585000.00	2026-04-10	2028-01-22
5	5	3	Nikola Petrovic	2303003710003	A	Zavrsen	201	570000.00	2026-03-15	2028-01-22

Слика 7.25 Приказ резултата погледа Uplate_Po_Obuci.

7.5.4. vw_Rezultati_Polaganja

Употребом овог погледа приказујемо резултате полагања кандидата. Он повезује полагање, обуку, кандидата, категорију и тип полагања. Додатно се кроз CASE израз текстуално приказује резултат, па се вредности из базе лакше читају као положио, није положио или није евидентирано.

```
CREATE OR ALTER VIEW dbo.vw_Rezultati_Polaganja
AS
SELECT
    p.Id AS Polaganje_id,
    p.Pocetak,
    p.Kraj,
    t.Tip AS Tip_polaganja,
    o.Id AS Obuka_id,
    k.Id AS Kandidat_id,
    k.Ime + N' ' + k.Prezime AS Kandidat,
    k.JMBG,
    kv.Oznaka AS Kategorija,
    pk.Uspesno,
    pk.Broj_Poenta,
    CASE
        WHEN pk.Uspesno IS NULL THEN N'Niје evidentirano'
        WHEN pk.Uspesno = 1 THEN N'Polozio'
        ELSE N'Niје polozio'
    END AS Rezultat
FROM Polaganje_kandidat pk
JOIN Polaganje p ON p.Id = pk.Polaganje_id
JOIN Tip_obuke t ON t.Id = p.Tip_id
JOIN Obuka o ON o.Id = pk.Obuka_id
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id;
```

Прилог 7.22 Дефиниција погледа Rezultati_Polaganja.

Results Messages

	Polaganje_id	Pocetak	Kraj	Tip_polaganja	Obuka_id	Kandidat_id	Kandidat	JMBG	Kategorija	Uspesno	Broj_Poenta	Rezultat
1	1	2026-05-20 09:00:00.0000000	2026-05-20 10:00:00.0000000	Teorijska	3	1	Marko Markovic	0101000710001	B	1	92	Polozio
2	1	2026-05-20 09:00:00.0000000	2026-05-20 10:00:00.0000000	Teorijska	4	2	Jelena Jovanovic	1502002715002	B	0	63	Nije polozio
3	2	2026-05-25 08:00:00.0000000	2026-05-25 12:00:00.0000000	Practicna	5	3	Nikola Petrovic	2303003710003	A	1	95	Polozio
4	3	2026-06-01 10:00:00.0000000	2026-06-01 11:00:00.0000000	Teorijska	2	4	Ana Ilic	1204004715004	B	NULL	NULL	Nije evidentirano

Слика 7.26 Приказ резултата погледа Rezultati_Polaganja.

7.5.5. vw_Zaposleni_Funkcije

Поглед vw_Zaposleni_Funkcije приказује запослене и функције које имају у аутошколи. Пошто је однос запослених и функција реализован преко спојне табеле Zaposleni_Funkcija, овај поглед поједностављује читање тих података и приказује их као једну целину.

```
CREATE OR ALTER VIEW dbo.vw_Zaposleni_Funkcije
AS
SELECT
    z.Id AS Zaposleni_id,
    z.Ime,
    z.Prezime,
    z.JMBG,
    z.Telefon,
    z.Email,
    z.Kvalifikacija,
    z.Aktivni_ugovor,
    f.Id AS Funkcija_id,
    f.Ime_funkcije,
    f.Minimalna_kvalifikacija,
    f.Opis AS Opis_funkcije
FROM Zaposleni_Funkcija zf
JOIN Zaposleni z ON z.Id = zf.Id_zaposlenog
JOIN Funkcije_Zaposlenih f ON f.Id = zf.Id_funkcije;
```

Прилог 7.23 Дефиниција погледа Zaposleni_Funkcije.

	Zaposleni_id	Ime	Prezime	JMBG	Telefon	Email	Kvalifikacija	Aktivni_ugovor	Funkcija_id	Ime_funkcije	Minimalna_kvalifikacija	Opis_funkcije
1	1	Milos	Savic	0202000710006	0611111111	milos.savic@autoskolatest.rs	Diplomirani inženjer saobraćaja	1	1	Direktor	VII stepen	Organizacija rada auto škole
2	2	Dejan	Kostic	0303000710007	0612222222	dejan.kostic@autoskolatest.rs	Instruktor B kategorije	1	2	Instruktor	Instruktor vožnje	Izводи практичне часове
3	3	Ivan	Pavlovic	0404000710008	0613333333	ivan.pavlovic@autoskolatest.rs	Instruktor A i B kategorije	1	2	Instruktor	Instruktor vožnje	Izводи практичне часове
4	4	Marija	Ristic	0505000715009	0614444444	marija.istic@autoskolatest.rs	Administrativni radnik	1	4	Administrator	SSS	Vodi evidenciju kandidata i uplata
5	5	Stefan	Lazic	0606000710010	0615555555	stefan.lazic@autoskolatest.rs	Ispitivač teorije	1	3	Predavac teorije	Licenca za teorijsku nastavu	Izводи teorijsku nastavu
6	5	Stefan	Lazic	0606000710010	0615555555	stefan.lazic@autoskolatest.rs	Ispitivač teorije	1	5	Nadzornik polaganja	Licenca ispitivaca	Nadzire polaganja kandidata

Слика 7.27 Приказ резултата погледа Zaposleni_Funkcije.

7.5.6. vw_Stanje_Obuke

Овај приказ обједињује збирно стање обуке кроз број часова, број полагања и укупан уплаћен износ. У њему су коришћени СТЕ изрази како би се посебно израчунали часови, полагања и уплате, а затим приказали уз основне податке о обуци. Овај поглед даје најшири преглед напретка кандидата кроз обуку.

```
CREATE OR ALTER VIEW dbo.vw_Stanje_Obuke
AS
WITH Casovi AS (
    SELECT Obuka_id, COUNT(*) AS Broj_casova
    FROM Cas
    WHERE Obuka_id IS NOT NULL
    GROUP BY Obuka_id
),
Polaganja AS (
    SELECT Obuka_id, COUNT(*) AS Broj_polaganja
    FROM Polaganje_kandidat
    GROUP BY Obuka_id
),
Uplate AS (
    SELECT Obuka_id, SUM(Iznos) AS Ukupno_uplaceno
    FROM Uplata
    GROUP BY Obuka_id
)
SELECT
    o.Id AS Obuka_id,
    k.Ime + N' ' + k.Prezime AS Kandidat,
    k.JMBG,
    kv.Oznaka AS Kategorija,
    t.Tip AS Tip_obuke,
    o.Status,
    o.Datum_pocetka,
    o.Datum_zavrsetka,
    COALESCE(c.Broj_casova, 0) AS Broj_casova,
```

```
COALESCE(p.Broj_polaganja, 0) AS Broj_polaganja,
COALESCE(u.Ukupno_uplaceno, 0) AS Ukupno_uplaceno
FROM Obuka o
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id
JOIN Tip_obuke t ON t.Id = o.Tip_Obuke
LEFT JOIN Casovi c ON c.Obuka_id = o.Id
LEFT JOIN Polaganja p ON p.Obuka_id = o.Id
LEFT JOIN Uplate u ON u.Obuka_id = o.Id;
```

Прилог 7.24 Дефиниција погледа Stanje_Obuke.

	Obuka_id	Kandidat	JMBG	Kategorija	Tip_obuke	Status	Datum_pocetka	Datum_zavrsetka	Broj_casova	Broj_polaganja	Ukupno_uplaceno
1	1	Luka Nikolic	0505005710005	C	Prakticna	Prekinut	2026-02-01	2026-03-05	600	0	540000.00
2	2	Ana Ilic	1204004715004	B	Teorijska	Aktivan	2026-05-05	NULL	600	1	520000.00
3	3	Marko Markovic	0101000710001	B	Prakticna	Aktivan	2026-04-05	NULL	601	1	555000.00
4	4	Jelena Jovanovic	1502002715002	B	Prakticna	Aktivan	2026-04-10	NULL	601	1	585000.00
5	5	Nikola Petrovic	2303003710003	A	Prakticna	Zavrsen	2026-03-15	2026-05-20	601	1	570000.00

Слика 7.28 Приказ резултата погледа Stanje_Obuke.

7.6. Функције

Функције су креиране у фајлу 09_funkcije.sql. Оне враћају израчунату вредност или табеларни резултат који се може користити у другим упитима, погледима или процедурама.

У систему су коришћене функције за израчунавање уплата, броја часова, броја полагања, проверу активне обуке и приказ распореда инструктора.

7.6.1. fn_UkupnoUplacenoZaObuku

Ова функција користи се када је потребно проверити колико је новца уплаћено за одређену обуку. На основу прослеђеног идентификатора обуке читају се уплате из табеле Uplata и сабирају њихови износи. Посебно је важно што функција враћа вредност 0 и у случају да за ту обуку још нема евидентираних уплата, јер се на тај начин избегава приказ NULL вредности у резултату.

```
CREATE OR ALTER FUNCTION dbo.fn_UkupnoUplacenoZaObuku
(
    @Obuka_id INT
)
RETURNS DECIMAL(10, 2)
AS
BEGIN
    DECLARE @Ukupno DECIMAL(10, 2);

    SELECT @Ukupno = COALESCE(SUM(u.Iznos), 0)
    FROM Uplata u
    WHERE u.Obuka_id = @Obuka_id;

    RETURN COALESCE(@Ukupno, 0);
END;
```

Прилог 7.25 Дефиниција функције UkupnoUplacenoZaObuku.

```
SELECT dbo.fn_UkupnoUplacenoZaObuku(2) AS Ukupno_uplaceno;
```

Прилог 7.26 Наредба за позивање функције UkupnoUplacenoZaObuku.

	Ukupno_uplaceno
1	520000.00

Слика 7.29 Резултат функције UkupnoUplacenoZaObuku.

7.6.2. fn_BrojCasovaZaObuku

Број часова у оквиру једне обуке добија се помоћу функције `fn_BrojCasovaZaObuku`. Поред идентификатора обуке, функцији се прослеђује и параметар `Samo_odrzani`, који одређује да ли се рачунају сви часови или само часови са статусом `Odrzan`. Ово је корисно јер се исти објекат може употребити и за укупан број заказаних часова и за преглед стварно одржаних часова.

```
CREATE OR ALTER FUNCTION dbo.fn_BrojCasovaZaObuku
(
    @Obuka_id INT,
    @Samo_odrzani BIT
)
RETURNS INT
AS
BEGIN
    DECLARE @Broj INT;

    SELECT @Broj = COUNT(*)
    FROM Cas c
    WHERE c.Obuka_id = @Obuka_id
    AND (@Samo_odrzani = 0 OR c.Status = N'Odrzan');

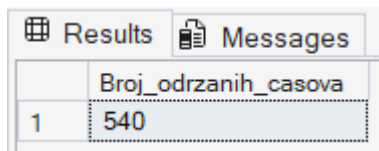
    RETURN COALESCE(@Broj, 0);
END;
```

Прилог 7.27 Дефиниција функције `BrojCasovaZaObuku`.

```
SELECT dbo.fn_BrojCasovaZaObuku(1, 1) AS Broj_odrzanih_casova;
```

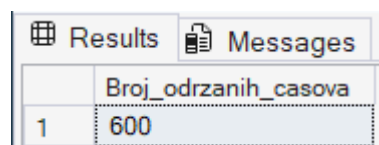
Прилог 7.28 Наредба за позивање функције `BrojCasovaZaObuku` са параметром `Samo_odrzani`.

```
SELECT dbo.fn_BrojCasovaZaObuku(1, 0) AS Broj_odrzanih_casova;
```



	Broj_odrzanih_casova
1	540

Слика 7.30 Резултат функције `BrojCasovaZaObuku` са параметром `Samo_odrzani`.



	Broj_odrzanih_casova
1	600

Слика 7.31 Резултат функције `BrojCasovaZaObuku` без параметра `Samo_odrzani`.

Прилог 7.29 Наредба за позивање функције `BrojCasovaZaObuku` без параметра `Samo_odrzani`.

7.6.3. fn_BrojPolaganjaZaObuku

Ова функција броји колико пута је кандидат излазио на полагање у оквиру једне обуке. Као улазни податак користи се идентификатор обуке, а број полагања се добија пребројавањем записа у табели `Polaganje_kandidat`. Та табела је важна јер повезује конкретну обуку са полагањима која су за њу евидентирана.

```
CREATE OR ALTER FUNCTION dbo.fn_BrojPolaganjaZaObuku
```

```
(
    @Obuka_id INT
)
RETURNS INT
AS
BEGIN
    DECLARE @Broj INT;

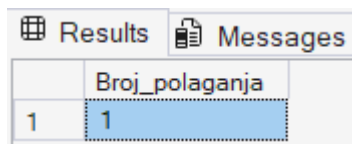
    SELECT @Broj = COUNT(*)
    FROM Polaganje_kandidat pk
    WHERE pk.Obuka_id = @Obuka_id;

    RETURN COALESCE(@Broj, 0);
END;
GO
```

Прилог 7.30 Дефиниција функције BrojPolaganjaZaObuku.

```
SELECT dbo.fn_BrojPolaganjaZaObuku(4) AS Broj_polaganja;
```

Прилог 7.31 Наредба за позивање функције BrojPolaganjaZaObuku.



	Broj_polaganja
1	1

Слика 7.32 Резултат функције BrojPolaganjaZaObuku.

7.6.4. fn_KandidatImaAktivnuObuku

Пре отварања нове обуке потребно је проверити да ли кандидат већ има активну обуку за исту категорију. Управо ту проверу обавља функција `fn_KandidatImaAktivnuObuku`, која као резултат враћа BIT вредност. Ако функција врати вредност 1, кандидат већ има активну обуку, па процедура за упис може да заустави додавање новог записа и тако спречи дуплирање активних обука.

```
CREATE OR ALTER FUNCTION dbo.fn_KandidatImaAktivnuObuku
(
    @Kandidat_id INT,
    @Kategorija_id INT
)
RETURNS BIT
AS
BEGIN
    DECLARE @Postoji BIT = 0;

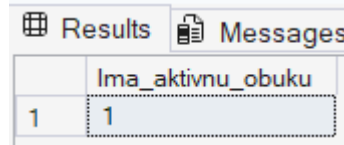
    IF EXISTS (
        SELECT 1
        FROM Obuka o
        WHERE o.Kandidat_id = @Kandidat_id
              AND o.Kategorija_id = @Kategorija_id
              AND o.Status = N'Aktivan'
    )
    BEGIN
        SET @Postoji = 1;
    END;

    RETURN @Postoji;
END;
```

Прилог 7.32 Дефиниција функције KandidatImaAktivnuObuku.

```
SELECT dbo.fn_KandidatImaAktivnuObuku(1, 2) AS Ima_aktivnu_obuku;
```

Прилог 7.33 Наредба за позивање функције KandidatImaAktivnuObuku.



	Ima_aktivnu_obuku
1	1

Слика 7.33 Резултат функције KandidatImaAktivnuObuku.

7.6.5. fn_RasporedInstruktora

Распоред инструктора представља скуп више повезаних података, па је за његов приказ употребљена табеларна функција `fn_RasporedInstruktora`. Она враћа редове са подацима о часовима у задатом периоду, укључујући датум, време, статус, локацију, кандидата и возило. За разлику од скаларних функција, које враћају једну вредност, ова функција враћа резултат у облику табеле и може се користити у `SELECT` упиту као извор података.

```
CREATE OR ALTER FUNCTION dbo.fn_RasporedInstruktora
(
    @Instruktor_id INT,
    @Datum_od DATE,
    @Datum_do DATE
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        c.Id AS Cas_id,
        c.Datum,
        c.Pocetak,
        c.Kraj,
        c.Status,
        c.Lokacija,
        t.Tip AS Tip_casa,
        o.Id AS Obuka_id,
        k.Ime + N' ' + k.Prezime AS Kandidat,
        v.Registracija
    FROM Cas c
    JOIN Tip_obuke t ON t.Id = c.Tip_id
    LEFT JOIN Obuka o ON o.Id = c.Obuka_id
    LEFT JOIN Kandidat k ON k.Id = o.Kandidat_id
    LEFT JOIN Vozilo v ON v.Id = c.Vozilo_id
    WHERE c.Instruktor_id = @Instruktor_id
        AND c.Datum BETWEEN @Datum_od AND @Datum_do
);
```

Прилог 7.34 Дефиниција табеларне функције RasporedInstruktora.

```
SELECT *
FROM dbo.fn_RasporedInstruktora(1, '2026-06-01', '2026-06-30')
```

Прилог 7.35 Наредба за позивање табеларне функције RasporedInstruktora.

	Cas_id	Datum	Pocetak	Kraj	Status	Lokacija	Tip_casa	Obuka_id	Kandidat	Registracija
1	2406	2026-06-02	09:00:00.0000000	09:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	1	Luka Nikolic	KG-321-DD
2	2407	2026-06-03	10:00:00.0000000	10:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	1	Luka Nikolic	KG-321-DD
3	2408	2026-06-04	11:00:00.0000000	11:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	1	Luka Nikolic	KG-321-DD
4	2409	2026-06-05	12:00:00.0000000	12:45:00.0000000	Odrzan	Dodatni prakticni cas	Prakticna	1	Luka Nikolic	KG-321-DD

Слика 7.34 Резултат функције RasporedInstruktora.

7.7. Процедуре и трансакције

Процедуре су креиране у фајлу: 10_procedure.sql. Користе се за операције које мењају податке и у којима је потребно извршити више провера пре самог уноса или измене. [3]

Све процедуре су реализоване са трансакцијама, како би се обезбедило да се више повезаних операција изврши као једна целина. [1] [2]

У процедурама су коришћене наредбе:

- BEGIN TRANSACTION
- COMMIT TRANSACTION
- ROLLBACK TRANSACTION

То значи да се више повезаних корака извршава као једна логичка целина. Ако су све провере успешне, промене се потврђују наредбом COMMIT TRANSACTION.

Уколико се током извршавања појави грешка, блок CATCH проверава да ли постоји активна трансакција и извршава ROLLBACK TRANSACTION. На тај начин база не остаје у делимично измењеном стању. Наредба SET XACT_ABORT ON додатно обезбеђује да се трансакција прекине ако дође до грешке приликом извршавања SQL наредби. [3]

7.7.1. usp_UpisKandidataNaObuku

Упис кандидата на обуку није само један INSERT у табелу Obuka, јер пре тога база мора да провери да ли су сви повезани подаци исправни. Процедура usp_UpisKandidataNaObuku зато најпре проверава постојање кандидата, категорије, типа обуке, инструктора и групе ако је она прослеђена. Посебно је важна провера преко функције fn_KandidatImaAktivnuObuku, јер се њом спречава да исти кандидат има две активне обуке за исту категорију. Ако је приликом уписа изабрана и група, процедура након креирања обуке додаје и одговарајући запис у табелу Kandidat_grupa.

```
CREATE OR ALTER PROCEDURE dbo.usp_UpisKandidataNaObuku
    @Kandidat_id INT,
    @Kategorija_id INT,
    @Glavni_Instruktor_id INT = NULL,
    @Tip_Obuke INT,
    @Datum_pocetka DATE,
    @Grupa_id INT = NULL,
    @Nova_Obuka_id INT OUTPUT
AS
BEGIN
```

```
SET XACT_ABORT ON;

BEGIN TRY
    BEGIN TRANSACTION;
    IF NOT EXISTS (SELECT 1 FROM Kandidat WHERE Id = @Kandidat_id)
        THROW 50001, 'Kandidat ne postoji.', 1;
    IF NOT EXISTS (SELECT 1 FROM Kategorija_vozacke WHERE Id = @Kategorija_id)
        THROW 50002, 'Kategorija ne postoji.', 1;
    IF NOT EXISTS (SELECT 1 FROM Tip_obuke WHERE Id = @Tip_Obuke)
        THROW 50003, 'Tip obuke ne postoji.', 1;
    IF @Glavni_Instruktor_id IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM Zaposleni WHERE Id = @Glavni_Instruktor_id)
        THROW 50004, 'Glavni instruktor ne postoji.', 1;
    IF @Grupa_id IS NOT NULL
        AND NOT EXISTS (SELECT 1 FROM Grupa WHERE Id = @Grupa_id)
        THROW 50005, 'Grupa ne postoji.', 1;
    IF dbo.fn_KandidatImaAktivnuObuku(@Kandidat_id, @Kategorija_id) = 1
        THROW 50006, 'Kandidat vec ima aktivnu obuku za ovu kategoriju.', 1;
    INSERT INTO Obuka (
        Kategorija_id, Glavni_Instruktor_id, Kandidat_id, Tip_Obuke,
        Datum_pocetka, Datum_zavrsetka, Status
    )
    VALUES (
        @Kategorija_id, @Glavni_Instruktor_id, @Kandidat_id,
        @Tip_Obuke, @Datum_pocetka, NULL, N'Aktivan'
    );
    SET @Nova_Obuka_id = CONVERT(INT, SCOPE_IDENTITY());
    IF @Grupa_id IS NOT NULL
    BEGIN
        INSERT INTO Kandidat_grupa (Grupa_id, Obuka_id, Datum_od, Datum_do)
        VALUES (@Grupa_id, @Nova_Obuka_id, @Datum_pocetka, NULL);
    END;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    THROW;
END CATCH;
END;
```

Прилог 7.36 Дефиниција процедуре UpisKandidataNaObuku.

7.7.2. usp_ZakaziPrakticniCas

Практични час може да се закаже тек када су испуњени услови који се односе на обуку, инструктора и возило. Због тога ова процедура пре уноса проверава да ли наведена обука постоји, да ли је изабрани инструктор исправан и да ли је возило доступно. Посебно се проверава да ли се термин преклапа са већ заказаним часовима, јер исти инструктор или исто возило не могу бити ангажовани у два часа у исто време. Ако су све провере успешне, креира се нови практични час са статусом заказан.

```
CREATE OR ALTER PROCEDURE dbo.usp_ZakaziPrakticniCas
    @Obuka_id INT,
    @Instruktor_id INT,
    @Vozilo_id INT,
    @Datum DATE,
    @Pocetak TIME,
    @Kraj TIME,
    @Lokacija NVARCHAR(200) = NULL,
    @Novi_Cas_id INT OUTPUT
AS
BEGIN
    SET XACT_ABORT ON;
```

```
DECLARE @Tip_id INT;
BEGIN TRY
    BEGIN TRANSACTION;
    SELECT @Tip_id = Id
    FROM Tip_obuke
    WHERE Tip = N'Prakticna';
    IF @Tip_id IS NULL
        THROW 50101, 'Tip obuke Prakticna ne postoji.', 1;
    IF NOT EXISTS (SELECT 1 FROM Obuka WHERE Id = @Obuka_id)
        THROW 50102, 'Obuka ne postoji.', 1;
    IF NOT EXISTS (SELECT 1 FROM Zaposleni WHERE Id = @Instruktor_id)
        THROW 50103, 'Instruktor ne postoji.', 1;
    IF NOT EXISTS (SELECT 1 FROM Vozilo WHERE Id = @Vozilo_id)
        THROW 50104, 'Vozilo ne postoji.', 1;
    IF @Kraj <= @Pocetak
        THROW 50105, 'Kraj casa mora biti posle pocetka.', 1;
    IF EXISTS (
        SELECT 1
        FROM Cas c
        WHERE c.Instruktor_id = @Instruktor_id
            AND c.Datum = @Datum
            AND c.Status <> N'Otkazan'
            AND @Pocetak < c.Kraj
            AND @Kraj > c.Pocetak
    )
        THROW 50107, 'Instruktor vec ima cas u tom terminu.', 1;
    IF EXISTS (
        SELECT 1
        FROM Cas c
        WHERE c.Vozilo_id = @Vozilo_id
            AND c.Datum = @Datum
            AND c.Status <> N'Otkazan'
            AND @Pocetak < c.Kraj
            AND @Kraj > c.Pocetak
    )
        THROW 50108, 'Vozilo je vec zauzeto u tom terminu.', 1;
    INSERT INTO Cas (
        Instruktor_id, Tip_id, Lokacija, Datum, Pocetak, Kraj,
        Status, Obuka_id, Vozilo_id, Grupa_id
    )
    VALUES (
        @Instruktor_id, @Tip_id, @Lokacija, @Datum, @Pocetak,
        @Kraj, N'Zakazan', @Obuka_id, @Vozilo_id, NULL
    );
    SET @Novi_Cas_id = CONVERT(INT, SCOPE_IDENTITY());
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    THROW;
END CATCH;
END;
```

Прилог 7.37 Дефиниција процедуре ZakaziPrakticniCas.

7.7.3. usp_EvidentirajUplatu

Евидентирање уплате није само унос износа, већ мора бити повезано са конкретном обуком и одговарајућом ставком ценовника. Пре уписа се проверава да ли обука и ставка ценовника постоје, да ли је износ већи од нуле и да ли је начин плаћања једна од дозвољених вредности. Тек након тих провера уплата се уписује у табелу Uplata, чиме финансијска евиденција остаје повезана са током обуке кандидата.

```
CREATE OR ALTER PROCEDURE dbo.usp_EvidentirajUplatu
```

```
@Obuka_id INT,  
@Cenovnik_id INT,  
@Iznos DECIMAL(10, 2),  
@Datum DATE,  
@Nacin_placanja NVARCHAR(20),  
@Nova_Uplata_id INT OUTPUT  
AS  
BEGIN  
    SET XACT_ABORT ON;  
  
    BEGIN TRY  
        BEGIN TRANSACTION;  
        IF NOT EXISTS (SELECT 1 FROM Obuka WHERE Id = @Obuka_id)  
            THROW 50201, 'Obuka ne postoji.', 1;  
        IF NOT EXISTS (SELECT 1 FROM Cenovnik WHERE Id = @Cenovnik_id)  
            THROW 50202, 'Stavka cenovnika ne postoji.', 1;  
        IF @Iznos = 0  
            THROW 50203, 'Iznos uplate ne sme biti nula.', 1;  
        IF @Nacin_placanja NOT IN (N'Gotovina', N'Kartica', N'Prenos')  
            THROW 50204, 'Nacin placanja nije dozvoljen.', 1;  
        INSERT INTO Uplata (  
            Cenovnik_id, Obuka_id, Iznos, Datum, Nacin_placanja  
        )  
        VALUES (  
            @Cenovnik_id, @Obuka_id, @Iznos, @Datum,  
            @Nacin_placanja  
        );  
        SET @Nova_Uplata_id = CONVERT(INT, SCOPE_IDENTITY());  
        COMMIT TRANSACTION;  
    END TRY  
    BEGIN CATCH  
        IF @@TRANCOUNT > 0  
            ROLLBACK TRANSACTION;  
        THROW;  
    END CATCH;  
END;
```

Прилог 7.38 Дефиниција процедуре EvidentirajUplatu.

7.7.4. usp_PrijaviKandidataNaPolaganje

Пријава кандидата на полагање издвојена је као посебан корак, јер у тренутку пријаве још увек не постоји резултат полагања. Пре уписа се проверава да ли су изабрано полагање и обука исправни, као и да исти кандидат није већ пријављен на то полагање. Та провера је важна због јединственог ограничења над комбинацијом Polaganje_id и Obuka_id, које спречава да се једна иста пријава унесе више пута. Након успешне провере у табелу Polaganje_kandidat додаје се запис који касније може бити допуњен резултатом.

```
CREATE OR ALTER PROCEDURE dbo.usp_PrijaviKandidataNaPolaganje  
    @Polaganje_id INT,  
    @Obuka_id INT  
AS  
BEGIN  
    SET XACT_ABORT ON;  
  
    BEGIN TRY  
        BEGIN TRANSACTION;  
        IF NOT EXISTS (SELECT 1 FROM Polaganje WHERE Id = @Polaganje_id)  
            THROW 50301, 'Polaganje ne postoji.', 1;  
        IF NOT EXISTS (SELECT 1 FROM Obuka WHERE Id = @Obuka_id)  
            THROW 50302, 'Obuka ne postoji.', 1;  
        IF EXISTS (  
            SELECT 1
```

```
FROM Polaganje_kandidat
WHERE Polaganje_id = @Polaganje_id
AND Obuka_id = @Obuka_id
)
    THROW 50303, 'Kandidat je vec prijavljen na ovo polaganje.', 1;
INSERT INTO Polaganje_kandidat (
    Polaganje_id, Obuka_id, Uspesno, Broj_Poenta
)
VALUES (
    @Polaganje_id, @Obuka_id, NULL, NULL
);
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    THROW;
END CATCH;
END;
```

Прилог 7.39 Дефиниција процедуре PrijaviKandidataNaPolaganje.

7.7.5. usp_EvidentirajRezultatPolaganja

Резултат полагања се не уноси као нова пријава, већ се допуњује постојећи запис кандидата на полагању. Пре ажурирања проверава се да ли та пријава постоји и да ли је број поена у дозвољеном опсегу од 0 до 100. Након успешне провере ажурирају се поља Uspesno и Broj_Poenta у табели Polaganje_kandidat, чиме се раздваја тренутак пријаве од тренутка евидентирања резултата.

```
CREATE OR ALTER PROCEDURE dbo.usp_EvidentirajRezultatPolaganja
    @Polaganje_id INT,
    @Obuka_id INT,
    @Uspesno BIT,
    @Broj_Poenta TINYINT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (
            SELECT 1
            FROM Polaganje_kandidat
            WHERE Polaganje_id = @Polaganje_id
            AND Obuka_id = @Obuka_id
        )
            THROW 50401, 'Prijava kandidata na polaganje ne postoji.', 1;
        IF @Broj_Poenta NOT BETWEEN 0 AND 100
            THROW 50402, 'Broj poena mora biti izmedju 0 i 100.', 1;
        UPDATE Polaganje_kandidat
        SET
            Uspesno = @Uspesno,
            Broj_Poenta = @Broj_Poenta
        WHERE Polaganje_id = @Polaganje_id
            AND Obuka_id = @Obuka_id;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

7.8. Тригери

Тригери су специјални објекти базе података који се аутоматски покрећу као одговор на одређени догађај над табелом. За разлику од процедура, тригери се не позивају ручно, већ се извршавају када се над табелом догоди: [6] [3]

- INSERT
- UPDATE
- DELETE

У овом раду тригери су креирани у фајлу 11_trigeri.sql и користе се за техничко праћење измена, заштиту података и аутоматско извршавање последичних измена у повезаним табелама.

У SQL Server-у најчешће се користе:

- **DML тригери**, који реагују на измене података у табелама, као што су INSERT, UPDATE и DELETE. [3]
- **DDL тригери**, који реагују на измене структуре базе, као што су CREATE, ALTER и DROP. [3]

7.8.1. Аутоматско ажурирање поља Izmenjen_datum

Поље Izmenjen_datum служи као технички траг о последњој измени записа. За табеле Kandidat, Zaposleni, Obuka, Vozilo, Cas, Polaganje и Uplata креирани су тригери који након измене реда аутоматски уписују време промене. У тригерима постоји провера IF UPDATE(Izmenjen_datum), како се ажурирање овог поља не би претворило у поновно покретање истог тригера.

```
CREATE OR ALTER TRIGGER dbo.trg_Kandidat_SetIzmenjenDatum
ON dbo.Kandidat
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Izmenjen_datum)
        RETURN;

    UPDATE k
    SET Izmenjen_datum = sysdatetime()
    FROM Kandidat k
    JOIN inserted i ON i.Id = k.Id;
END;
```

Прилог 7.41 Дефиниција тригера Kandidat_SetIzmenjenDatum.

7.8.2. trg_Uplata_ZabraniBrisanje

Код финансијских података није пожељно да се уплате физички бришу из базе, јер би се тиме изгубио траг о пословању. Због тога је над табелом Uplata дефинисан INSTEAD OF DELETE тригер који спречава брисање уплата. Ако је потребна исправка, она се евидентира корективном негативном уплатом, чиме историја финансијских промена остаје очувана.

```
CREATE OR ALTER TRIGGER dbo.trg_Uplata_ZabraniBrisanje
ON dbo.Uplata
INSTEAD OF DELETE
```

```
AS
BEGIN
    THROW 51001, 'Uplate se ne brisu. Za ispravku unesite korektivnu negativnu
uplatu.', 1;
END;
```

Прилог 7.42 Дефиниција тригера Uplata_ZabraniBrisanje.

7.8.3. trg_Vozilo_Kilometraza_NeSmanjuje

Километража возила је податак који у нормалном раду треба само да расте. Овај тригер проверава измену километраже у табели **Vozilo** и не дозвољава да нова вредност буде мања од претходне. На тај начин се спречава нелогичан унос и чува поузданост евиденције возила.

```
CREATE OR ALTER TRIGGER dbo.trg_Vozilo_Kilometraza_NeSmanjuje
ON dbo.Vozilo
AFTER UPDATE
AS
BEGIN
    IF NOT UPDATE(Kilometraza)
        RETURN;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        JOIN deleted d ON d.Id = i.Id
        WHERE i.Kilometraza < d.Kilometraza
    )
        THROW 51002, 'Kilometraza vozila ne sme da se smanji.', 1;
END;
```

Прилог 7.43 Дефиниција тригера Vozilo_Kilometraza_NeSmanjuje.

7.8.4. trg_Cas_Odrzan_PovecajKilometrazu

Када се практични час означи као одржан, то утиче и на евиденцију возила. Ако час први пут добије статус **Odrzan** и има повезано возило, километража тог возила се аутоматски увећава за 10. Овај пример показује како тригер може да повеже промену у табели **Cas** са изменом података у табели **Vozilo**.

```
CREATE OR ALTER TRIGGER dbo.trg_Cas_Odrzan_PovecajKilometrazu
ON dbo.Cas
AFTER UPDATE
AS
BEGIN
    IF NOT UPDATE(Status)
        RETURN;

    UPDATE v
    SET Kilometraza = v.Kilometraza + 10
    FROM Vozilo v
    JOIN inserted i ON i.Vozilo_id = v.Id
    JOIN deleted d ON d.Id = i.Id
    WHERE i.Status = N'Odrzan'
        AND d.Status <> N'Odrzan'
        AND i.Vozilo_id IS NOT NULL;
END;
```

Прилог 7.44 Дефиниција тригера Cas_Odrzan_PovecajKilometrazu.

7.8.5. trg_PolaganjeKandidat_Uspesno_ZavrsiObuku

Успешно полагање није изолован податак, већ директно утиче на статус обуке кандидата. Када се у табели Polaganje_kandidat евидентира да је кандидат положио, тригер поставља обуку на статус Završen и као датум завршетка користи датум полагања. Тако се резултат полагања одмах одражава на ток обуке.

```
CREATE OR ALTER TRIGGER dbo.trg_PolaganjeKandidat_Uspesno_ZavrsiObuku
ON dbo.Polaganje_kandidat
AFTER UPDATE
AS
BEGIN
    IF NOT UPDATE(Uspesno)
        RETURN;

    UPDATE o
    SET
        Status = N'Završen',
        Datum_zavrsetka = CONVERT(date, p.Pocetak)
    FROM Obuka o
    JOIN inserted i ON i.Obuka_id = o.Id
    JOIN Polaganje p ON p.Id = i.Polaganje_id
    WHERE i.Uspesno = 1
        AND (
            o.Status <> N'Završen'
            OR o.Datum_zavrsetka IS NULL
        );
END;
```

Прилог 7.45 Дефиниција тригера PolaganjeKandidat_Uspesno_ZavrsiObuku.

7.8.6. trg_Obuka_PromenaInstruktora_PrebaciBuduceCasove

Промена главног инструктора на обуци важна је и за будући распоред часова. Овај тригер све будуће заказане часове те обуке пребацује на новог инструктора. Прошли и већ одржани часови се не мењају, јер представљају историјску евиденцију.

```
CREATE OR ALTER TRIGGER dbo.trg_Obuka_PromenaInstruktora_PrebaciBuduceCasove
ON dbo.Obuka
AFTER UPDATE
AS
BEGIN
    IF NOT UPDATE(Glavni_Instruktor_id)
        RETURN;

    UPDATE c
    SET Instruktor_id = i.Glavni_Instruktor_id
    FROM Cas c
    JOIN inserted i ON i.Id = c.Obuka_id
    JOIN deleted d ON d.Id = i.Id
    WHERE i.Glavni_Instruktor_id IS NOT NULL
        AND (
            d.Glavni_Instruktor_id IS NULL
            OR i.Glavni_Instruktor_id <> d.Glavni_Instruktor_id
        )
        AND c.Status = N'Zakazan'
        AND c.Datum >= CONVERT(date, getdate());
END;
```

Прилог 7.46 Дефиниција тригера Obuka_PromenaInstruktora_PrebaciBuduceCasove.

8. ОПТИМИЗАЦИЈА И АНАЛИЗА ПЕРФОРМАНСИ

Оптимизација је обрађена кроз анализу најчешћих упита читања и проверу начина на који SQL Server извршава те упите. За ту сврху коришћен је план извршавања (Execution Plan) у SSMS алату, као и поређење извршавања упита пре и после додавања индекса. [4]

8.1. План извршавања

План извршавања приказује кораке које SQL Server користи приликом извршавања упита. На основу њега се може видети да ли се подаци траже преко индекса, да ли се чита већи део табеле, који JOIN алгоритам је употребљен и где се налази највећи проценат трошка упита.

Слика 8.1 Укључење плана извршавања.

У плану извршавања посматрани су оператори који показују приступ подацима, спајање табела и додатну обраду резултата.

Оператори за приступ подацима:

- **Index Seek** - означава прецизно коришћење индекса за проналажење тражених редова
- **Index Scan** - означава читање већег дела индекса или целог индекса
- **Clustered Index Scan** - означава читање већег дела табеле или целе табеле

Оператори за спајање табела:

- **Nested Loops** - приказује спајање табела које је погодно када једна страна враћа мали број редова
- **Hash Match** - приказује спајање табела или обраду већег броја редова

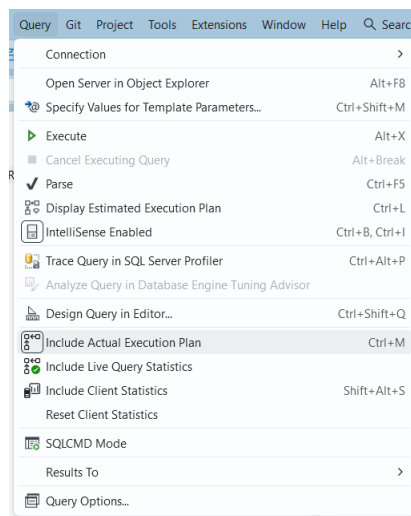
Оператори за додатну обраду резултата:

- **Sort** - означава додатно сортирање резултата
- **Stream Aggregate** - јавља се код агрегатних упита, као што су COUNT и SUM

Посебна пажња је посвећена ситуацијама у којима:

- упит чита велики број редова
- упит користи Sort оператор
- SQL Server предлаже недостајући индекс (missing index)

Ови показатељи нису сами по себи довољни за коначну одлуку, али помажу да се уочи где постоји простор за оптимизацију.

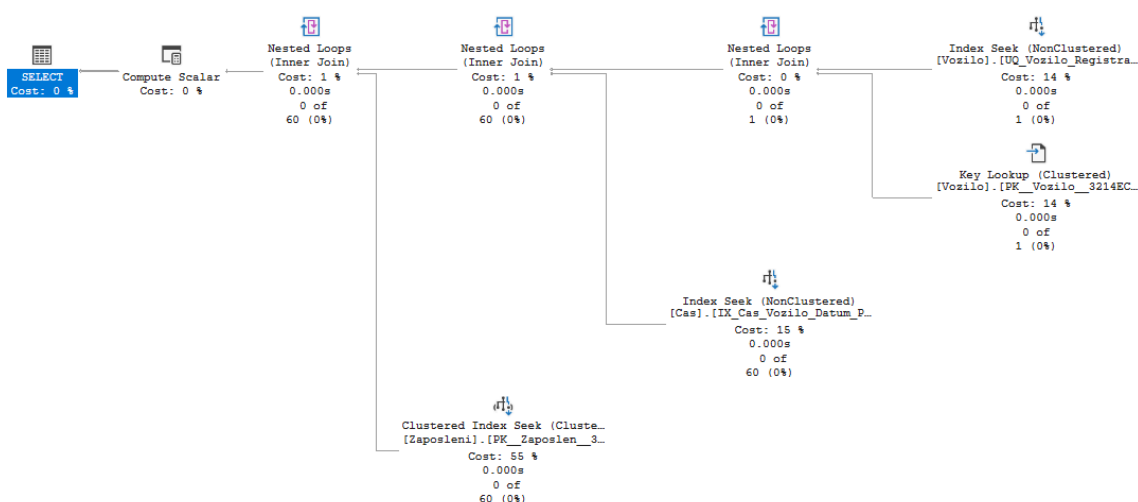


У плану извршавања пре додавања индекса види се да се над табелом Cas користи Clustered Index Scan. То значи да SQL Server чита већи део табеле часова како би пронашао редове који одговарају услову за возило и датумски период. Због тога је индекс постављен над колонама **Vozilo_id**, **Datum** и **Pocetak**, јер те колоне одговарају начину на који се подаци у овом упиту претражују и сортирају.

```
CREATE INDEX IX_Cas_Vozilo_Datum_Pocetak ON Cas (Vozilo_id, Datum, Pocetak)
INCLUDE (Kraj, Status, Lokacija, Instruktor_id);
```

Прилог 8.2 Команда за креирање индекса над табелом Cas.

Команда за креирање индекса приказана је у прилогу 8.2. Индекс IX_Cas_Vozilo_Datum_Pocetak постављен је тако да прво издвоји часове за једно возило, затим ограничи датумски период, а на крају помогне при редоследу по времену почетка.



Слика 8.3 Приказ плана извршавања упита возило по датуму, са индексом.

После креирања индекса, што је приказано на слици 8.3, у плану се види Index Seek над табелом Cas. То значи да SQL Server не чита већи део табеле, већ преко индекса директније долази до часова за изабрано возило и период.

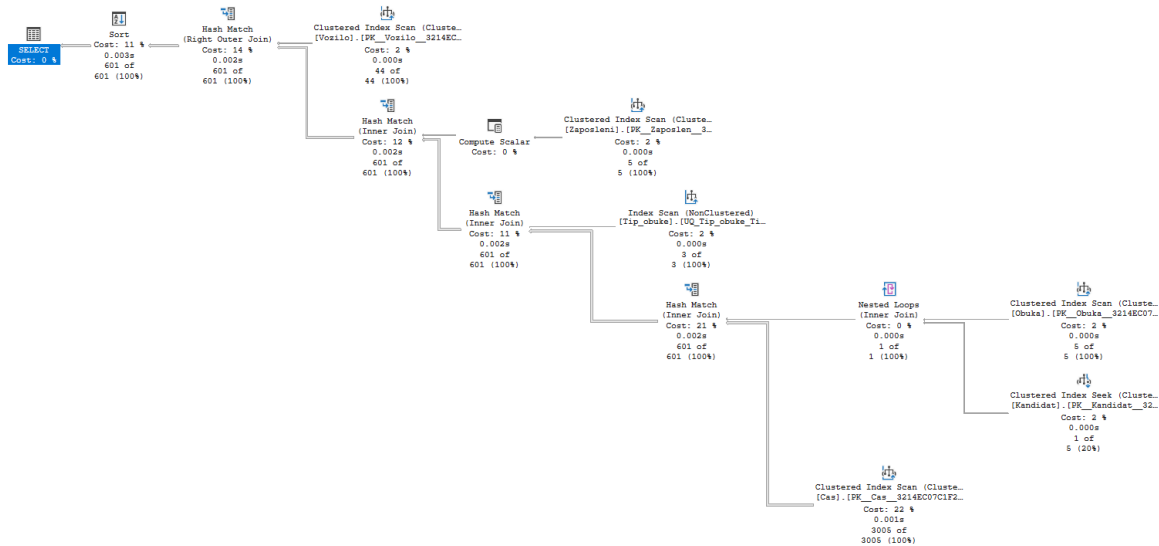
8.2.2. Часови по кандидату

Код овог примера прати се ток обуке једног кандидата кроз часове који су му додељени. Упит је дат у прилогу 8.3, а почетни план извршавања на слици 8.4. Такав преглед је користан када треба проверити одржане часове, статус часа, инструктора, возило и тип обуке.

```
SELECT
  c.Id,
  c.Datum,
  c.Pocetak,
  c.Kraj,
  c.Status,
  c.Lokacija,
  t.Tip AS Tip_casa,
  z.Ime + N' ' + z.Prezime AS Instruktor,
  v.Registracija,
  v.Marka,
  v.Model
FROM Cas c
```

```
JOIN Obuka o ON o.Id = c.Obuka_id
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Tip_obuke t ON t.Id = c.Tip_id
JOIN Zaposleni z ON z.Id = c.Instruktor_id
LEFT JOIN Vozilo v ON v.Id = c.Vozilo_id
WHERE k.JMBG = '1502002715002'
ORDER BY c.Datum, c.Pocetak;
```

Прилог 8.3 Приказ упита часови по кандидату.

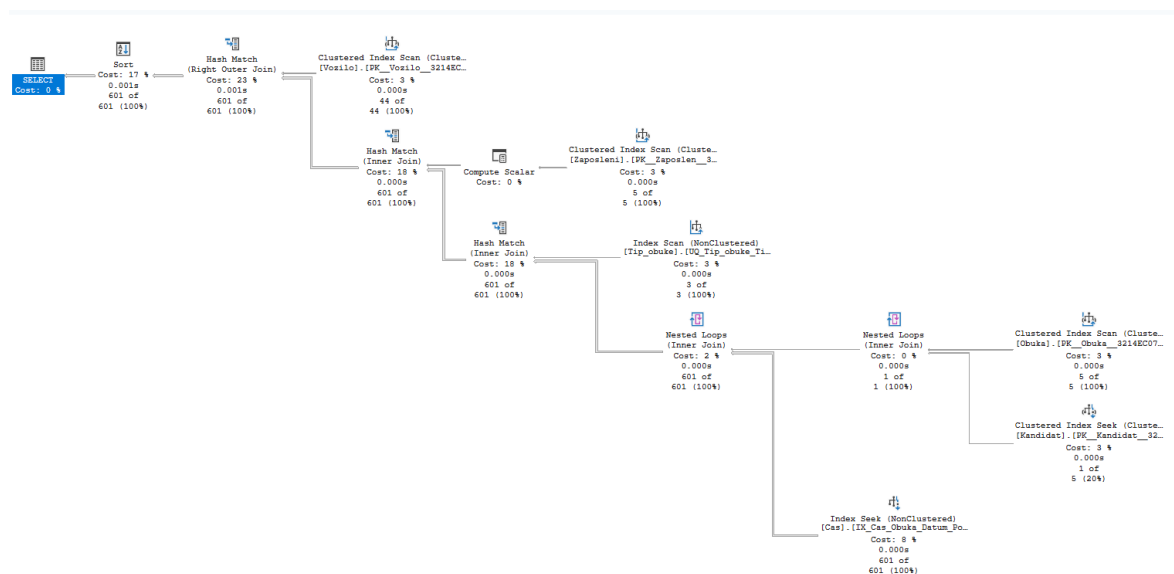


Слика 8.4 Приказ плана извршавања упита часови по кандидату, без индекса.

Пре оптимизације план показује да се из табеле Cas чита много више редова него што је потребно за једног кандидата. Пошто се до часова долази преко обуке, индекс је усмерен на колоне Obuka_id, Datum и Pocetak.

```
CREATE INDEX IX_Cas_Obuka_Datum_Pocetak ON Cas (Obuka_id, Datum, Pocetak)
INCLUDE (Kraj, Status, Lokacija, Instruktor_id, Tip_id, Vozilo_id);
```

Прилог 8.4 Приказ команде за креирање индекса над табелом Cas.



Слика 8.5 Приказ плана извршавања упита часови по кандидату, са индексом.

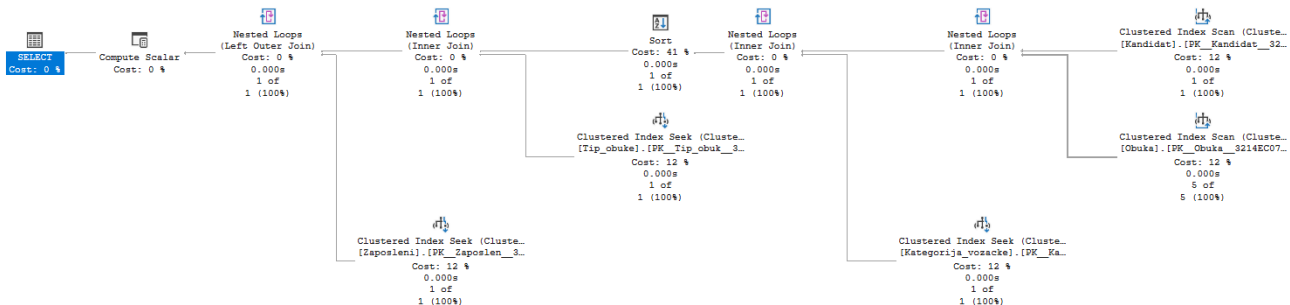
После додавања индекса, што се види на слици 8.5, појављује се Index Seek над табелом Cas. Број прочитаних редова смањен је са око 3005 на око 601, што показује да SQL Server уже чита податке потребне за овај упит.

8.2.3. Обуке по кандидату и категорији

Овај пример је важан јер кандидат може имати више обука, али је често потребно пронаћи баш обуку за одређену категорију возачке дозволе. Упит је приказан у прилогу 8.5, а план пре индекса на слици 8.6.

```
SELECT
o.Id,
o.Datum_pocetka,
o.Datum_zavrsetka,
o.Status,
k.Ime,
k.Prezime,
kv.Oznaka AS Kategorija,
t.Tip AS Tip_obuke,
z.Ime + N' ' + z.Prezime AS Glavni_instruktor
FROM Obuka o
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id
JOIN Tip_obuke t ON t.Id = o.Tip_Obuke
LEFT JOIN Zaposleni z ON z.Id = o.Glavni_Instruktor_id
WHERE k.JMBG = '0101000710001'
AND kv.Oznaka = 'B'
ORDER BY o.Datum_pocetka DESC;
```

Прилог 8.5 Приказ упита обука по кандидату и категорији.



Слика 8.6 Приказ плана извршавања упита обука по кандидату и категорији, без индекса.

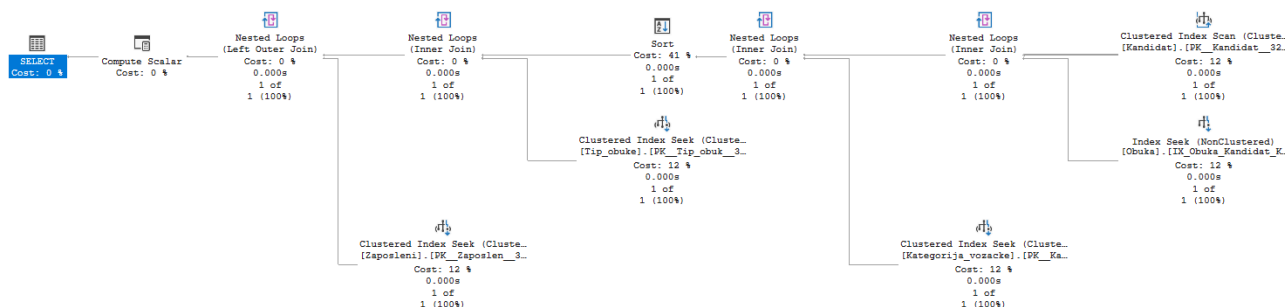
Пре додавања индекса у плану се види Clustered Index Scan над табелом Obuka. Ова табела је кључна за пример, јер у њој стоји веза између кандидата, категорије и саме обуке. Иако се у упиту користе ЈМБГ кандидата и ознака категорије, SQL Server после повезивања табела у Obuka тражи одговарајуће вредности Kandidat_id и Kategorija_id.

```
CREATE INDEX IX_Obuka_Kandidat_Kategorija_Datum ON Obuka (Kandidat_id,
Kategorija_id, Datum_pocetka)
INCLUDE (Datum_zavrsetka, Status, Tip_Obuke, Glavni_Instruktor_id);
```

Прилог 8.6 Приказ команде за креирање индекса над табелом Obuka.

Команда за креирање индекса дата је у прилогу 8.6. Индекс IX_Obuka_Kandidat_Kategorija_Datum обухвата колоне Kandidat_id, Kategorija_id и Datum_pocetka. Прве две колоне сужавају претрагу, док

Datum_rosetka прати сортирање резултата. INCLUDE део чува додатне колоне у индексу, како SQL Server не би морао посебно да их чита из основне табеле.



Слика 8.7 Приказ плана извршавања упита обука по кандидату и категорији, са индексом.

После додавања индекса, приказаног на слици 8.7, у плану се види Index Seek над табелом Obuka. Тражена обука се проналази директније, преко вредности кандидата и категорије.

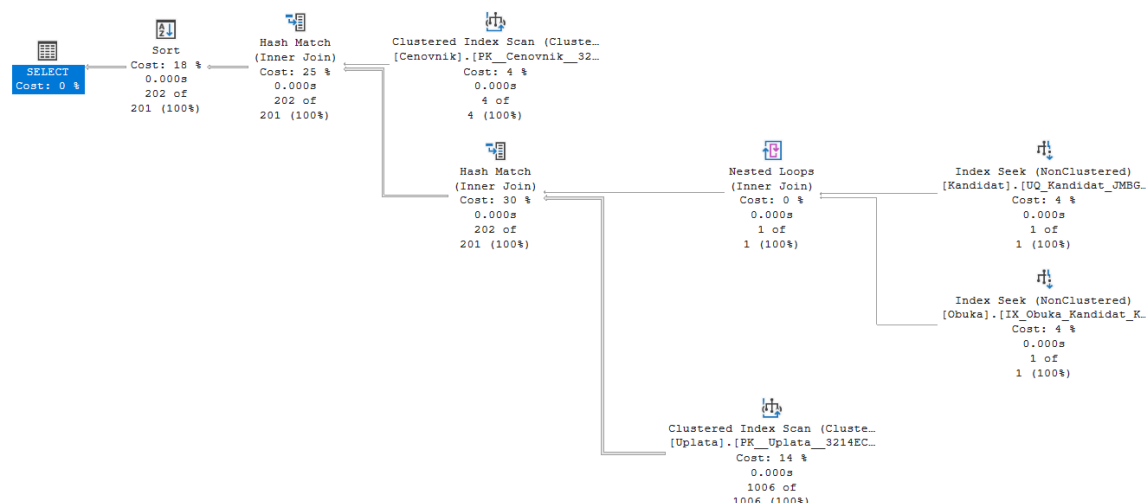
8.2.4. Уплате по обуци

Финансијски преглед једне обуке захтева брзо издвајање свих уплата које јој припадају. Упит је приказан у прилогу 8.7, а план без индекса на слици 8.8. Тако администрација може да провери које су ставке плаћене, који је износ уплаћен и када је уплата евидентирана.

```
SELECT
    u.Id,
    u.Datum,
    u.Iznos,
    u.Nacin_placanja,
    c.Naziv AS Stavka_cenovnika
FROM Uplata u
JOIN Obuka o ON o.Id = u.Obuka_id
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Cenovnik c ON c.Id = u.Cenovnik_id
WHERE k.JMBG = '0101000710001'
ORDER BY u.Datum;
```

Прилог 8.7 Приказ упита уплате по обуци.

Пре додавања индекса табела Uplata се чита преко Clustered Index Scan оператора. Пошто је свака уплата повезана са обуком, најважнија колона за сужавање резултата је Obuka_id.

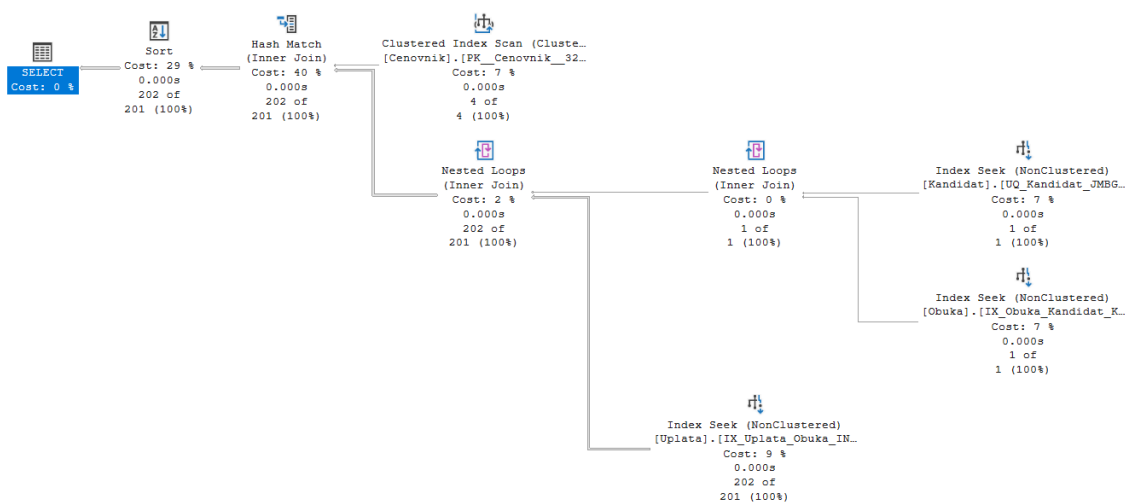


Слика 8.8 Приказ плана извршавања упита уплате по обуци, без индекса.

Команда за креирање индекса дата је у прилогу 8.8, а план после индекса на слици 8.9. Индекс IX_Uplata_Obuka_INCLUDE постављен је над колоном Obuka_id, док су Iznos, Datum, Nacin_placanja и Cenovnik_id додати кроз INCLUDE део.

```
CREATE INDEX IX_Uplata_Obuka_INCLUDE ON Uplata (Obuka_id)
INCLUDE (Iznos, Datum, Nacin_placanja, Cenovnik_id);
```

Прилог 8.8 Приказ команде за креирање индекса над табелом Uplata.



Слика 8.9 Приказ плана извршавања упита уплате по обуци, са индексом.

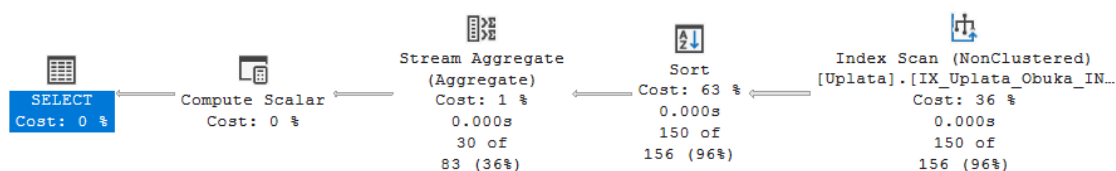
После креирања индекса план користи Index Seek над табелом Uplata, па SQL Server директније долази до уплата за изабрану обуку.

8.2.5. Уплате по датуму

Ова анализа је занимљива јер упит не враћа само појединачне уплате, већ их групише по датуму и начину плаћања. Упит је дат у прилогу 8.9, а почетни план на слици 8.10. Такав приказ је погодан за дневне, месечне или кварталне финансијске прегледе.

```
SELECT
    u.Datum,
    u.Nacin_placanja,
    COUNT(*) AS Broj_uplata,
    SUM(u.Iznos) AS Ukupno
FROM Uplata u
WHERE u.Datum BETWEEN '2026-06-01' AND '2026-08-31'
GROUP BY u.Datum, u.Nacin_placanja
ORDER BY u.Datum, u.Nacin_placanja;
```

Прилог 8.9 Приказ упита уплате по датуму.



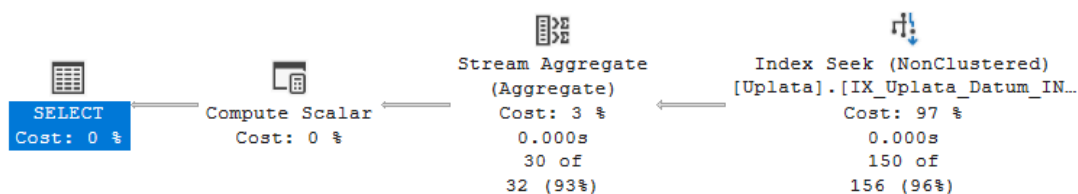
Слика 8.10 Приказ плана извршавања упита уплате по датуму, без индекса.

Команда за креирање индекса приказана је у прилогу 8.10. У плану извршавања пре додавања индекса види се Index Scan над табелом Uplata, као и Sort оператор који има највећи трошак. Због тога је индекс IX_Uplata_Datum_INCLUDE креиран над колонама Datum и Nacin_placanja, јер се оне користе за претрагу, груписање и сортирање. Колона Iznos је додата кроз INCLUDE део, како би вредност потребна за SUM била доступна из индекса.

```
CREATE INDEX IX_Uplata_Datum_INCLUDE ON Uplata (Datum, Nacin_placanja)
INCLUDE (Iznos);
```

Прилог 8.10 Приказ команде за креирање индекса над табелом Uplata.

После додавања индекса, што је приказано на слици 8.11, SQL Server користи Index Seek над табелом Uplata, а Sort оператор више није потребан. Разлог је то што индекс већ чува податке поређане по колонама Datum и Nacin_placanja, истим редоследом који се користи у GROUP BY и ORDER BY делу упита.



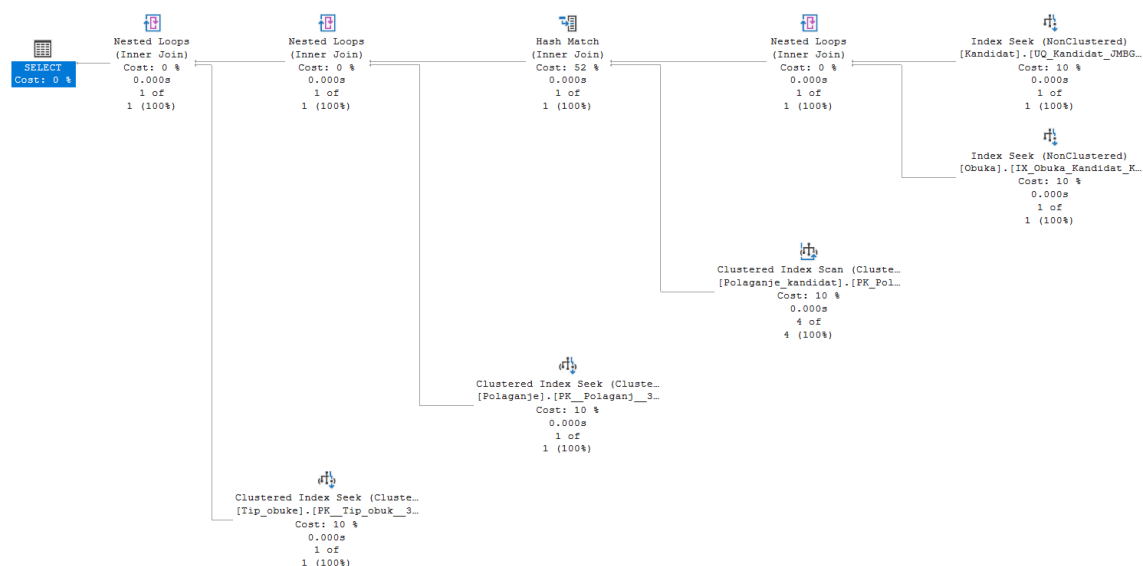
Слика 8.11 Приказ плана извршавања упита уплате по датуму, са индексом.

8.2.6. Полагања по кандидату

Код полагања је важно брзо добити покушаје кандидата и њихове резултате. Упит је приказан у прилогу 8.11, а план пре индекса на слици 8.12. Приказују се тип полагања, успешност и број поена, па је пример користан за увид у напредак кандидата током обуке.

```
SELECT
p.Id AS Polaganje_id,
p.Pocetak,
p.Kraj,
t.Tip AS Tip_polaganja,
pk.Uspesno,
pk.Broj_Poenta
FROM Polaganje_kandidat pk
JOIN Polaganje p ON p.Id = pk.Polaganje_id
JOIN Obuka o ON o.Id = pk.Obuka_id
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Tip_obuke t ON t.Id = p.Tip_id
WHERE k.JMBG = '1204004715004'
```

Прилог 8.11 Приказ упита полагање по кандидату.

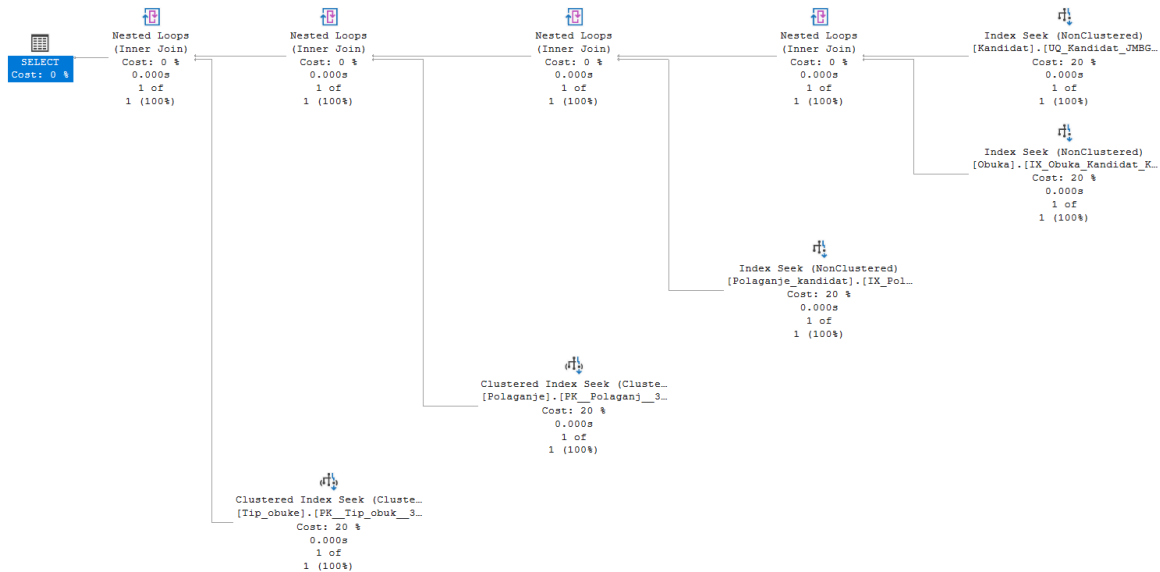


Слика 8.12 Приказ плана извршавања упита полагање по кандидату, без индекса.

У плану пре додавања индекса види се **Clustered Index Scan** над табелом **Polaganje_kandidat**. Индекс се поставља баш на ову табелу јер она чува везу између обуке и полагања кандидата, док се остале табеле у плану већ читају преко постојећих кључева. Зато је за оптимизацију најважнија колона **Obuka_id**, преко које се проналазе полагања за конкретну обуку.

```
CREATE INDEX IX_Polaganje_kandidat_Obuka_INCLUDE ON Polaganje_kandidat (Obuka_id)
INCLUDE (Polaganje_id, Uspesno, Broj_Poenta);
```

Прилог 8.12 Приказ команде за креирање индекса над табелом **Polaganje_kandidat**.



Слика 8.13 Приказ плана извршавања упита полагање по кандидату, са индексом.

После додавања индекса, приказаног на слици 8.13, над табелом Polaganje_kandidat користи се Index Seek, а Hash Match више није присутан. Пошто индекс сужава резултат на мали број редова, SQL Server спајање табела извршава преко оператора Nested Loops.

8.2.7. Кандидати по групи

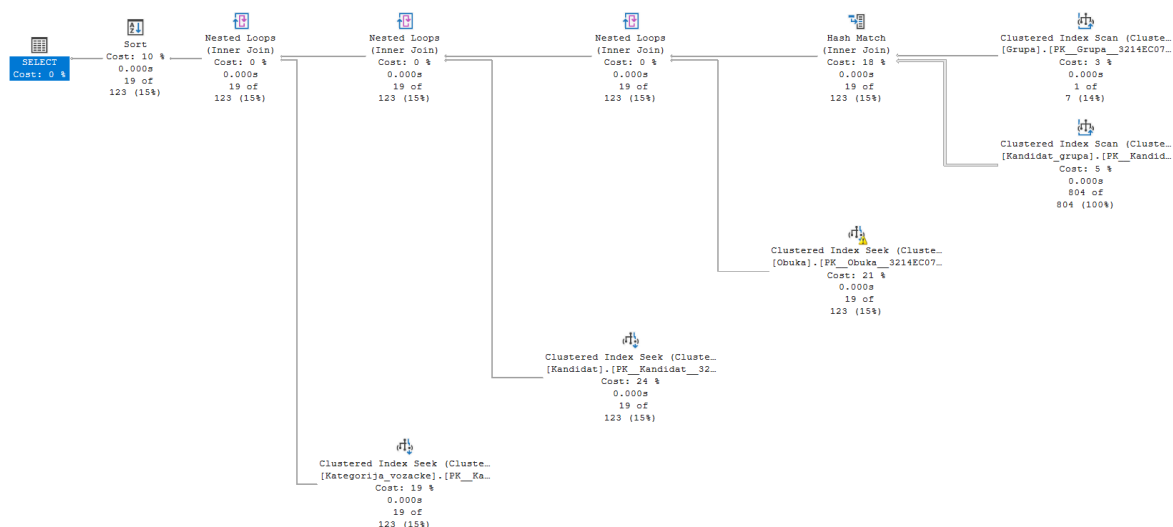
Овај пример се односи на преглед чланова једне групе. Упит је приказан у прилогу 8.13, а план пре додавања индекса на слици 8.14. Пошто се кандидати у групама воде преко спојне табеле `Kandidat_grupa`, управо је та табела најважнија за анализу.

```

SELECT
    k.Ime,
    k.Prezime,
    k.Telefon,
    k.Email,
    kg.Datum_od,
    kg.Datum_do,
    o.Status,
    kv.Oznaka AS Kategorija
FROM Kandidat_grupa kg
JOIN Grupa g ON g.Id = kg.Grupa_id
JOIN Obuka o ON o.Id = kg.Obuka_id
JOIN Kandidat k ON k.Id = o.Kandidat_id
JOIN Kategorija_vozacke kv ON kv.Id = o.Kategorija_id
WHERE g.Datum_kreiranja = '2026-04-01'
ORDER BY k.Prezime, k.Ime;

```

Прилог 8.13 Приказ упита кандидати по групи.

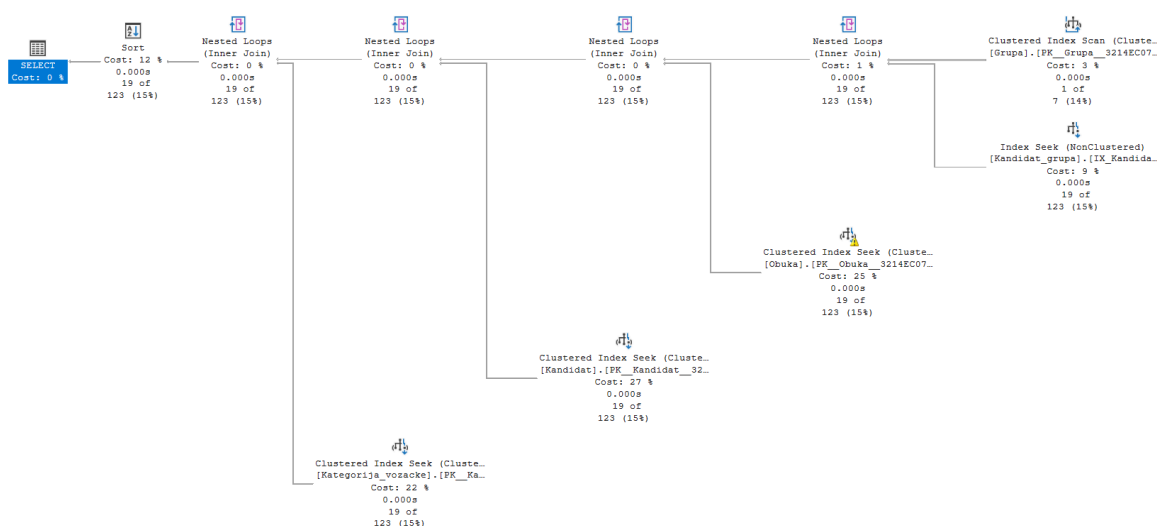


Слика 8.14 Приказ плана извршавања упита кандидати по групи, без индекса.

У плану пре додавања индекса види се Clustered Index Scan над табелом Kandidat_grupa и Hash Match при повезивању са табелом Grupa, што значи да SQL Server чита већи део спојне табеле пре него што издвоји кандидате из тражене групе. Због тога је индекс постављен над колоном Grupa_id, док су Obuka_id, Datum_od и Datum_do додати кроз INCLUDE део јер су потребни за повезивање са обуком и приказ периода чланства.

```
CREATE INDEX IX_Kandidat_grupa_Grupa_INCLUDE ON Kandidat_grupa (Grupa_id)
INCLUDE (Obuka_id, Datum_od, Datum_do);
```

Прилог 8.14 Приказ команде за креирање индекса над табелом Kandidat_grupa.



Слика 8.15 Приказ плана извршавања упита кандидати по групи, са индексом.

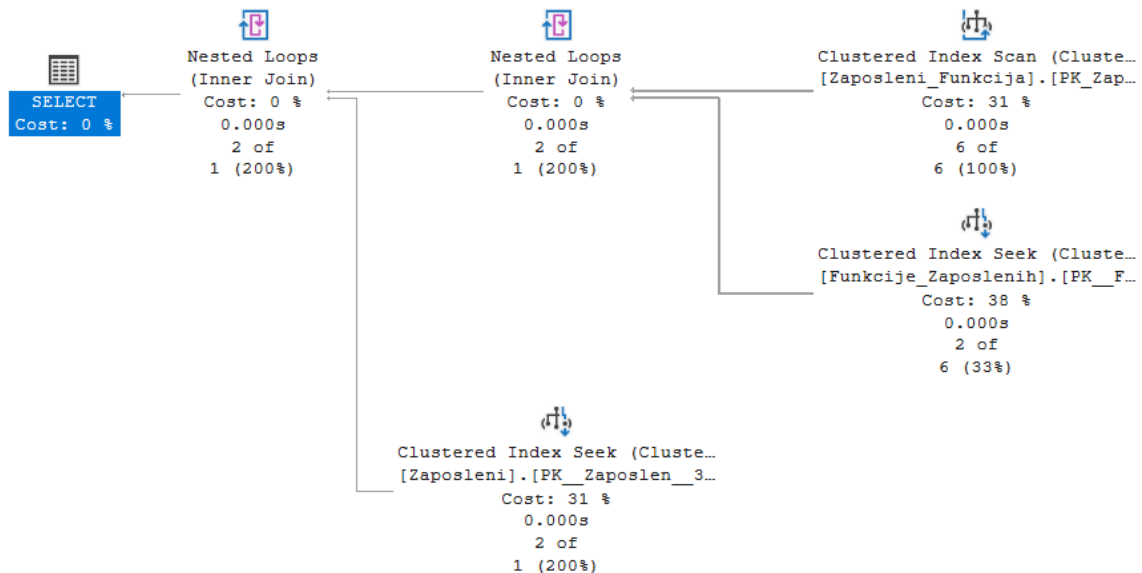
Команда за креирање индекса дата је у прилогу 8.14, а план после индекса на слици 8.15. После додавања индекса SQL Server може директније да издвоји чланове тражене групе, што је корисно када број група и кандидата порасте.

8.2.8. Запослени по функцији

Претрага запослених по функцији ослања се на спојну табелу Zaposleni_Funkcija. Упит је приказан у прилогу 8.15, а план пре индекса на слици 8.16. На овај начин могу се брзо издвојити инструктори, предавачи, администратори или друге улоге у аутошколи.

```
SELECT
    z.Ime,
    z.Prezime,
    z.Telefon,
    z.Email,
    z.Aktivni_ugovor,
    f.Ime_funkcije,
    f.Minimalna_kvalifikacija
FROM Zaposleni_Funkcija zf
JOIN Zaposleni z ON z.Id = zf.Id_zaposlenog
JOIN Funkcije_Zaposlenih f ON f.Id = zf.Id_funkcije
WHERE f.Ime_funkcije = N'Instruktor'
```

Прилог 8.15 Приказ упита запослени по функцији.

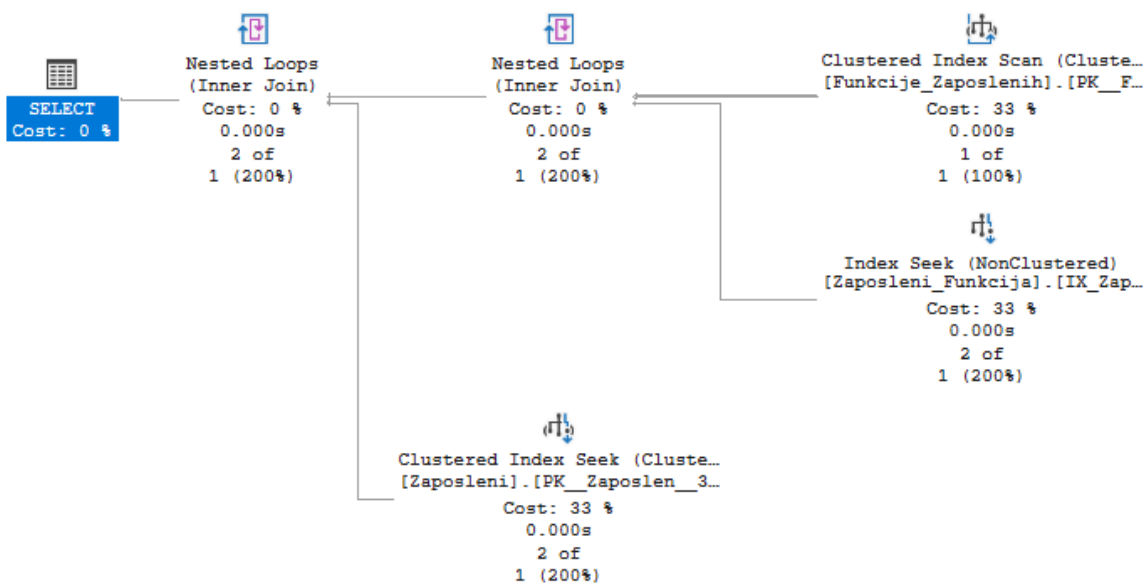


Слика 8.16 Приказ плана извршавања упита запослени по функцији, без индекса.

У приказаном плану извршавања табела Funkcije_Zaposlenih чита се преко оператора Clustered Index Seek, док се над спојном табелом Zaposleni_Funkcija јавља Clustered Index Scan. SQL Server брзо проналази тражену функцију, али затим мора да прегледа спојну табелу како би пронашао запослене који имају ту функцију. Због тога је индекс постављен над табелом Zaposleni_Funkcija, са колоном Id_funkcije на првом месту.

```
CREATE INDEX IX_Zaposleni_Funkcija_Funkcija_Zaposleni ON Zaposleni_Funkcija
(Id_funkcije, Id_zaposlenog);
```

Прилог 8.16 Приказ команде за креирање индекса над табелом Zaposleni_Funkcija.



Слика 8.17 Приказ плана извршавања упита запослени по функцији, са индексом.

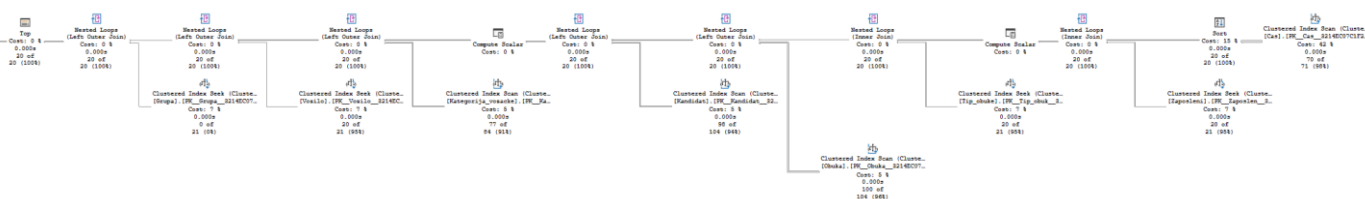
Команда за креирање индекса приказана је у прилогу 8.16, а план после индекса на слици 8.17. После додавања индекса план може да користи Index Seek над спојном табелом, што је ефикасније од читања целе табеле.

8.2.9. Распоред часова

Последњи пример користи поглед `vw_Raspored_Casova` за приказ распореда часова у задатом временском периоду. Упит је приказан у прилогу 8.17, а план пре индекса на слици 8.18. Практична вредност овог упита је брз преглед планираних и одржаних часова по датуму и времену почетка.

```
SELECT TOP (20) *
FROM dbo.vw_Raspored_Casova
WHERE Datum BETWEEN '2026-06-01' AND '2026-06-15'
ORDER BY Datum, Pocetak;
```

Прилог 8.17 Приказ упита распоред часова.



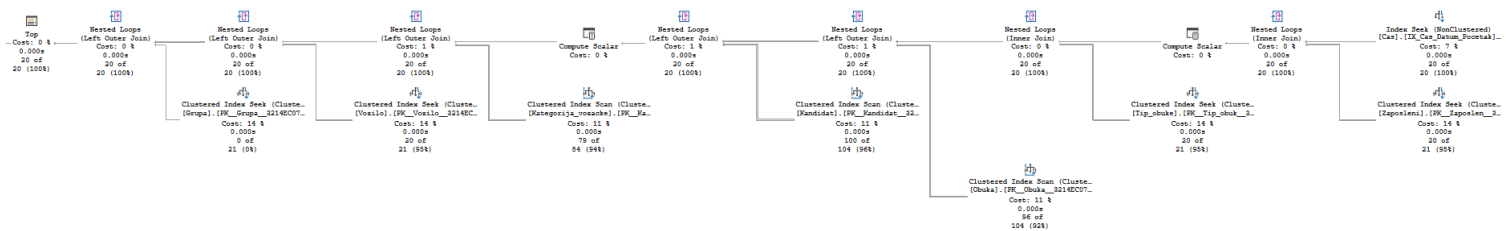
Слика 8.18 Приказ плана извршавања упита распоред часова, без индекса.

У плану извршавања види се да се поглед не извршава као посебан објекат, већ SQL Server чита табеле које улазе у његов састав. Највећи проблем је читање табеле Cas, где се јавља Clustered Index Scan, а затим и Sort оператор због редоследа по Datum и Pocetak. Зато је индекс усмерен на табелу Cas, јер се преко ње филтрира распоред по датуму, а исте колоне учествују и у сортирању резултата.

```
CREATE INDEX IX_Cas_Datum_Pocetak ON Cas (Datum, Pocetak)
INCLUDE (Kraj, Status, Lokacija, Instruktor_id, Tip_id, Obuka_id, Vozilo_id, Grupa_id);
```

Прилог 8.18 Приказ команде за креирање индекса над табелом Cas.

Команда за креирање индекса дата је у прилогу 8.18, а план после индекса на слици 8.19. Индекс IX_Cas_Datum_Pocetak креиран је над колонама Datum и Pocetak, јер оне одговарају WHERE и ORDER BY делу упита. Ако се у плану после додавања индекса не види велика промена, то значи да SQL Server за тренутну количину тест података процењује да је постојећи начин читања довољно јефтин. У реалном систему, са већим бројем часова, овакав индекс би имао више смисла.



Слика 8.19 Приказ плана извршавања упита распоред часова, са индексом.

9. СМЕРНИЦЕ ЗА УНАПРЕЂЕЊЕ ОПИСАНОГ СИСТЕМА

Описани систем представља основу за даљи развој информационог система аутошколе. У наредним фазама систем би могао да се прошири корисничком апликацијом, пријавом корисника, детаљнијим правима приступа, резервним копијама и напреднијим извештајима. Ова унапређења нису директно имплементирана у оквиру SQL скрипти рада, али представљају важне смернице за практичну примену система.

9.1. Корисничка апликација и пријава корисника

Једно од главних унапређења било би креирање корисничке апликације преко које би запослени у аутошколи радили са базом података. Уместо директног извршавања SQL скрипти у SSMS-у, корисници би податке уносили и прегледали кроз форме, меније и извештаје. На тај начин би рад са системом био једноставнији и безбеднији за кориснике који немају техничко знање о базама података.

Пријава корисника омогућила би да се зна ко приступа систему и које операције извршава. Сваки корисник би имао свој налог, а систем би могао да бележи важне активности, као што су унос новог кандидата, измена обуке, заказивање часа или евидентирање уплате.

9.2. Корисничке улоге и права приступа

У практичној примени било би потребно увести различите корисничке улоге. Администратор базе имао би потпуна права над структуром базе, администрација аутошколе би уносила и мењала податке о кандидатима, обукама, часовима и уплатама, док би инструктори имали ограничен приступ распореду и подацима о часовима које воде.

Оваква подела права смањује ризик од случајне или неовлашћене измене података. Сваки корисник би имао само она овлашћења која су му потребна за обављање посла, што је посебно важно јер база садржи личне податке кандидата и запослених.

9.3. Заштита личних података

Поред контроле приступа, било би корисно увести и јасна правила о томе ко може да прегледа, мења или брише осетљиве податке. На пример, инструкторима би могао бити доступан распоред часова и основни подаци о кандидату, док би ЈМБГ и подаци о документима били доступни само администрацији.

9.4. Резервне копије и опоравак базе

За реалну употребу система потребно је дефинисати начин прављења резервних копија базе података. Без резервних копија, квар сервера, грешка при раду са подацима или случајно брисање записа могли би довести до трајног губитка података о кандидатима, обукама, уплатама и полагањима.

Поред самог прављења резервних копија, потребно је дефинисати и поступак опоравка базе у случају губитка података. То подразумева одређивање колико често се резервна копија прави, где се чува, ко је одговоран за његову проверу и како се база враћа у исправно стање након проблема.

9.5. Додатни извештаји

Систем би могао да се прошири додатним извештајима који би помогли управити аутошколе да боље прати пословање. Поред основних приказа који су реализовани кроз погледе и упите, могуће је додати месечни преглед прихода, број одржаних часова по инструктору, искоришћеност возила и успешност кандидата на полагањима.

Такви извештаји би омогућили да се подаци из базе користе не само за евиденцију, већ и за доношење пословних одлука. На пример, анализа оптерећења инструктора могла би да помогне при распоређивању часова, док би преглед прихода омогућио бољу финансијску контролу.

9.6. Проширење модела базе података

Модел базе података могао би да се прошири додатним табелама које би покриле делове пословања који нису били обухваћени овим радом. Један пример је евиденција трошкова возила, као што су гориво, сервисирање, регистрација и осигурање. Посебна табела за гориво могла би да чува податке о возилу, датуму сипања, количини горива, цени и пређеној километражи.

Поред тога, могла би се додати табела за сервисирање возила, како би се пратили датуми сервиса, врста интервенције, трошак и напомена. Овакво проширење би омогућило бољу контролу возног парка и прецизније праћење трошкова аутошколе.

10. ЗАКЉУЧАК

У овом раду је пословање аутошколе посматрано као скуп повезаних процеса који захтевају уређену и поуздану евиденцију података. Уместо да се подаци о кандидатима, обукама, часовима, возилима, полагањима и уплатама воде одвојено, база је пројектована тако да ти подаци чине једну целину. Таквим повезивањем смањује се могућност дуплирања података и олакшава праћење кандидата од уписа до завршетка обуке.

Током израде рада показано је да добро пројектована база података не служи само за чување информација, већ и за спровођење правила која су важна у свакодневном раду аутошколе. Ограничења, везе између табела, процедуре и тригери омогућавају да се велики део провере података извршава на нивоу саме базе. То је посебно значајно код података који морају бити тачни и међусобно усклађени, као што су активне обуке, заказани часови, резултати полагања и уплате.

Посебну вредност рада представља то што база није посматрана само као статична структура, већ и као систем који треба да подржи стварне упите и извештаје. Анализа планова извршавања показала је да избор индекса треба да буде заснован на начину на који се подаци заиста читају, повезују и сортирају. На тај начин оптимизација постаје део разумевања система, а не само технички додатак на крају имплементације.

Иако реализовано решење представља функционалну основу, оно оставља простор за даљи развој. Најважнија унапређења односила би се на израду корисничке апликације, увођење корисничких улога, бољу заштиту личних података, редовне резервне копије и додатне извештаје. Таквим проширењима база би могла да прерасте у целовит информациони систем за подршку раду аутошколе.

11. ИНДЕКС ПОЈМОВА

А

алтернативни кључ

атрибут

Б

база података

В

веза

Е

ентитет

И

идентификатор

индекс

интегритет података

К

кардиналност

кандидат

концептуални модел

корисничка улога

Л

логички модел

М

модел базе података

Н

нормализација

О

обука

ограничење

П

поглед

примарни кључ

процедура

Р

релација

релациони модел

референцијални интегритет

резервна копија

С

скаларна функција

страни кључ

Т

табела

табеларна функција

транзакција

тригер

У

упит

Ф

физички модел

12. ЛИТЕРАТУРА

- [1] Висока техничка школа струковних студија Суботица, Пројектовање база података, скрипта из предмета. Доступно на: <https://people.vts.su.ac.rs/~peti/Baze%20podataka/Literatura/Skripta%20iz%20predmeta.%20Projektovanje%20baza%20podataka.pdf> (приступљено: 24. 8. 2026).
- [2] Microsoft Learn, SQL Server technical documentation. Доступно на: <https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver17> (приступљено: 24. 8. 2026).
- [3] Microsoft Learn, Transact-SQL reference. Доступно на: <https://learn.microsoft.com/en-us/sql/t-sql/language-reference?view=sql-server-ver17> (приступљено: 24. 8. 2026).
- [4] Microsoft Learn, SQL Server Management Studio (SSMS). Доступно на: <https://learn.microsoft.com/en-us/ssms/sql-server-management-studio-ssms> (приступљено: 24. 8. 2026).
- [5] SAP Help Portal, PowerDesigner Documentation. Доступно на: https://help.sap.com/docs/SAP_POWERDESIGNER/abd3434b4987485c92057ab9392aadbe/c7da64d26e1b1014babfeec490561c31.html (приступљено: 24. 8. 2026).
- [6] Габријела Димић и Милош Пејановић, Релационе базе података, прво издање, Академија техничко-уметничких струковних студија Београд, Београд, 2024. Доступно на: https://www.viser.edu.rs/uploads/2025/01/Relacione_baze_podataka_udzbenik.pdf (приступљено: 24. 8. 2026).
- [7] Математички факултет, Универзитет у Београду, Релационе базе података, скрипта за школску 2023/2024. годину. Доступно на: https://www.bazepodataka.matf.bg.ac.rs/rbp/2024/rbp_skripta_2024.pdf (приступљено: 24. 8. 2026).
- [8] Електротехнички факултет, Универзитет у Београду, Нормализација, материјали за вежбе из предмета Базе података 1. Доступно на: <https://rti.etf.bg.ac.rs/rti/bp1/materijali/vezbe/2.Normalizacija.pdf> (приступљено: 24. 8. 2026).
- [9] Електротехнички факултет, Универзитет у Београду, Индекси, материјали из предмета Софтверски алати база података. Доступно на: <https://sab.etf.bg.ac.rs/Nastava/Vezbe/2%20-%20Fizicko%20projektovanje/2.3%20-%20Indeksi.pdf> (приступљено: 24. 8. 2026).

13. СПИСАК СЛИКА

Слика 3.1 T-SQL логотип.	4
Слика 3.2 MS SQL Server 2025 логотип.	4
Слика 3.3 SQL Server Management Studio (SSMS).	5
Слика 3.4 SAP PowerDesigner логотип.	5
Слика 6.1 Концептуални модел базе података аутошколе.	15
Слика 6.2 Логички модел базе података аутошколе.	16
Слика 6.3 Физички модел базе података аутошколе.	18
Слика 7.1 Приказ релације Zaposleni - Zaposleni_Funkcija.	30
Слика 7.2 Приказ релације Funkcije_Zaposlenih - Zaposleni_Funkcija.	30
Слика 7.3 Приказ релације Zaposleni - Zaposleni_Izostanak.	31
Слика 7.4 Приказ релације Zaposleni - Nadzornici_polaganja.	31
Слика 7.5 Приказ релације Polaganje - Nadzornici_polaganja.	32
Слика 7.6 Приказ релације Kandidat – Obuka.	32
Слика 7.7 Приказ релације Kategorija_vozacke – Obuka.	33
Слика 7.8 Приказ релације Tip_obuke – Obuka.	33
Слика 7.9 Приказ релације Zaposleni – Obuka.	34
Слика 7.10 Приказ релације Grupa - Kandidat_grupa.	34
Слика 7.11 Приказ релације Obuka – Kandidat_grupa.	35
Слика 7.12 Приказ релације Kategorija_vozacke – Vozilo.	35
Слика 7.13 Приказ релације Zaposleni – Cas.	36
Слика 7.14 Приказ релације Tip_obuke – Cas.	36
Слика 7.15 Приказ релације Obuka – Cas.	37
Слика 7.16 Приказ релације Vozilo – Cas.	37
Слика 7.17 Приказ релације Grupa – Cas.	38
Слика 7.18 Приказ релације Tip_obuke – Polaganje.	38
Слика 7.19 Приказ релације Polaganje - Polaganje_kandidat.	39
Слика 7.20 Приказ релације Obuka - Polaganje_kandidat.	39
Слика 7.21 Приказ релације Cenovnik – Uplata.	40
Слика 7.22 Приказ релације Obuka – Uplata.	40
Слика 7.23 Приказ резултата погледа Kandidati_Obuke.	41
Слика 7.24 Приказ резултата погледа Raspored_Casova.	42
Слика 7.25 Приказ резултата погледа Uplate_Po_Obuci.	43
Слика 7.26 Приказ резултата погледа Rezultati_Polaganja.	44
Слика 7.27 Приказ резултата погледа Zaposleni_Funkcije.	45
Слика 7.28 Приказ резултата погледа Stanje_Obuke.	46
Слика 7.29 Резултат функције UkupnoUplacenoZaObuku.	46
Слика 7.30 Резултат функције BrojCasovaZaObuku са параметром Samo_odrzani.	47
Слика 7.31 Резултат функције BrojCasovaZaObuku без параметра Samo_odrzani.	47
Слика 7.32 Резултат функције BrojPolaganjaZaObuku.	48
Слика 7.33 Резултат функције KandidatImaAktivnuObuku.	49
Слика 7.34 Резултат функције RasporedInstruktora.	50
Слика 8.1 Укључење плана извршавања.	58
Слика 8.2 Приказ плана извршавања упита возило по датуму, без индекса.	59
Слика 8.3 Приказ плана извршавања упита возило по датуму, са индексом.	60
Слика 8.4 Приказ плана извршавања упита часови по кандидату, без индекса.	61
Слика 8.5 Приказ плана извршавања упита часови по кандидату, са индексом.	61
Слика 8.6 Приказ плана извршавања упита обука по кандидату и категорији, без индекса.	62
Слика 8.7 Приказ плана извршавања упита обука по кандидату и категорији, са индексом.	63
Слика 8.8 Приказ плана извршавања упита уплате по обуци, без индекса.	64
Слика 8.9 Приказ плана извршавања упита уплате по обуци, са индексом.	64
Слика 8.10 Приказ плана извршавања упита уплате по датуму, без индекса.	65
Слика 8.11 Приказ плана извршавања упита уплате по датуму, са индексом.	65
Слика 8.12 Приказ плана извршавања упита полагање по кандидату, без индекса.	66
Слика 8.13 Приказ плана извршавања упита полагање по кандидату, са индексом.	67
Слика 8.14 Приказ плана извршавања упита кандидати по групи, без индекса.	68
Слика 8.15 Приказ плана извршавања упита кандидати по групи, са индексом.	68
Слика 8.16 Приказ плана извршавања упита запослени по функцији, без индекса.	69

<i>Слика 8.17 Приказ плана извршавања упита запослени по функцији, са индексом.</i>	<i>70</i>
<i>Слика 8.18 Приказ плана извршавања упита распоред часова, без индекса.</i>	<i>70</i>
<i>Слика 8.19 Приказ плана извршавања упита распоред часова, са индексом.</i>	<i>71</i>

14. СПИСАК ПРИЛОГА

Прилог 7.1 Креирање и избор базе података.	20
Прилог 7.2 Дизајн табеле Kandidat.	21
Прилог 7.3 Дизајн табеле Zaposleni.	22
Прилог 7.4 Дизајн табеле Funkcije_Zaposlenih.	23
Прилог 7.5 Дизајн табеле Zaposleni_Funkcija.	23
Прилог 7.6 Дизајн табеле Zaposleni_Izostanak.	23
Прилог 7.7 Дизајн табеле Kategorija_vozacke.	24
Прилог 7.8 Дизајн табеле Tip_obuke.	24
Прилог 7.9 Дизајн табеле Grupa.	24
Прилог 7.10 Дизајн табеле Obuka.	25
Прилог 7.11 Дизајн табеле Kandidat_grupa.	25
Прилог 7.12 Дизајн табеле Vozilo.	26
Прилог 7.13 Дизајн табеле Polaganje.	26
Прилог 7.14 Дизајн табеле Polaganje_kandidat.	27
Прилог 7.15 Дизајн табеле Nadzornici_polaganja.	27
Прилог 7.16 Дизајн табеле Cas.	28
Прилог 7.17 Дизајн табеле Cenovnik.	29
Прилог 7.18 Дизајн табеле Uplata.	29
Прилог 7.19 Дефиниција погледа Kandidati_Obuke.	41
Прилог 7.20 Дефиниција погледа Raspored_Casova.	42
Прилог 7.21 Дефиниција погледа Uplate_Po_Obuci.	43
Прилог 7.22 Дефиниција погледа Rezultati_Polaganja.	44
Прилог 7.23 Дефиниција погледа Zaposleni_Funkcije.	45
Прилог 7.24 Дефиниција погледа Stanje_Obuke.	46
Прилог 7.25 Дефиниција функције UkupnoUplacenoZaObuku.	46
Прилог 7.26 Наредба за позивање функције UkupnoUplacenoZaObuku.	46
Прилог 7.27 Дефиниција функције BrojCasovaZaObuku.	47
Прилог 7.28 Наредба за позивање функције BrojCasovaZaObuku са параметром Samo_odrzani.	47
Прилог 7.29 Наредба за позивање функције BrojCasovaZaObuku без параметра Samo_odrzani.	47
Прилог 7.30 Дефиниција функције BrojPolaganjaZaObuku.	48
Прилог 7.31 Наредба за позивање функције BrojPolaganjaZaObuku.	48
Прилог 7.32 Дефиниција функције KandidatImaAktivnuObuku.	48
Прилог 7.33 Наредба за позивање функције KandidatImaAktivnuObuku.	49
Прилог 7.34 Дефиниција табеларне функције RasporedInstruktora.	49
Прилог 7.35 Наредба за позивање табеларне функције RasporedInstruktora.	50
Прилог 7.36 Дефиниција процедуре UpisKandidataNaObuku.	51
Прилог 7.37 Дефиниција процедуре ZakaziPrakticniCas.	52
Прилог 7.38 Дефиниција процедуре EvidentirajUplatu.	53
Прилог 7.39 Дефиниција процедуре PrijaviKandidataNaPolaganje.	54
Прилог 7.40 Дефиниција процедуре EvidentirajRezultatPolaganja.	55
Прилог 7.41 Дефиниција тригера Kandidat_SetIzmenjenDatum.	55
Прилог 7.42 Дефиниција тригера Uplata_ZabraniBrisanje.	56
Прилог 7.43 Дефиниција тригера Vozilo_Kilometraza_NeSmanjuje.	56
Прилог 7.44 Дефиниција тригера Cas_Odrzan_PovecajKilometrazu.	56
Прилог 7.45 Дефиниција тригера PolaganjeKandidat_Uspesno_ZavrshiObuku.	57
Прилог 7.46 Дефиниција тригера Obuka_PromenaInstruktora_PrebaciBudućeCasove.	57
Прилог 8.1 Приказ упита возило по датуму.	59
Прилог 8.2 Команда за креирање индекса над табелом Cas.	60
Прилог 8.3 Приказ упита часови по кандидату.	61
Прилог 8.4 Приказ команде за креирање индекса над табелом Cas.	61
Прилог 8.5 Приказ упита обука по кандидату и категорији.	62
Прилог 8.6 Приказ команде за креирање индекса над табелом Obuka.	62
Прилог 8.7 Приказ упита уплате по обуци.	63
Прилог 8.8 Приказ команде за креирање индекса над табелом Uplata.	64
Прилог 8.9 Приказ упита уплате по датуму.	65
Прилог 8.10 Приказ команде за креирање индекса над табелом Uplata.	65
Прилог 8.11 Приказ упита полагање по кандидату.	66

Прилог 8.12 Приказ команде за креирање индекса над табелом Polaganje_kandidat.	66
Прилог 8.13 Приказ упита кандидати по групи.	67
Прилог 8.14 Приказ команде за креирање индекса над табелом Kandidat_grupa.	68
Прилог 8.15 Приказ упита запослени по функцији.	69
Прилог 8.16 Приказ команде за креирање индекса над табелом Zaposleni_Funkcija.	70
Прилог 8.17 Приказ упита распоред часова.	70
Прилог 8.18 Приказ команде за креирање индекса над табелом Cas.	71

15. ИЗЈАВА О АКАДЕМСКОЈ ЧЕСТИТОСТИ

ИЗЈАВА О АКАДЕМСКОЈ ЧЕСТИТОСТИ

Студент (име, име
једног родитеља и презиме):

Алекса, Ненад, Стојановић

Број индекса:

НРТ – 4/22

Под пуном моралном, материјалном, дисциплинском и кривичном одговорношћу изјављујем да је завршни рад, под насловом:

Пројектовање и реализација релационог модела базе података за управљање и анализу пословања Ауто школе “AS” у MS SQL сервер систему

1. резултат сопственог истраживачког рада;
2. да овај рад, ни у целини, нити у деловима, нисам пријављивао на другим високошколским установама;
3. да нисам повредио ауторска права, нити злоупотребио интелектуалну својину других лица;
4. да сам рад и мишљења других аутора које сам користио у овом раду назначио или цитирао у складу са Упутством;
5. да су сви радови и мишљења других аутора наведени у списку литературе/референци који је саставни део овог рада, пописани у складу са Упутством;
6. да сам свестан да је плагијат коришћење туђих радова у било ком облику (као цитата, парафраза, слика, табела, дијаграма, дизајна, планова, фотографија, филма, музике, формула, веб-сајтова, компјутерских програма и сл.) без навођења аутора или представљање туђих ауторских дела као мојих, кажњиво по закону (Закон о ауторском и сродним правима), као и других закона и одговарајућих аката Академије техничко-уметничких струковних студија Београд;
7. да је електронска верзија овог рада идентична штампаном примерку овог рада и да пристајем на његово објављивање под условима прописаним актима Академије техничко-уметничких струковних студија Београд;
8. да сам свестан последица уколико се докаже да је овај рад плагијат.

У Београду, __. __. 2026. године

Својеручни потпис студента
